LECTURE NOTES · HARDWARE, 2026

Introduction to GPU Architectures

Companion notes to the slides

How graphics chips became throughput processors, from the fixed-function pipeline of the 1980s and 1990s through programmable shaders and GPGPU to the unified G80 design and CUDA (2006–07), the SIMT execution model and latency hiding by multithreading, the energy argument behind the design, the memory system and the roofline, the Tesla, Fermi, Kepler and Maxwell generations, Pascal (16 nm, HBM2, NVLink, FP16) and Volta (tensor cores, independent thread scheduling), the systems built from them (DGX-1, NVSwitch, Titan, Piz Daint, Summit), the alternatives that lost the decade, and what carried forward. One section per slide, every term defined, and a reading list for each part.

For advanced undergraduates and beginning graduate students

November 2026

These notes follow the deck *Introduction to GPU Architectures* (31 slides) slide by slide. Numbers quoted from a paper or an NVIDIA whitepaper were checked against that document and the version is named where versions differ; numbers from product datasheets, press releases, vendor pages and press coverage are marked †; numbers marked "derived" were computed from quoted ones and the formulas are given; curves the slides label "illustrative" are sketches of a mechanism, not data. The history ends with the Volta generation (2017–18); later architectures are named only where a thread continues.

Contents

How to use these notes **3**

1 Overview (slides 1–4) **3**
1.1 Slide 1: Title: Introduction to GPU Architectures . . . 3
1.2 Slide 2: A GPU is a throughput processor . . . 4
1.3 Slide 3: Three numbers: FLOP/s, bytes per second, watts . . . 6
1.4 Slide 4: A timeline: from the Geometry Engine to Volta . . . 8

2 The graphics pipeline, fixed and programmable (slides 5–6) **10**
2.1 Slide 5: The fixed-function graphics pipeline . . . 10
2.2 Slide 6: Programmable shaders: a processor hiding in each stage . . . 11

3 GPGPU and the unified G80 (slides 7–8) **13**
3.1 Slide 7: GPGPU: computing with pixels . . . 13
3.2 Slide 8: G80 (2006): one unified array of processors, and CUDA . . . 15

4 The CUDA model and its mapping to the chip (slides 9–10) **16**
4.1 Slide 9: The CUDA programming model: grids of blocks of threads . . . 16
4.2 Slide 10: Mapping the model to the chip: blocks to SMs, threads to warps . . . 19

5 SIMT, latency hiding and energy (slides 11–13) **21**
5.1 Slide 11: SIMT: SIMD hardware with a thread's face . . . 21
5.2 Slide 12: Latency hiding: enough threads in flight to cover the memory . . . 23
5.3 Slide 13: Energy is the real constraint: why the design is what it is . . . 25

6 The memory system and the roofline (slides 14–15) **27**
6.1 Slide 14: The memory system: wide DRAM, coalescing, small caches . . . 27
6.2 Slide 15: The roofline: which kernels a GPU can feed . . . 30

7 The generations: Tesla, Fermi, Kepler, Maxwell, Pascal (slides 16–21) **32**
7.1 Slide 16: Tesla to Fermi (2008–2010): double precision, caches, ECC . . . 32
7.2 Slide 17: Kepler and Maxwell (2012–2015): wider SMs, then efficiency . . . 34
7.3 Slide 18: The generations in one table . . . 36
7.4 Slide 19: Deep learning arrives (2012–2016) and the architecture answers . . . 38
7.5 Slide 20: Pascal GP100 (2016): 16 nm FinFET, HBM2, NVLink . . . 40
7.6 Slide 21: Reading a Pascal or Volta SM: four schedulers, many lanes . . . 42

8 Volta and the tensor core (slides 22–24) **43**
8.1 Slide 22: Volta GV100 (2017): the largest die, and a new kind of unit . . . 43
8.2 Slide 23: Tensor cores: a $4\times4\times4$ matrix multiply-add per clock . . . 45
8.3 Slide 24: Independent threads, and what microbenchmarks measure . . . 48

9 Systems: interconnect, supercomputers, alternatives (slides 25–27) **50**
9.1 Slide 25: Beyond PCIe: NVLink, the DGX-1 cube, NVSwitch . . . 50
9.2 Slide 26: GPUs in supercomputers: Tianhe-1A to Summit . . . 52
9.3 Slide 27: The alternatives, and why they lost the decade . . . 54

10 What carried forward, practice, summary, sources (slides 28–31) 56
10.1 Slide 28: What stayed constant for a decade, and what changed 57
10.2 Slide 29: In practice: reading a spec sheet, estimating, measuring 58
10.3 Slide 30: Summary: what to take from this lecture 60
10.4 Slide 31: Sources 61

A Notation 63

B Formulas at a glance 64

C Exercises 66

D Reading list 68

Index of terms 78

How to use these notes

Each section of these notes corresponds to one section of the deck, and each subsection to one slide. A subsection has four parts:

- **What the slide shows**: a description of the figure or table, so the notes can be read without the slides.
- **Details**: the material the slide compresses, with the arithmetic written out and the numbers given their sources.
- A **Terms** box defining every technical term the slide uses, the first time it appears. All terms are collected in the index at the end.
- An **Intuition** box: the picture to keep in mind, and the question to ask when the picture seems to fail.

Conventions. A *GPU* here is a discrete graphics processor used for computing: NVIDIA's compute dies from G80 (2006) to GV100 (2017), with AMD and Intel parts for comparison. An *SM* is a streaming multiprocessor, the unit that holds a block's threads; a *lane* (NVIDIA: "CUDA core") is one FP32 arithmetic pipeline in it; a *warp* is a group of 32 threads scheduled together. Peak FLOP/s counts a fused multiply-add as two floating-point operations. Bandwidths are in bytes per second with decimal prefixes (1 GB/s $= 10^9$ B/s); memory capacities use binary prefixes where the vendor does. A dagger (†) marks any number whose source is a datasheet, product page, press release, benchmark list or press coverage rather than an architecture paper or whitepaper.

A reading plan for a single lecture: Sections 1–3 (what a throughput processor is, the graphics pipeline, GPGPU and G80) take about a quarter of the session; Sections 4–6 (CUDA, SIMT and latency hiding, energy, memory and roofline) a third; Sections 7–8 (the generations, Pascal, Volta) a quarter; Sections 9–10 (systems, alternatives, practice) the rest. The exercises in Appendix C are arithmetic on the deck's own numbers and can be set as homework. Appendix D is the reading list, section by section, and matches the paper kit that accompanies these notes.

1 Overview (slides 1–4)

1.1 Slide 1: Title: Introduction to GPU Architectures

How graphics chips became throughput processors: the fixed-function pipeline, programmable shaders and GPGPU, the unified G80 design and CUDA, SIMT execution and latency hiding, the memory system and the roofline, the Tesla to Volta generations, HBM2, NVLink and tensor cores, and the systems built from them.

What the slide shows The title slide carries one picture on its right: a stylized die floorplan. Forty-eight small orange rectangles in six rows, shaded from light to dark, stand for the streaming multiprocessors; a single green band below them is the L2 cache shared by all of them; four violet blocks along the bottom edge are stacks of HBM2 memory. The caption reads: 80 streaming multiprocessors, one shared L2 cache, four stacks of HBM2: the floorplan of a throughput processor (Volta, 2017). The subtitle lists the lecture's route, which is also the order of the deck's ten footer stages: Overview, Graphics, GPGPU, CUDA, SIMT, Memory, Generations, Volta, Systems and Wrap-up.

Details The floorplan is the lecture's thesis in one image. Almost all of the die is a repeated unit, the streaming multiprocessor (SM), each a bundle of arithmetic lanes, registers and a small local memory; the structures a CPU die spends most of its area on (large caches, branch predictors, out-of-order machinery) are either absent or small and shared. The memory is not a few chips on a board but four stacks of DRAM dies bonded onto the same silicon interposer as the processor, connected by a 4,096-bit bus. The real Volta

GV100 die is drawn with 84 SMs, of which 80 are enabled on the Tesla V100 product; the picture simplifies this to 48 blocks, and the counts in the caption are the product's, not the picture's.

The scope of the lecture is the history of the GPU as a computing device from its origins in graphics hardware to the Volta generation of 2017–18. That endpoint is chosen deliberately. By Volta the architecture had acquired every feature that the later generations varied rather than invented: a unified array of scalar processors (G80, 2006), a C programming model (CUDA, 2007), double precision and error-correcting memory (GT200 and Fermi, 2008–10), stacked memory and a dedicated inter-GPU link (Pascal, 2016), and a specialized matrix unit (Volta, 2017). The lecture also covers the systems these chips were built into (DGX-1, NVSwitch, Titan, Piz Daint, Summit) and the competing designs from AMD and Intel that did not win the decade.

Two conventions run through the deck and these notes. A dagger (†) marks any number whose source is a product datasheet, a press release, a vendor web page, a benchmark list or press coverage, rather than an architecture paper or an NVIDIA architecture whitepaper. "Derived" marks numbers computed from quoted ones, and the formula is given. Plots labeled "illustrative" are sketches of a mechanism drawn to the shape the literature reports, not data from one experiment.

Terms

GPU Graphics processing unit: originally a chip that implemented the 3D graphics pipeline, and from 2006 onward also a general-purpose throughput processor programmed through CUDA or OpenCL. The subject of the lecture.

streaming multiprocessor The SM: NVIDIA's name for the repeated processing unit of a GPU, holding a set of arithmetic lanes, a register file, a scheduler and a block of on-chip memory. A block of CUDA threads runs entirely on one SM.

throughput processor A processor designed to maximize the total rate of work completed by many independent tasks, at the expense of the time any one task takes; the opposite design point from a latency-oriented CPU core.

HBM2 High Bandwidth Memory, second generation: DRAM dies stacked vertically and connected to the processor through a silicon interposer with a 1,024-bit interface per stack; used on Pascal and Volta.

Volta NVIDIA's 2017 GPU architecture (the GV100 die, the Tesla V100 product), the last generation covered by the lecture.

dagger The † mark used throughout for numbers from datasheets, press releases, vendor pages, benchmark lists or press rather than papers and whitepapers.

Intuition

Look at the floorplan and count what repeats: eighty copies of the same small processor, one cache, four memory stacks. The design question for the whole decade was what to put inside the repeated block and how to feed it, and almost nothing else.

1.2 Slide 2: A GPU is a throughput processor

Spend the transistors on arithmetic and on threads in flight, not on making one thread fast.

What the slide shows Two cartoons of die area on the left. The upper one is a CPU core: a wide violet block for control (out-of-order execution, branch prediction, prefetching), a narrow orange block for the arithmetic units, a green block for L1 cache and registers, and a large green band below for megabytes of

L2 and L3 cache. The lower one is a GPU: eight tall orange columns of arithmetic lanes, each topped by a thin strip of control and footed by a strip of register file and shared memory, over a modest band of L2 cache shared by all. Two stat tiles on the right give the scale on a V100: 5,120 FP32 lanes and 900 GB/s of HBM2 bandwidth. A panel, “The contract”, states three rules the design imposes on the program: many threads rather than fast threads, regular work, and bandwidth as the budget, with the note that a host CPU launches the work.

Details The two cartoons are drawn to the same width and the same proportions on purpose: a CPU core and a GPU SM are built from the same kind of transistors on the same kind of process, and the difference is entirely in where the area goes. A CPU core devotes most of its area to making one instruction stream run fast: reordering instructions, predicting branches, prefetching data, and keeping a large cache so that the one stream rarely waits. A GPU devotes most of its area to arithmetic lanes and to the registers that hold the state of many threads at once, and accepts that any one thread will wait often. The design works only if there are enough other threads ready to run while it waits, which is the first rule of the contract.

The numbers on the tiles are the Tesla V100’s (Volta, 2017). It has 80 SMs with 64 FP32 lanes each, 5,120 in all, and each SM can hold up to 2,048 resident threads, so the chip can keep $80 \times 2{,}048 = 163{,}840$ threads in flight at once (derived from the whitepaper’s per-SM limit). The hardware does not switch between these threads individually: it groups them into warps of 32 and issues one instruction per warp, so that a single instruction fetch and decode is amortized over 32 data items. This is the second rule, regular work: the 32 threads of a warp execute the same instruction at the same time, and a branch that sends them down different paths is executed once per path with the other threads masked off (slide 11). The 900 GB/s on the second tile is the memory bandwidth of the four HBM2 stacks, and the third rule follows from it: with 15.7 TFLOP/s of FP32 arithmetic and 900 GB/s of memory, the chip can perform about 17 floating-point operations for every byte it reads from memory, so a kernel that does less arithmetic per byte than that is limited by the memory, not the lanes (slide 15).

The panel’s last clause, that a host CPU launches the kernels, states what kind of device a GPU of this era is. Through Volta, a GPU is an accelerator attached to a CPU over PCIe or NVLink, with its own memory; a program running on the CPU copies data to the GPU’s memory, launches a kernel (a function run by many threads) and copies results back, or, from Pascal onward, lets a unified virtual address space migrate pages on demand (slide 20). The GPU does not run an operating system, and it does not fetch its own work.

What the slide does not claim is that a GPU is faster than a CPU. It is faster on work that fits the contract: a very large number of independent, identical operations with regular memory access, enough of them to hide memory latency, and enough arithmetic per byte to use the lanes. On work that does not fit, a single-threaded CPU core with its large cache and speculative execution wins, and slide 27 returns to how much the difference is on work that fits.

Terms

latency The time from issuing an operation to receiving its result; a CPU core is built to minimize it for one instruction stream.

throughput The number of operations completed per unit time across all threads; a GPU is built to maximize it.

lane One arithmetic pipeline that executes one thread’s instruction per clock; NVIDIA calls an FP32 lane a “CUDA core”. A V100 SM has 64 of them.

CUDA core NVIDIA’s marketing name for a single FP32 arithmetic lane; the counts quoted for GPUs (128 on G80, 5,120 on V100) are lane counts.

warp A group of 32 threads that the SM schedules as a unit, issuing one instruction for all of them at

once.

resident thread A thread whose state (registers, program counter) is held on the SM so that it can be scheduled; a V100 SM holds up to 2,048 of them.

kernel A function written to be executed by many GPU threads in parallel, launched from a program on the host CPU.

host The CPU and its memory, from which a GPU program is controlled; the GPU is the device.

SIMT Single instruction, multiple threads: NVIDIA's execution model in which one instruction is issued for a warp of 32 threads, each thread with its own registers and addresses. Slide 11.

roofline A performance model bounding a kernel's attainable rate by the smaller of the peak arithmetic rate and the memory bandwidth times the kernel's arithmetic per byte. Slide 15.

Intuition

A CPU core is one very good clerk with a large desk; a GPU is a hall of a hundred thousand clerks with tiny desks who all work from the same instruction sheet. The hall is faster only if the job can be cut into a hundred thousand identical pieces and the filing cabinet can be reached quickly enough.

1.3 Slide 3: Three numbers: FLOP/s, bytes per second, watts

Peak arithmetic grew about 45× from G80 to V100 while board power doubled; memory bandwidth grew about 10×.

What the slide shows A plot on the left with the year on the horizontal axis (2006 to 2018) and vendor-quoted peak GFLOP/s on a logarithmic vertical axis from 100 to 100,000, for NVIDIA's compute GPUs. A solid orange line with dots traces FP32 peak through seven points: G80 (2006, 346), Tesla C1060 (2008, 933), Fermi C2050 (2010, 1,030), K20X (late 2012, 3,950), M40 (late 2015, 6,800), P100 (2016, 10,600) and V100 (2017, 15,700). A dashed line traces FP64 from the C1060's 78 through 515 (C2050), 1,310 (K20X), 5,300 (P100) and 7,800 (V100). A dotted line in the accent color marks the two half-precision points: the P100's FP16 at 21,200 and the V100's tensor-core rate at 125,000. All points carry a dagger because they are vendor peak numbers. Two stat tiles on the right give memory bandwidth, 86 to 900 GB/s, and board power, 155 to 300 W, from G80 to V100. A panel, "What the curves say", makes three points: arithmetic scaled with transistors, bandwidth lagged, and precision became a dial.

Details The vertical axis is a logarithm, so the roughly straight FP32 line is exponential growth: from 346 GFLOP/s on the GeForce 8800 GTX of November 2006 to 15,700 on the Tesla V100 of 2017, a factor of about 45 in eleven years, or about 40% per year. The G80 figure counts only the multiply-add instruction (128 lanes $\times$ 2 FLOP $\times$ 1.35 GHz $=$ 346 GFLOP/s); NVIDIA's own marketing counted a second multiply the lanes could dual-issue and quoted "up to 518 GFLOPS" for the Tesla C870†, and the C1060's 933 GFLOP/s likewise includes the extra multiply (622 counting the multiply-add alone)†. From Fermi onward the lanes perform only fused multiply-adds and the peak is simply lanes $\times$ 2 $\times$ clock. The growth came almost entirely from transistor count, 681 million on G80 to 21.1 billion on GV100, spent on more lanes: clock speeds moved only from 1.35 GHz to 1.53 GHz, and board power only from 155 W to 300 W. The 155 W is NVIDIA's later official "maximum board power" for the 8800 GTX; at launch the company told reviewers about 185 W†, so the growth in board power from G80 to V100 is between 1.6× and 2× depending on which figure is used.

The panel's second point is the one that shapes the rest of the deck. Bandwidth grew from 86.4 GB/s (G80's 384-bit GDDR3 at 900 MHz) to 900 GB/s (V100's four HBM2 stacks), a factor of about ten, against a factor of 45 in arithmetic. Dividing one by the other gives bytes per FLOP: $86.4/346 = 0.25$ bytes for every

floating-point operation on G80, $900/15{,}700 = 0.057$ on V100 (derived). Every generation therefore has less memory bandwidth per unit of arithmetic than the last, and a kernel that balanced the two in 2006 is memory-bound by a factor of four in 2017. This is why the slides on the memory system and the roofline (14 and 15) matter, and why the design kept adding on-chip storage (caches, larger register files, shared memory) and a new memory technology (HBM2) to move the ratio back.

The third point is the two dotted points. Double precision appeared with GT200 in 2008 at a twelfth of the single-precision rate, because scientific computing needed it; Fermi raised it to half rate in 2010. Half precision appeared with Pascal in 2016 at twice the single-precision rate, and the tensor cores of Volta in 2017 performed half-precision matrix multiply-adds at eight times the single-precision rate, because deep learning needed throughput more than precision. The GPU's arithmetic therefore became a set of rates, one per precision, that the programmer chooses among (slides 19 and 23).

The plot does not show two things. It omits the consumer GeForce parts, whose FP32 rates are similar but whose FP64 rates were deliberately cut from Kepler onward (slide 16). And it shows peak, not attained, rates; slide 15 is about the gap between the two.

Peak arithmetic (derived): FLOP/s $=$ SMs $\times$ lanes per SM $\times\, 2 \times f$, with 2 for a fused multiply-add. G80: $16 \times 8 \times 2 \times 1.35$ GHz $= 346$ GFLOP/s. V100: $80 \times 64 \times 2 \times 1.53$ GHz $= 15{,}667$ GFLOP/s, quoted as 15.7 TFLOP/s. Bytes per FLOP (derived): bandwidth / peak FLOP/s; G80 $86.4/346 = 0.25$, V100 $900/15{,}700 = 0.057$. Memory bandwidth from the bus: width in bits $\times$ data rate $/8$; G80: 384×1.8 Gb/s$/8 = 86.4$ GB/s.

Terms

FLOP/s Floating-point operations per second; a fused multiply-add counts as two. GFLOP/s is 10^9, TFLOP/s 10^{12} per second.

peak The arithmetic rate if every lane completes one operation every clock; a vendor-quoted upper bound that real kernels approach only when they are compute-bound and well tuned.

FP32 IEEE single-precision floating point, 32 bits; the native precision of the graphics pipeline and of a GPU's main lanes.

FP64 IEEE double-precision floating point, 64 bits; the precision most scientific computing requires, provided by separate units at a lower rate.

FP16 IEEE half-precision floating point, 16 bits with a 10-bit mantissa; adequate for many deep-learning computations, and the input format of Volta's tensor cores.

memory bandwidth The rate at which data can move between the GPU's DRAM and the chip, in bytes per second; 86.4 GB/s on G80, 900 GB/s on V100.

bytes per FLOP Memory bandwidth divided by peak arithmetic rate; the amount of memory traffic the chip can supply per operation, which fell from 0.25 to 0.057 over the decade.

compute-bound A kernel whose running time is set by the arithmetic rate rather than by memory bandwidth.

board power The maximum electrical power a GPU board is designed to draw, in watts; also called TDP. 155 W (official) for the 8800 GTX, 300 W for the V100 SXM2.

tensor core A unit in Volta and later SMs that performs a small matrix multiply-add per clock on half-precision inputs with single-precision accumulation. Slide 23.

Intuition

Arithmetic got cheap faster than memory got fast, by a factor of four over the decade. Every slide that follows about caches, registers, shared memory and stacked DRAM is a response to that one widening gap.

1.4 Slide 4: A timeline: from the Geometry Engine to Volta

Thirty-five years in four moves: fixed function, programmable shaders, unified compute, specialized compute.

What the slide shows A horizontal timeline with eight dots, labeled by period and alternating above and below the line: 1982–93, Clark's Geometry Engine and SGI's RealityEngine, the pipeline in hardware; 1996–99, 3dfx's Voodoo and NVIDIA's GeForce 256, sold as "the first GPU"; 2001–04, programmable shaders (GeForce 3, Radeon 9700), the shading languages Cg, HLSL and GLSL, and Brook and GPGPU; 2006–07, unified shaders (Xenos, G80), CUDA, the Tesla products, OpenCL; 2008–10, FP64 on GT200, caches, ECC and FMA on Fermi, Tianhe-1A at the top of the TOP500; 2012–14, Kepler and Titan, AlexNet on two GTX 580s, cuDNN, Maxwell's efficiency turn; 2016, Pascal P100 with 16 nm FinFET, HBM2, NVLink and FP16, the DGX-1 and Piz Daint; and 2017–18 in the accent color, Volta V100 with tensor cores, NVLink 2 and independent thread scheduling, and Summit at the top of the TOP500. Two panels below: "What the order tells you" and "What stayed the same".

Details The dates on the timeline are the ones the rest of the deck depends on, so they are worth fixing precisely. James Clark's Geometry Engine paper was presented at SIGGRAPH 1982, and Kurt Akeley's RealityEngine paper at SIGGRAPH 1993. The first retail 3dfx Voodoo Graphics board shipped in October 1996[†]. NVIDIA announced the GeForce 256 on August 31, 1999 and shipped it in October[†]. The GeForce 3 was announced in February 2001 (the Macintosh version at Macworld Tokyo on February 22, the PC version on February 26)[†], and ATI announced the Radeon 9700 on July 18, 2002, shipping within a month[†]. Brook for GPUs was presented at SIGGRAPH 2004. ATI's Xenos shipped in the Xbox 360 in November 2005. The GeForce 8800 GTX (G80) and CUDA were announced together on November 8, 2006[†]; the CUDA software development kit entered public beta on February 16, 2007 and version 1.0 followed on June 26, 2007[†]; the Tesla C870, D870 and S870 were announced on June 20, 2007[†]. The Khronos Group released the OpenCL 1.0 specification on December 8, 2008[†].

The second half of the timeline is the computing era proper. GT200 (GeForce GTX 280, Tesla C1060) arrived in June 2008 with the first double-precision units. Fermi was unveiled at NVIDIA's GPU Technology Conference on September 30, 2009 and shipped in volume as the Tesla C2050 in 2010; Tianhe-1A, with 7,168 Tesla M2050s, was number one on the TOP500 list of November 2010[†]. Kepler's Tesla K20X was announced in November 2012 and Titan, with 18,688 of them, was number one that same month[†]; AlexNet, trained on two GeForce GTX 580s, won the ImageNet competition in 2012; cuDNN appeared in 2014; Maxwell's GTX 980 shipped in September 2014 and the Tesla M40 in November 2015[†]. Pascal's Tesla P100 was announced on April 5, 2016, and Volta's Tesla V100 on May 10, 2017[†]. Summit, with 27,648 V100s, was number one on the TOP500 list of June 2018[†].

The left panel reads the order as a chain of causes, and this is the lecture's organizing argument. The fixed-function pipeline (Section 2) could only add effects by adding stages, so the stages became programmable; the programmable stages were separate units that sat idle when a frame was heavy in one kind of work, so they were unified into one pool of processors (Section 3); a unified pool of floating-point processors with a memory system is a general parallel computer, so researchers programmed it, first through the graphics API and then through CUDA (Sections 3–4); and once the dominant programmers were deep-learning researchers, whose work is matrix multiplication, the processors specialized again around that one operation (Section 8). Each move was a response to a limitation of the previous one, not a plan.

The right panel is the complementary claim: what did not change. The hardware from the RealityEngine to Volta exploits one property of its workload, that the work is a very large number of independent, identical computations (vertices, pixels, threads, matrix elements) with regular memory access. Every design choice the following slides describe, SIMT execution, warps of 32, wide memory buses, small caches, many threads in flight, is a way to exploit that regularity cheaply in silicon. Where the workload lacks the property, the design is at a disadvantage, and slide 27's debate about how large the GPU's advantage really is turns on exactly this.

Terms

Geometry Engine Clark's 1982 VLSI chip for the geometric transformations of the graphics pipeline, the origin of putting the pipeline's arithmetic in dedicated hardware; the basis of Silicon Graphics.

RealityEngine Silicon Graphics' 1993 high-end graphics system, a complete hardware pipeline built from boards of custom chips.

Voodoo 3dfx's 1996 consumer 3D accelerator, the first widely successful PC card to put rasterization and texturing in hardware.

GeForce 256 NVIDIA's 1999 graphics chip, the first consumer part with hardware transform and lighting, marketed as "the world's first GPU".

programmable shader A stage of the graphics pipeline that runs a program supplied by the application per vertex or per pixel, replacing a fixed function; introduced in 2001–02. Slide 6.

GPGPU General-purpose computation on graphics processing units: using the programmable pipeline, through a graphics API, for non-graphics computation, roughly 2003–06. Slide 7.

unified shader architecture A GPU in which one pool of identical processors executes every programmable stage instead of separate vertex and pixel units; Xenos (2005), G80 (2006). Slide 8.

CUDA NVIDIA's programming model and software for general computation on its GPUs, announced in November 2006 and released in 2007. Slide 9.

Tesla NVIDIA's brand for compute-only GPU products from 2007 to the V100 (not to be confused with the Tesla microarchitecture name used for G80 and GT200).

OpenCL The Khronos Group's vendor-neutral standard for programming GPUs and other parallel devices, version 1.0 released in December 2008.

TOP500 The twice-yearly ranking of the world's fastest supercomputers by their performance on the HPL linear-algebra benchmark.

FinFET A transistor with a vertical fin channel, used from TSMC's 16 nm process onward; the process technology of Pascal and Volta.

NVLink NVIDIA's point-to-point link between GPUs (and, on POWER systems, between GPU and CPU), introduced with Pascal. Slide 25.

Intuition

Read the timeline as four verbs: fix (1982–99), program (2001–04), unify (2005–07), specialize (2016–17). Each verb was adopted because the previous one ran out of room, and none of them changed the one fact the hardware exploits: the work is the same thing done a million times.

2 The graphics pipeline, fixed and programmable (slides 5–6)

2.1 Slide 5: The fixed-function graphics pipeline

Vertices in, pixels out: a hardware assembly line whose stages were fixed in silicon until 2001.

What the slide shows Six boxes in a row across the top, connected by arrows, each with a one-line gloss beneath it: vertices (the application sends triangles); transform and lighting (model coordinates to screen coordinates, light computed per vertex); rasterization (triangles to fragments); texture and shading (look up texels, blend colors); z-test and blend (depth, alpha, write); and the framebuffer (pixels to the display). The boxes darken from left to right, and the caption names the GeForce 256 of 1999 as the chip that implemented this pipeline whole. Two panels below: "Why a pipeline, and why hardware", and "GeForce 256: the word 'GPU' ".

Details The graphics pipeline is the sequence of operations that turns a description of a three-dimensional scene into a two-dimensional image. The application supplies geometry as triangles, each defined by three vertices with positions, normals and texture coordinates. The transform stage multiplies every vertex by a 4×4 matrix to move it from the model's coordinate system through the world's and the camera's to the screen's, and the lighting stage evaluates a shading equation at every vertex from the light sources' positions and the surface normal. Rasterization determines which pixels each screen-space triangle covers and generates a fragment for each, interpolating the vertex attributes across the triangle. The texture stage looks up colors in images (textures) indexed by the interpolated coordinates and combines them with the interpolated color; the depth test discards fragments hidden behind ones already drawn; blending combines the surviving fragment's color with the pixel already in the framebuffer, which is the memory holding the image sent to the display.

Two properties of this computation determined the hardware. First, within a stage the work is independent: every vertex is transformed by the same matrix without reference to any other vertex, and every fragment is shaded without reference to any other fragment. Second, the stages form a pipeline: vertices flow to rasterization, fragments flow to shading, and all stages can be busy at once on different triangles. Hardware exploits both properties directly, by replicating the units of a stage to process many items in parallel and by keeping every stage busy at the same time. Clark's Geometry Engine (1982) implemented the transformation arithmetic in a custom VLSI chip, Silicon Graphics' RealityEngine (1992; Akeley's paper is SIGGRAPH 1993) implemented the entire pipeline across boards of programmable geometry processors (Intel i860XP parts) and custom raster chips for workstations, and 3dfx's Voodoo Graphics (1996) brought rasterization and texturing, the stages with the most parallel work, onto a single affordable card for PC games; transform and lighting still ran on the CPU.

NVIDIA's GeForce 256, announced on August 31, 1999, moved transform and lighting onto the graphics chip, so that for the first time a single consumer die held the whole pipeline. NVIDIA marketed it as "the world's first GPU" and defined the term by this feature†. The definition was a marketing one, but it named a real architectural fact: a chip containing a fixed set of pipeline stages, each a wide, specialized unit, fed by a memory system designed for texture reads. NVIDIA quoted "nearly 23 million transistors" at launch and 480 million pixels per second of fill rate†; hardware databases list 17 million transistors, so the count should be quoted as 17–23 million with the source named†. The chip was made on a 220 nm process and clocked at 120 MHz†. ATI's Radeon followed in 2000†. (ATI later coined its own term, "visual processing unit", for the Radeon 9700 in July 2002†.)

What the fixed-function design could not do is in the slide's title. The effects a developer could use were those the hardware's stages implemented, chosen by setting modes and parameters; an effect the designers had not anticipated could not be added by the application. Each new graphics API version added stages and modes, and each chip grew to implement them, until the only way to keep adding effects was to let the application supply a program to run inside a stage. That is slide 6.

Terms

graphics pipeline The fixed sequence of stages (transform and lighting, rasterization, texturing and shading, depth test and blending) that turns triangles into an image.

vertex A corner of a triangle, carrying a position and attributes (normal, color, texture coordinates); the unit of work for the geometry stages.

transform and lighting The stage that moves vertices from model to screen coordinates by matrix multiplication and computes lighting per vertex; abbreviated T&L. Done on the CPU until the GeForce 256 put it on the chip.

rasterization The stage that determines which pixels a screen-space triangle covers and emits a fragment for each, interpolating the vertex attributes.

fragment The data for one pixel-sized piece of a triangle produced by rasterization (position, interpolated color, texture coordinates, depth); a fragment becomes a pixel if it survives the depth test.

texture An image mapped onto a surface; the texture stage reads it at the interpolated coordinates. Its elements are texels.

depth test The comparison of a fragment's depth against the stored depth of the pixel it covers, discarding fragments hidden behind nearer ones; also called the z-test.

blending Combining a fragment's color with the color already in the framebuffer, for example for transparency.

framebuffer The region of memory holding the image being drawn, read out to the display.

fill rate The number of pixels (or texels) a graphics chip can write per second; the headline performance figure of the fixed-function era.

fixed-function hardware A pipeline stage implemented as dedicated circuitry with selectable modes rather than as a programmable processor.

Intuition

A fixed-function pipeline is a factory line where every station does one job on every item that passes: fast, because each station is built for exactly that job, and inflexible, because adding a new product means building a new station.

2.2 Slide 6: Programmable shaders: a processor hiding in each stage

2001–2004: vertex and pixel stages become small SIMD processors; separate units per stage leave silicon idle.

What the slide shows At the top, the DirectX 9 era pipeline in four boxes: vertex shader (red, programmable), rasterizer (blue, fixed), pixel shader (red, programmable), and the raster operations unit for depth and blending (blue, fixed). Below it, an illustrative pair of bar charts labeled "load imbalance with separate units": in a geometry-heavy frame the vertex units are almost fully busy and the pixel units less than a third busy; in a pixel-heavy frame the reverse. A caption draws the conclusion that whichever stage is light, its units idle, and that the fix is one pool of processors that any stage can use. The right panel, "The steps", lists the GeForce 3 (2001), the Radeon 9700 (2002), the GeForce FX and 6800 (2003–04), the shading languages, and a definition of what a shader unit is.

Details The first programmable stage was the vertex stage. NVIDIA's GeForce 3 (NV20, February 2001) replaced the fixed transform-and-lighting unit with a programmable vertex engine: the application supplied a short program in an assembly-like language, and the hardware ran it once per vertex, with the same program applied to every vertex of a draw call. Lindholm, Kilgard and Moreton described the design at SIGGRAPH 2001 ("A user-programmable vertex engine"); the chip had about 57 million transistors† and implemented Microsoft's DirectX 8. Programmability reached the pixel stage with ATI's Radeon 9700 (R300), announced in July 2002, the first chip to implement DirectX 9's floating-point pixel shaders; ATI quoted about 110 million transistors†. NVIDIA's GeForce FX (2003) and GeForce 6800 (April 2004) extended both stages with longer programs, dynamic branching and, in the 6800's Shader Model 3.0, texture reads from the vertex program; Montrym and Moreton's IEEE Micro article (2005) describes the 6800 as the last of the designs with separate vertex and pixel processors.

The programs were written first in assembly and then in C-like languages: NVIDIA's Cg (described by Mark, Glanville, Akeley and Kilgard at SIGGRAPH 2003), Microsoft's HLSL for DirectX 9, and the OpenGL Shading Language, GLSL, which entered the OpenGL core in version 2.0 (2004)†. The slide's definition is the important one: a shader unit is a small SIMD processor that runs one program on many vertices or many pixels at once, whose only way of reading memory is the texture unit and whose only way of writing is the framebuffer. Each unit holds a few registers per item, executes a limited instruction set with floating-point arithmetic, and cannot communicate with its neighbors. Researchers noticed that a chip with several of these, each running dozens of items in parallel over hundreds of megabytes of memory at tens of gigabytes per second, was a parallel computer with an unusual interface, and slide 7 describes what they did with it.

The slide's lower-left chart is the design problem that programmable stages created. Vertex and pixel shaders were separate hardware units, sized for a typical frame. A frame with many small triangles and simple shading (geometry-heavy) keeps the vertex units busy while the pixel units wait; a frame with few large triangles and complex per-pixel shading (pixel-heavy) does the reverse. Because the two kinds of unit were fixed in silicon, whichever was less loaded was idle, and since different games, and different frames of one game, have different ratios, no fixed division of the die was right. The bars are illustrative; the point is qualitative. The fix, adopted by ATI's Xenos in 2005 and NVIDIA's G80 in 2006, was to build one pool of identical processors and let the scheduler assign vertex or pixel work to any of them as the frame demands (slide 8). Microsoft's Direct3D 10, described by Blythe at SIGGRAPH 2006, made the common shader model the API's assumption.

Terms

vertex shader A program the application supplies to run once per vertex in place of the fixed transform-and-lighting stage; introduced with the GeForce 3 in 2001.

pixel shader A program run once per fragment in place of the fixed texturing and shading stage; also called a fragment shader. Floating-point pixel shaders arrived with the Radeon 9700 in 2002.

raster operations unit The fixed-function unit (ROP) that performs the depth test and blending and writes pixels to the framebuffer.

DirectX Microsoft's family of graphics APIs for Windows; DirectX 8 (2001) introduced programmable vertex shaders, DirectX 9 (2002) floating-point pixel shaders, Direct3D 10 (2006) the unified shader model.

Shader Model 3.0 The DirectX 9 shader specification implemented by the GeForce 6800 (2004), adding dynamic branching and texture reads in the vertex stage.

Cg "C for graphics", NVIDIA's C-like shading language (2002–03), described by Mark et al. at SIG-

GRAPH 2003.

HLSL Microsoft's High Level Shading Language for DirectX 9 (2002).

GLSL The OpenGL Shading Language, part of OpenGL 2.0 (2004).

SIMD Single instruction, multiple data: one instruction applied to several data items at once; the execution style of a shader unit.

load imbalance Idle hardware caused by a fixed division of resources among stages whose demand varies; the motive for the unified shader architecture.

draw call One request from the application to render a set of triangles with a given program and state; the unit over which a shader program is constant.

Intuition

Each programmable stage was a small computer bolted into a slot on the assembly line, able to run only the kind of job that slot received. Having two kinds of slot meant one kind was always half empty; the obvious repair was to make the computers interchangeable.

3 GPGPU and the unified G80 (slides 7–8)

3.1 Slide 7: GPGPU: computing with pixels

2003–2006: arrays became textures, kernels became fragment shaders, and a graphics API hid a parallel computer.

What the slide shows On the left, the GPGPU mapping as five correspondences with arrows: an array $A[i, j]$ becomes a texture; a loop over the elements becomes the drawing of a full-screen quadrilateral; the kernel body becomes a fragment shader; the output array becomes a texture rendered to; and reading the result back becomes a copy to the host. A caption states the mechanism: every output pixel is one kernel invocation, the rasterizer is the loop, and texture fetches are the only reads. Below it a panel, "What it could do": linear algebra, FFTs, fluid simulation, ray tracing and sorting, Owens et al.'s survey, Brook and Sh. On the right, "What it could not do": no scatter, no on-chip sharing, odd arithmetic, fixed resources, and the lesson drawn for 2006.

Details General-purpose computation on GPUs, GPGPU, was the practice of using the programmable shader hardware of slide 6 for computations that had nothing to do with graphics, through the graphics API, because that was the only way to reach the hardware. The mapping is mechanical once seen. A two-dimensional array is stored as a texture, since textures are the only large arrays a shader can read. To run a computation over every element, the program draws a single rectangle covering the whole output: the rasterizer generates one fragment per output pixel, and the fragment shader, which the programmer has written to contain the body of the loop, runs once for each, reading its inputs by texture fetches at coordinates derived from its pixel position and writing its one result as the pixel's color. The output is directed not to the display but to another texture (render-to-texture), which can then serve as the input of the next pass. When the computation is finished, the result is copied from the graphics memory to the host.

Within these constraints a great deal was done. Owens and colleagues' survey, first presented as a Eurographics state-of-the-art report in 2005 and published in *Computer Graphics Forum* in 2007, catalogs several hundred papers: dense and sparse linear algebra, fast Fourier transforms, fluid and cloth simulation, ray tracing, sorting networks, database operations and image processing, many running several times faster

than the CPUs of the day. Ian Buck and colleagues' Brook for GPUs (SIGGRAPH 2004) and the Sh system gave the pattern a programming language, with streams in place of textures and kernels in place of shaders, and compiled it to DirectX or OpenGL shader code. The earliest machine-learning work on GPUs belongs to this period: Steinkraus, Simard and Buck (2005) ran neural networks through the graphics API, and Chellapilla, Puri and Simard (2006) formulated convolutional networks as matrix multiplications that the shader hardware could execute. The people who did this work went on to write CUDA and its first libraries; Buck led the CUDA effort at NVIDIA.

The right panel is the catalog of what the graphics API withheld, and it reads as a specification for G80. A fragment could write only to its own pixel, so any computation that scatters results to data-dependent addresses, such as a histogram or a sparse update, needed indirect tricks. There was no memory shared among fragments and no way for fragments to synchronize, so any communication went to the framebuffer and back in another pass, through DRAM. The arithmetic was whatever the graphics hardware offered: 16- or 24-bit floating point on some parts, no integer operations, no guarantee of IEEE rounding, and no double precision at all. Register counts, program lengths and the number of outputs per fragment were capped by the API version, and the driver, not the programmer, decided how work was scheduled onto the units.

The lesson the slide draws is the one NVIDIA drew. The valuable parts were the wide SIMD arrays of floating-point units and the memory system built for high-bandwidth texture reads; the obstacle was the graphics API standing between the programmer and them. G80 kept the first and removed the second, adding the features the list shows missing: arbitrary writes, on-chip shared memory with synchronization within a block, IEEE single-precision arithmetic with integer support, and a programming model that exposes the processor count and scheduling.

Terms

GPGPU General-purpose computation on graphics processing units, specifically the 2003–06 practice of programming shader hardware through a graphics API; by extension, any non-graphics GPU computing.

render-to-texture Directing a rendering pass's output into a texture rather than the display, so that the result can be read by a later pass; the GPGPU mechanism for writing an array.

full-screen quad A rectangle covering the whole output, drawn so that the rasterizer generates exactly one fragment per output element; the GPGPU equivalent of a loop.

texture fetch A read from a texture by a shader at given coordinates; in GPGPU, the only way to read an input array.

scatter Writing results to data-dependent addresses, as in a histogram; impossible for a fragment shader, which writes only its own pixel.

gather Reading from data-dependent addresses, which texture fetches allow; GPGPU programs had gather but not scatter.

Brook for GPUs Buck et al.'s 2004 stream-programming language and compiler that expressed GPGPU computations as kernels over streams and compiled them to shaders; the direct ancestor of CUDA.

stream Brook's term for an array over which a kernel is applied element-wise.

pass One rendering operation over the whole output; GPGPU computations that needed communication required several passes with DRAM traffic between them.

IEEE 754 The standard for floating-point arithmetic and rounding; early GPU shaders did not guarantee it, and G80 did for single precision.

Intuition

GPGPU was writing a physics simulation by convincing a graphics card that the simulation was a picture. It worked well enough to prove the hardware was a computer and badly enough to show exactly what a computer version of the hardware would need.

3.2 Slide 8: G80 (2006): one unified array of processors, and CUDA

The GeForce 8800 GTX replaced separate vertex and pixel units with 128 identical processors, then got a C compiler.

What the slide shows A block diagram of G80 on the left: eight texture/processor clusters side by side, each containing sixteen green squares (two streaming multiprocessors of eight streaming processors) over a blue texture unit, with a violet band beneath for the six memory partitions of 384-bit GDDR3 at 86 GB/s. The caption gives the per-SM resources (8 processors, 16 KB of shared memory, 8,192 registers, a warp scheduler), the chip's 681 million transistors on a 470 mm^2 die at 90 nm, and a typical board power of about 150 W, citing Lindholm et al. (2008). Below it a panel, "Why unify", and on the right a panel, "CUDA (2007) and what followed", with four points: a C compiler for the array, the Tesla products, the competition's answers, and why CUDA stuck.

Details The GeForce 8800 GTX, announced on November 8, 2006, was built on the G80 die: 681 million transistors on TSMC's 90 nm process, the largest graphics chip to that date†. Its organization is the template for the rest of the deck. The programmable work is done by 128 identical scalar processors, called streaming processors (SPs), running at 1.35 GHz; eight of them form a streaming multiprocessor (SM) together with 16 KB of shared memory, a register file of 8,192 32-bit entries, an instruction unit that schedules warps of 32 threads, and two special-function units for transcendentals. Two SMs and a texture unit form a texture/processor cluster, and there are eight clusters. Six memory partitions, each with a 64-bit interface, give a 384-bit bus to GDDR3 memory at 900 MHz (1.8 Gb/s per pin), for 86.4 GB/s†. The 8800 GTX carried 768 MB; the compute version, the Tesla C870, carried 1.5 GB at 76.8 GB/s†. There was no double-precision arithmetic. Lindholm, Nickolls, Oberman and Montrym's IEEE Micro article of 2008 is the architecture paper.

Two decisions in this design mattered beyond graphics. The first is unification: the same processors run vertex programs, geometry programs, pixel programs and, through CUDA, arbitrary computation, and a hardware scheduler assigns whichever kind of work is waiting. The load imbalance of slide 6 disappears, because a geometry-heavy frame simply gives more of the 128 processors to vertex work. ATI's Xenos, the GPU of Microsoft's Xbox 360 (November 2005; Andrews and Baker, IEEE Micro 2006), was the first unified-shader design; G80 was the first for the PC and the first exposed through a common API (DirectX 10). The second decision is that the processors are scalar: each executes one operation on one value per clock, rather than the four-wide vector operations of earlier shader units. A scalar design is simpler to compile for and keeps every lane busy regardless of the data's natural width, and it is what makes the "thread" of CUDA a natural unit.

CUDA was announced with the chip on the same day†. The software development kit and C compiler entered public beta on February 16, 2007 and version 1.0 was released on June 26, 2007†. It presents the SMs to a C programmer as a hierarchy of threads, blocks and grids (slide 9), with the shared memory and a barrier instruction exposed directly and the graphics API absent; a kernel is a C function, and the hardware's warp scheduler, not the driver, decides what runs. The Tesla C870 card, with the D870 deskside and S870 rack units, followed on June 20, 2007: the same chip without display output, with 1.5 GB of memory and "up to 518 gigaflops" quoted†. Tesla remained the name of NVIDIA's compute line through the V100.

The competition answered in kind. ATI released Close To Metal in November 2006†, a low-level interface to its shader hardware, and then the Stream SDK; the Khronos Group published OpenCL 1.0 in December

2008, a vendor-neutral standard with the same model under different names (work-items for threads, work-groups for blocks, local memory for shared memory)[†]. The slide's last point is why CUDA, rather than OpenCL or a vendor alternative, became the way GPUs were programmed for the next decade: the model matched the hardware closely enough that performance was predictable (slide 10); the hardware was in every gaming PC, so there was no installed-base problem; and NVIDIA shipped libraries, cuBLAS and cuFFT first and cuDNN in 2014, that made the common cases fast without writing a kernel. Nickolls, Buck, Garland and Skadron's article in *ACM Queue* (2008) is the programming-model paper.

Terms

G80 The NVIDIA die of the GeForce 8800 GTX (November 2006) and Tesla C870 (2007): the first unified-shader, CUDA-capable GPU, with 128 scalar processors in 16 SMs.

streaming processor The SP, G80's name for a scalar FP32 arithmetic lane; eight per SM. Renamed "CUDA core" from Fermi onward.

texture/processor cluster G80's grouping of two SMs with a shared texture unit; eight per chip.

shared memory A small, fast, programmer-managed memory on each SM, shared by the threads of one block; 16 KB on G80, up to 96 KB on Volta.

register file The per-SM storage holding every resident thread's registers; 8,192 32-bit entries on G80, 65,536 from Kepler onward.

memory partition One of the independent slices of the GPU's memory system, each with its own DRAM interface; six 64-bit partitions on G80.

GDDR3 Graphics double-data-rate DRAM, third generation; the memory of G80 and GT200.

scalar ISA An instruction set in which each instruction operates on one value per thread, as opposed to a vector ISA operating on several; G80's choice, inherited by every later NVIDIA GPU.

Xenos ATI's GPU for the Xbox 360 (2005), the first unified-shader design.

Tesla C870 The first compute-only product from G80 (June 2007), with 1.5 GB of memory and no display output.

Close To Metal ATI's 2006 low-level interface to its shader hardware; the predecessor of AMD's Stream SDK.

work-item OpenCL's name for a thread; a work-group is OpenCL's name for a block.

cuBLAS NVIDIA's CUDA implementation of the BLAS dense linear-algebra routines; with cuFFT, the first CUDA library.

Intuition

G80 is the GPGPU wish list built in silicon: the same wide array of floating-point units and the same texture memory system as before, with the graphics API lifted off the top and a C compiler put in its place. Everything after it is this chip with the counts changed.

4 The CUDA model and its mapping to the chip (slides 9–10)

4.1 Slide 9: The CUDA programming model: grids of blocks of threads

Write the program for one thread; launch tens of thousands; let the hardware schedule blocks onto SMs.

What the slide shows A monospace box at the top left holds the shortest useful CUDA program, SAXPY ($y \leftarrow ax+y$): a kernel `saxpy(int n, float a, float *x, float *y)` marked `__global__`, whose body computes a global index `i = blockIdx.x * blockDim.x + threadIdx.x` and, if `i < n`, updates one element; and a host-side launch `saxpy<<<(n + 255) / 256, 256>>>(n, 2.0f, d_x, d_y)` that asks for blocks of 256 threads and enough blocks to cover n. Below the code, a diagram of the launch: a grid containing six blocks, each a rectangle of eight small thread cells, with a caption that blocks run in any order, on any SM, with no communication between them, and a note that threads in a block share `__shared__` memory and can wait at `__syncthreads()` while threads in different blocks cannot. The right panel lists the memory spaces as the programmer sees them: registers, shared memory, global memory, constant and texture memory, local memory; and a closing remark that the model is the hardware of slide 10 with the names changed, with Nickolls et al. (2008) as the paper and Kirk and Hwu's textbook as the course.

Details The CUDA model, announced with G80 on November 8, 2006 and released as a public beta on February 16, 2007, is three ideas stacked. The first is that the programmer writes the body of one thread and the hardware runs it for every thread in a launch. SAXPY makes this concrete: there is no loop over i in the kernel, because the loop is the launch. Each thread computes its own i from three built-in values: `blockIdx.x`, which block it belongs to; `blockDim.x`, how many threads a block has; and `threadIdx.x`, its position in the block. The guard `if (i < n)` handles the last block, which is only partly used when n is not a multiple of 256. The host launch computes the number of blocks as the ceiling of $n/256$ in integer arithmetic; the rest is the ordinary C of a function call, with the triple-chevron syntax carrying the launch configuration. The pointers `d_x` and `d_y` refer to GPU memory, allocated and filled by `cudaMalloc` and `cudaMemcpy` on the host before the launch, because through the Pascal generation the GPU is a device with its own memory and the host must move data to it or map it (slide 20).

The second idea is the two-level hierarchy. Threads are grouped into blocks, and blocks into a grid. The grouping is not decoration: a block is the unit that the hardware places on one streaming multiprocessor, so threads in a block can share the SM's on-chip memory and synchronize at a barrier, while threads in different blocks have neither. This is what makes the model scalable. A launch of ten thousand blocks runs correctly on a GPU with sixteen SMs and on one with eighty, because the program made no assumption about how many blocks run at once or in what order; the hardware's block scheduler simply hands blocks out as SMs free up. The price is that any algorithm needing communication across the whole problem (a global reduction, a prefix sum over the full array) must be written as a sequence of kernel launches or use atomic operations on global memory, each launch a global barrier. Nickolls, Buck, Garland and Skadron's 2008 paper in *ACM Queue* describes the model this way and calls the scalability across SM counts its central design goal.

The third idea is the explicit memory hierarchy. Registers are per thread and allocated by the compiler; they are the largest and fastest storage on the chip (128 KB per SM on Fermi, 256 KB from Kepler on, slide 10). Shared memory is per block, lives on the SM, and is managed by the programmer: 16 KB per SM on G80, 48 KB on Fermi and Kepler, 96 KB on Maxwell and Volta, 64 KB on Pascal. It is the tool for reuse (a tile of a matrix loaded once and read many times by the threads of a block) and for cooperation (a reduction tree within a block). Global memory is the GPU's DRAM, visible to every thread and to the host through copies, with latency measured in hundreds of cycles (slide 12). Constant and texture memory are read-only paths with their own small caches, inherited from the graphics pipeline, useful for coefficients and for lookup tables with spatial locality. Local memory is the compiler's name for register spills and large per-thread arrays; it lives in DRAM and is cached from Fermi on, and a kernel that uses much of it is usually in trouble. The slide's observation is that this list is not an abstraction chosen for elegance: it is the hardware of slide 10 with the names changed, and the programmer who keeps the chip in mind writes the faster program.

Two things the slide does not say but the course would. CUDA is one of two surviving expressions of this model; OpenCL 1.0 (December 2008) has the same structure under different names (work-items,

work-groups, local memory, NDRange), and the Khronos standard is the one AMD and Intel implemented. And the model's success rests as much on the libraries as on the language: cuBLAS and cuFFT shipped with CUDA 1.0, and most programs that run on GPUs call them rather than writing kernels.

Global index of a thread in a one-dimensional launch: $i = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}$. Blocks needed for n elements at B threads per block: $\lceil n/B \rceil = (n + B - 1) \operatorname{div} B$; for $B = 256$, $(n + 255) \operatorname{div} 256$. The last block has $n - 256\,(\lceil n/256 \rceil - 1)$ live threads; the guard `if (i < n)` idles the rest.

Terms

CUDA NVIDIA's programming model and toolchain for its GPUs (Compute Unified Device Architecture), announced November 2006 and released in 2007: C with extensions for kernels, a thread hierarchy and explicit memory spaces.

kernel A function marked `__global__` that runs on the GPU, once per thread of a launch.

thread (CUDA) One instance of a kernel's execution, with its own registers and index; the unit the programmer writes for.

block A group of threads (up to 1,024 from Fermi on) that runs on one SM, shares that SM's shared memory and can synchronize with a barrier. Also called a thread block or, in OpenCL, a work-group.

grid All the blocks of one kernel launch; blocks in a grid run in any order and cannot communicate except through global memory.

launch configuration The grid and block dimensions given in the triple chevrons of a kernel call, `«<blocks, threads»>`.

SAXPY The BLAS level-1 operation $y \leftarrow ax + y$ on single-precision vectors; the standard first kernel and the deck's example of a bandwidth-bound computation (slide 15).

shared memory Programmer-managed on-chip SRAM per SM, allocated per block and visible to that block's threads; 16 KB per SM on G80, up to 96 KB on Volta.

syncthreads The barrier `__syncthreads()` at which all threads of a block wait; the only synchronization primitive across a whole block.

global memory The GPU's DRAM, visible to every thread and to the host; hundreds of cycles of latency, hundreds of gigabytes per second of bandwidth.

constant memory A small read-only region with a dedicated cache, fast when all threads of a warp read the same address.

texture memory Read-only access to global memory through the texture units and their cache, with spatial locality and interpolation inherited from graphics.

local memory Per-thread storage in DRAM used by the compiler for register spills and large per-thread arrays; private to the thread despite living off-chip.

unified memory A single virtual address space spanning CPU and GPU memory, with pages migrated on demand; introduced in software with CUDA 6 and in hardware with Pascal's page faulting (slide 20).

OpenCL The Khronos Group's vendor-neutral standard for the same model (December 2008), with work-items, work-groups and local memory in place of threads, blocks and shared memory.

Intuition

A CUDA kernel is the body of a `for` loop with the loop header removed and handed to the hardware. The block is the one piece of structure the programmer keeps: it says which iterations may talk to each other, and the answer must be "only these few hundred," because that is what fits on one SM.

4.2 Slide 10: Mapping the model to the chip: blocks to SMs, threads to warps

A streaming multiprocessor runs several blocks at once; its scheduler issues one instruction for 32 threads at a time.

What the slide shows A block diagram of one Fermi streaming multiprocessor (2010), labeled the unit the rest of the deck is about. From top to bottom: an instruction cache and two warp schedulers; a register file of 32,768 32-bit entries (128 KB); two rows of sixteen orange cells, the 32 CUDA cores (FP32 and INT32 fused multiply-add lanes); a row of sixteen load/store units and four special function units for transcendentals (sine, reciprocal square root); and at the bottom 64 KB of on-chip SRAM configurable as 48 KB of shared memory with 16 KB of L1 cache or the reverse. A caption gives the resident limits: up to 8 blocks and 48 warps (1,536 threads) per SM, 16 SMs on a GF100 die, citing Wittenbrink et al. (2011). The right panel describes the mapping in three steps: block to SM, 32 threads to a warp, and occupancy as the ratio of resident warps to the maximum, with the worked example that a kernel using 64 registers per thread fills a Fermi register file with 512 threads, a third of the warp slots.

Details The streaming multiprocessor is the hardware counterpart of the block. When the GigaThread engine (NVIDIA's name for the chip-level block scheduler) places a block on an SM, it allocates that block's registers from the SM's register file, its shared memory from the SM's SRAM, and its warps from the SM's warp slots; the block stays there until every one of its threads has finished. Because the whole block is resident on one SM, its threads can read the same shared memory and can all reach a barrier, which is exactly what the programming model promised. Several blocks share an SM at once (up to 8 on Fermi, 16 on Kepler, 32 on Maxwell and later), and when one finishes its resources go back to the pool and the scheduler places the next. A launch with more blocks than the chip can hold simply queues them; a launch with fewer leaves SMs idle, which is why the practical advice is to launch several times more blocks than there are SMs.

Within the SM the unit of scheduling is not the thread but the warp: 32 consecutive threads of a block, grouped by the hardware (threads 0–31 form warp 0, 32–63 warp 1, and so on). The warp scheduler holds one program counter per warp (through Pascal; slide 24 describes Volta's change) and, each cycle, picks a warp whose next instruction is ready and issues that instruction to the lanes. On Fermi the 32 cores are organized as two groups of 16, and each of the two schedulers issues one warp instruction per cycle to one group, so a 32-thread warp instruction takes two cycles to pass through 16 lanes; the net rate is one warp instruction per cycle per SM. Kepler widened the SMX to 192 lanes with four schedulers each able to issue two instructions; Maxwell, Pascal and Volta settled on four schedulers over four partitions of 32, 16 and 16 lanes respectively (slide 21). The counts change; the scheme does not: a thread never has its own program counter or its own issue slot, and the SM's control logic is amortized over 32 lanes (slide 11).

The three resources a block consumes bound how many blocks and warps an SM can hold, and the ratio of resident warps to the maximum is the occupancy. The register file is the usual limit. Fermi has 32,768 registers per SM; a kernel compiled to use 64 registers per thread can therefore have at most $32{,}768/64 = 512$ threads resident, which is 16 warps of the 48 the SM can hold, an occupancy of one third. The same kernel with 21 registers per thread could fill all 48 warps. Shared memory bounds the count the same way: a block

that declares 24 KB of shared memory allows only two blocks per SM in the 48 KB configuration. Warp slots bound it directly: 48 warps on Fermi, 64 from Kepler on. The compiler reports registers and shared memory per kernel, and the occupancy calculator that ships with CUDA turns them into resident warps; slide 12 explains why the number matters, and why it is not the whole story.

The figures in the caption are the Fermi values from Wittenbrink, Kilgariff and Prabhu's *IEEE Micro* article (2011) and the Fermi whitepaper: 32 cores, 16 load/store units, 4 SFUs, two warp schedulers, 32,768 registers, 64 KB of SRAM split 48/16 or 16/48, 48 warps and 1,536 threads per SM, 8 blocks per SM. Kepler raised the register file to 65,536 entries (256 KB), the resident threads to 2,048 (64 warps) and the registers per thread to 255, and those three limits held through Volta; Pascal and Volta kept 256 KB of registers per SM while shrinking the SM to 64 lanes, which is why the register file per lane grew fourfold between Kepler and Volta. The picture, in other words, is stable from G80 to Volta with the counts changed, and the table on slide 18 gives the counts.

Occupancy $= \dfrac{\text{resident warps}}{\text{maximum warps per SM}}$, with resident warps the minimum of three limits: $\left\lfloor \dfrac{R_{\text{SM}}}{r \cdot 32} \right\rfloor$ from registers (R_{SM} entries per SM, r per thread), $\left\lfloor \dfrac{S_{\text{SM}}}{s} \right\rfloor \cdot \dfrac{B}{32}$ from shared memory (S_{SM} per SM, s per block of B threads), and the warp-slot limit (48 on Fermi, 64 from Kepler). Fermi example: $r = 64$ gives $32{,}768/(64 \cdot 32) = 16$ warps of 48, occupancy $1/3$; $r = 21$ gives 48. Issue rate: one warp instruction per scheduler per cycle over 16 lanes, so a 32-thread instruction occupies its lanes for two cycles.

Terms

streaming multiprocessor The SM: the unit of a GPU that holds resident blocks and executes their warps; contains the warp schedulers, register file, arithmetic lanes, load/store units, special function units and on-chip SRAM. 16 on G80 and GF100, 80 on GV100.

CUDA core NVIDIA's name for one FP32 (and, through Pascal, INT32) fused multiply-add lane in an SM; these notes also call it a lane. Not a core in the CPU sense: it has no fetch, decode or scheduler of its own.

warp A group of 32 consecutive threads of a block that the SM schedules and executes together, one instruction at a time; the hardware unit of SIMT execution (slide 11).

warp scheduler The SM logic that holds the resident warps' program counters, picks a ready warp each cycle and issues its next instruction to a group of lanes; two per SM on Fermi, four from Maxwell on.

register file The SM's per-thread fast storage: 32,768 32-bit entries (128 KB) on Fermi, 65,536 (256 KB) from Kepler through Volta; partitioned among resident threads by the compiler's register count.

load/store unit An SM unit that computes addresses and issues memory requests for a warp; 16 on Fermi (two cycles per warp load), 32 on Kepler and Volta.

special function unit The SFU: a lane for transcendentals (sine, cosine, reciprocal, reciprocal square root, logarithm) at lower throughput than the FMA lanes; 4 per Fermi SM, 16 per Volta SM.

L1 cache The SM-local data cache, introduced by Fermi as a configurable split of a 64 KB SRAM with shared memory, merged with the texture cache on Maxwell, and unified with shared memory in a 128 KB array on Volta.

GigaThread engine NVIDIA's name for the chip-level scheduler that assigns blocks to SMs as resources free up and queues the rest.

occupancy The ratio of warps resident on an SM to the maximum the SM can hold; determined by the registers per thread, shared memory per block and warp slots; the quantity that governs latency hiding (slide 12).

occupancy calculator The spreadsheet, and later the API, shipped with CUDA that computes resident warps from a kernel's register and shared-memory usage for a given GPU.

compute capability NVIDIA's version number for the hardware features and limits a GPU supports (1.0 for G80, 2.0 Fermi, 3.x Kepler, 5.x Maxwell, 6.x Pascal, 7.0 Volta); the limits in this section are indexed by it.

Intuition

Think of an SM as a classroom with a fixed number of desks (warp slots), a fixed supply of notebooks (registers) and one whiteboard (shared memory). A block is a class that arrives together and leaves together; how many classes fit depends on which of the three runs out first, and the teacher addresses thirty-two students at a time whether or not they are all listening.

5 SIMT, latency hiding and energy (slides 11–13)

5.1 Slide 11: SIMT: SIMD hardware with a thread's face

One instruction, 32 lanes, each lane its own thread: branches are allowed and divergent branches are paid for in time.

What the slide shows A timeline of one warp executing `if (x[i] > 0) A; else B;`, with eight of the warp's 32 lanes drawn as rows and six issue slots as columns labeled eval cond, A, A, B, B, join. In the first and last columns every lane is active (dark). In the two A columns the lanes whose condition was true are active and the others are masked off (light); in the two B columns the pattern is reversed. The caption reads: both paths issue, each lane keeps one; six issue slots for four slots of work. A panel below, "Three ways to be wide," contrasts SIMD (one instruction, one register of n lanes, the compiler packs the data), MIMD (independent instruction streams, each with its own fetch and decode) and SIMT (SIMD execution with per-lane program counters simulated by masks, so each lane is a scalar thread with its own addresses and branches). The right panel lists what SIMT costs and what it buys: divergence, memory divergence, reconvergence and the pre-Volta deadlock, and the amortization of one instruction fetch over 32 operations.

Details SIMT is NVIDIA's name, introduced with G80 and explained in Lindholm, Nickolls, Oberman and Montrym's 2008 *IEEE Micro* article, for the execution model behind the warp. The hardware is SIMD: one instruction is fetched, decoded and issued once, and 32 lanes execute it on 32 data items. The programming model is scalar: each lane runs what looks like an independent thread, with its own register values, its own memory addresses and its own control flow. The reconciliation is the active mask. The warp keeps a 32-bit mask saying which lanes should commit results from the current instruction; a branch whose condition differs across lanes is executed by issuing the taken path with the mask set to the lanes that took it, then the other path with the complementary mask, then reconverging at the point where the paths rejoin. Nothing in the instruction stream is skipped on a lane's behalf; a masked lane simply discards its result. The slide's example shows the accounting. Four slots of useful work (the condition, one path, the join, plus whichever path a lane takes) cost six issue slots because both paths are issued to the whole warp.

This is the cost of divergence, and it is proportional to the number of distinct paths, not to their imbalance. A warp whose 32 lanes take k different paths through a branch region spends the sum of the k path lengths; in the worst case, 32 different paths, the warp runs 32 times slower than a warp that agrees. A loop whose

trip count differs across lanes behaves the same way: the warp iterates until its longest-running lane finishes, with the others masked. The standard fixes are to arrange data so that neighboring threads make the same decisions (sort work items by type, process boundaries in separate kernels) and to replace short branches with predicated arithmetic, which the compiler often does on its own. Fung, Sham, Yuan and Aamodt's 2007 MICRO paper on dynamic warp formation studied the hardware alternative, regrouping lanes from different warps that happen to agree, and the cost it would impose on the register file; shipping GPUs kept the simple scheme.

Memory divergence is the same phenomenon on the load/store path. A warp load instruction carries 32 addresses. If they fall within one 128-byte line the memory system issues one request; if they scatter across 32 lines it issues 32, each returning a full sector of which the warp uses four bytes (slide 14). The instruction issued once either way; the cost appears as bandwidth and latency rather than issue slots.

Reconvergence is where the implementation detail leaks into semantics. Through Pascal a warp has a single program counter and a hardware stack of reconvergence points: when the warp diverges, the scheduler runs one path to the reconvergence point, pops the stack and runs the other. Lanes on the first path therefore cannot wait for lanes on the second, because the second path does not begin until the first ends. A spin lock in which lane 3 waits for a flag that lane 7 will set, both in the same warp, never terminates: lane 7's path is not scheduled until lane 3's path finishes, and lane 3's path finishes only when the flag is set. Programs that respected the rule "threads of one warp do not wait on one another" were fine, and a good deal of code silently relied on the opposite, lock-step property: the warp-synchronous idiom of omitting `__syncthreads()` inside a 32-thread reduction. Volta's independent thread scheduling (slide 24) gave every thread its own program counter and stack, so the scheduler can interleave the two paths and the lock terminates; the price was that the lock-step assumption, never guaranteed, now actually fails unless the programmer asks for it with `__syncwarp()`.

What SIMT buys is the point of slide 13 stated early: the control path of a processor (instruction fetch, decode, dependency tracking, issue) costs energy and area that a scalar core pays for every operation, and SIMT pays once per 32 operations. The GPU gets a scalar programming model, which is what made CUDA learnable, on hardware whose per-operation control overhead is a thirty-second of a CPU's. The vector alternative, SIMD with explicit vector registers, achieves the same amortization but asks the programmer or the compiler to pack the data and to write masks by hand; the multicore alternative, MIMD, gives every thread its own program counter at the full cost of a fetch and decode each. SIMT is a position between them chosen for a workload in which neighboring threads almost always agree.

Divergence cost. For a warp whose lanes take paths $P_1, \ldots, P_k$ of lengths $\ell_1, \ldots, \ell_k$ instructions through a branch region, the issue slots spent are $\sum_{j=1}^{k} \ell_j$, against $\max_j \ell_j$ if every lane agreed; the slowdown is $\sum_j \ell_j / \max_j \ell_j \le k \le 32$. For a data-dependent loop with trip counts $t_1, \ldots, t_{32}$, the warp runs $\max_i t_i$ iterations and the useful fraction of its lane-cycles is $\sum_i t_i / (32 \max_i t_i)$. Control amortization: one fetch and decode per warp instruction serves 32 operations, so the control energy per operation is $1/32$ of a scalar core's for the same instruction.

Terms

SIMT Single instruction, multiple threads: NVIDIA's execution model in which one instruction is issued to 32 lanes, each lane running what the programmer wrote as an independent thread, with divergent control flow handled by masking.

SIMD Single instruction, multiple data: one instruction operating on a vector register of several elements at once, with the data packed by the compiler or programmer; the hardware beneath SIMT.

MIMD Multiple instruction, multiple data: independent processors each fetching and executing its own

instruction stream; a multicore CPU.

active mask The per-warp bit vector saying which of the 32 lanes commit the current instruction's result; set by the hardware on divergent branches.

divergence The condition in which lanes of one warp take different paths through a branch or loop, so the warp issues every path with the others masked; the cost grows with the number of distinct paths.

predication Executing an instruction on all lanes and discarding results where a per-lane predicate is false; the compiler's way of removing short branches.

memory divergence The condition in which a warp's 32 addresses fall in several memory lines, so one load instruction becomes several transactions (slide 14).

reconvergence point The instruction at which divergent paths rejoin and the full warp resumes together; tracked on a hardware stack through Pascal.

warp-synchronous programming The pre-Volta idiom of relying on the 32 threads of a warp executing in lock step to omit explicit synchronization; never guaranteed, and broken by independent thread scheduling.

syncwarp The Volta-era primitive `__syncwarp()` that makes the named lanes of a warp reconverge and synchronize explicitly.

lane One of the 32 execution slots of a warp, and the arithmetic pipeline that serves it; the deck's word for a CUDA core.

Intuition

A warp is a marching band that can only play one bar at a time. If half the band has a rest, they stand there holding their instruments while the other half plays, and then the roles swap; the piece takes as long as both halves' parts added together. Write music in which the whole band plays the same bar, and the band is as efficient as it is loud.

5.2 Slide 12: Latency hiding: enough threads in flight to cover the memory

A warp that waits on DRAM is parked and another issues; Little's law says how many must be in flight to keep DRAM busy.

What the slide shows An illustrative plot of throughput, as a fraction of peak, against the number of resident warps per SM from 0 to 64. A solid line for a memory-bound kernel with one load in flight per warp rises linearly and reaches 100% at about 40 warps, where a dashed vertical line marks the knee, labeled bandwidth times latency. A dashed line for the same kernel rewritten with four independent loads per thread reaches the ceiling at about 10 warps. A dotted line for an arithmetic-bound kernel, whose latencies are short, saturates at about 6 warps. The right panel gives the arithmetic: Little's law, with V100 numbers from Jia et al. (2018); the two ways to get requests in flight, more warps or more independent loads per thread, with Volkov's (2010) observation that fewer warps with more instruction-level parallelism can beat full occupancy; and the contrast with a CPU's out-of-order core.

Details The GPU hides latency by multithreading. When a warp issues a load from DRAM, its next dependent instruction cannot issue for hundreds of cycles; rather than wait, the scheduler marks the warp not ready and issues from another resident warp, and keeps doing so until the first warp's data arrives. No instruction is reordered within a warp, no speculation occurs, and the warp scheduler's bookkeeping is a

scoreboard of which warps have outstanding dependencies. The method works exactly as long as there is always some ready warp, and Little's law says how many warps that takes. A memory system delivering bandwidth BW with latency L has $BW \times L$ bytes in flight when it is fully busy. For a V100, 900 GB/s of HBM2 bandwidth and a DRAM latency that Jia, Maggioni, Staiger and Scarpazza (2018) measured at about 375 cycles when the address translation hits in the TLB and 1,029 cycles when it misses, which at the 1.38 GHz clock of the PCIe card they measured is 0.27 to 0.75 microseconds, give 245 to 670 KB outstanding across the chip, or about 3 to 8 KB per SM. If each request moves one 128-byte line, that is 25 to 60 loads in flight per SM (derived). A kernel in which every thread issues one load and then waits for it needs 25 to 60 warps resident per SM to keep the memory busy, which on a 64-warp SM is an occupancy of 40 to 95 percent, and the solid line on the plot is drawn with its knee at 40.

The alternative to more warps is more independent loads per warp. A thread that issues four loads to independent addresses before using any of them has four requests in flight at once, and a quarter as many warps are needed to reach the same bytes in flight: the dashed line's knee at 10 warps. This is instruction-level parallelism, and the compiler exposes it when the source code permits (loads hoisted above their uses, loops unrolled so that several iterations' loads are issued together). Vasily Volkov's GTC 2010 talk, "Better performance at lower occupancy," made the case with measurements on Fermi: kernels at 25 percent occupancy with enough ILP matched or beat kernels at full occupancy, because the low-occupancy version could afford more registers per thread (the register file is fixed, so fewer threads means more registers each) and used them to keep more values in flight. His 2016 thesis works out the model in full. The lesson for the programmer is that occupancy is a means, not an end: the goal is bytes and instructions in flight, and there are two ways to get them.

The arithmetic-bound case is easier. An FP32 fused multiply-add has a latency of about four cycles on Volta (measured by Jia et al.; 24 cycles on GT200 in Wong et al.'s 2010 measurements), so a dependent chain of FMAs needs only a handful of warps per scheduler to issue every cycle, and the dotted line saturates at about six warps. The plot is illustrative: the knees depend on the GPU, the kernel's access pattern, and how much of the latency the L1 and L2 caches absorb, and real kernels have mixed behavior. The shape, a linear rise to a plateau whose position is set by bandwidth times latency divided by the bytes each warp keeps in flight, is general.

The contrast with the CPU is the one the deck returns to on slide 13. An out-of-order core also keeps many memory requests in flight, but it finds them inside one thread's instruction stream, by renaming registers, buffering a window of a hundred or more instructions, predicting branches and reordering issue. Those structures cost area and energy for every instruction executed. The GPU asks the programmer for the parallelism instead, in the form of many threads, and spends the saved area on lanes and registers. The register file is the limit this choice runs into: 65,536 registers per SM from Kepler on, divided among 2,048 threads at full occupancy, is 32 registers per thread, and a thread that needs more forces a lower occupancy, which is exactly the trade Volkov's measurements explored.

Little's law: bytes in flight $= BW \times L$. V100: 900 GB/s $\times$ (375 to 1,029 cycles/1.53 GHz) $= 900 \times 10^9 \times (0.245 \text{ to } 0.672) \times 10^{-6}$ s $\approx$ 220 to 605 KB; per SM, divided by 80: 2.8 to 7.6 KB; in 128-byte requests: 22 to 59 loads in flight per SM. Required warps $\approx$ loads in flight per SM $\div$ independent loads per warp. Registers per thread at full occupancy: $65{,}536/2{,}048 = 32$ (Kepler through Volta); at half occupancy, 64.

Terms

latency hiding Keeping the arithmetic lanes busy while some warps wait on memory, by issuing from other resident warps; the GPU's substitute for large caches and out-of-order execution.

Little's law The identity that the average number of items in a system equals the arrival rate times the

average time each spends there; applied to memory, bytes in flight equal bandwidth times latency.

memory latency The time from issuing a load to receiving the data; on V100 about 375 cycles for a DRAM access with a TLB hit and 1,029 with a TLB miss (Jia et al. 2018), about 440 cycles on GT200 (Wong et al. 2010).

bytes in flight The total data requested but not yet returned at one moment; what must reach bandwidth times latency for the memory system to be fully used.

thread-level parallelism Independent work supplied by having many threads resident, so that some are always ready to issue; TLP.

instruction-level parallelism Independent instructions within one thread that can be in flight at once, such as several loads issued before any is used; ILP.

scoreboard The per-warp record of outstanding dependencies the scheduler consults to decide whether a warp's next instruction is ready.

out-of-order execution A CPU technique that issues independent instructions from a window of one thread's instruction stream as their operands become ready, finding parallelism the GPU instead asks the programmer to supply.

TLB Translation lookaside buffer: the cache of virtual-to-physical address translations; a miss adds hundreds of cycles to a GPU memory access.

memory-bound A kernel whose run time is set by bytes moved divided by bandwidth; its throughput rises with warps in flight until bandwidth is saturated.

arithmetic-bound A kernel whose run time is set by operations divided by peak rate; it needs only enough warps to cover the short latency of the arithmetic pipeline.

Intuition

A memory system is a long conveyor belt: to keep it full you must have as many parcels on it as fit between the loading end and the unloading end, no matter how fast you can pick them off. Warps are workers who each put one parcel on the belt and then wait for it; the only question is whether you hire more workers or teach each one to carry four parcels.

5.3 Slide 13: Energy is the real constraint: why the design is what it is

After Dennard scaling ended (about 2005), performance per watt decided the design; moving data costs more than math.

What the slide shows A horizontal bar chart on a logarithmic axis of energy per operation in picojoules, at 40 nm, after Keckler, Dally, Khailany, Garland and Glasco (*IEEE Micro*, 2011): reading 64 bits from an 8 KB SRAM, 14 pJ; a double-precision fused multiply-add, 50 pJ; moving 64 bits over 10 mm of on-chip wire, about 154 pJ; reading 64 bits from DRAM, about 2,500 pJ. The right panel draws four consequences: amortize instructions (SIMT), keep data close (large register files, shared memory, reuse within a block; a 64-bit DRAM access costs about fifty FMAs and the ratio widens with every process node), go wide and slow (more lanes at 0.7–1.5 GHz instead of fewer at 3–4 GHz; dark silicon), and move memory closer and specialize (HBM2 on an interposer, slide 20; tensor cores, slide 23).

Details Dennard scaling is the rule, stated by Robert Dennard and colleagues in 1974, that as transistors shrink by a factor k their voltage and current shrink by the same factor, so a chip of the same area and clock

uses the same power while holding k^2 times as many transistors, and the transistors switch faster. For three decades this is what made each generation of processors faster at constant power. Around 2005 it stopped: threshold voltages could not fall further without leakage currents dominating, so supply voltage stalled, and clock speed stalled with it because every increase in frequency now raised power with nothing to offset it. From then on the power budget of a chip, a few hundred watts for a card and a hundred or so for a processor socket, was a fixed quantity to be spent, and performance per watt became the measure of an architecture. The GPU's design can be read entirely as a response to this, and Keckler et al.'s 2011 paper, written by the architects of Fermi and Kepler, makes the argument with the numbers.

Their tables give the energy of elementary operations at the 40 nm node of 2010 and projections for 10 nm. A double-precision fused multiply-add costs about 50 pJ (projected to fall to 8.7 pJ). Reading a 64-bit word from a small, 8 KB SRAM costs 14 pJ. Moving a bit one millimeter along an on-chip wire costs about 240 fJ, so moving a 256-bit operand set across 10 mm of a large die costs about 310 pJ by the paper's own count (which assumes half the bits toggle), or about 77 pJ per 64-bit word, more than the arithmetic on it; the deck's bar uses that 77 pJ figure. The DRAM interface and access together cost 28 to 45 pJ per bit, so a 64-bit word from DRAM costs roughly 1.8 to 2.9 nJ (the chart's bar is drawn at 2.5 nJ), fifty times the FMA; the paper's own summary is that fetching one instruction's 256 bits of operands from DRAM costs more than 10 nJ, two hundred times the arithmetic they feed. On these numbers a Fermi GPU of the day spent about 225 pJ per double-precision floating-point operation all told, against roughly 1.7 nJ for a contemporary Westmere CPU, and the paper set 20 pJ per FLOP as the target an exascale machine would need. Two cautions about the figures: they are from Keckler et al.'s Tables 1 and 2 and the wire and DRAM entries on the chart are scaled to 64 bits from the per-bit values, which the chart's title notes; and a different set of numbers in wide circulation, 20 pJ per FMA, 26 pJ for a 256-bit on-chip bus transfer and 16 nJ for a DRAM read, comes from William Dally's later talks on 28 nm hardware, not from this paper.

Four design rules follow. First, amortize the instruction. Fetching, decoding and scheduling an instruction is a sequence of SRAM reads and wire traversals whose energy is comparable to or larger than the arithmetic it controls; a scalar core pays it for every operation, and SIMT pays it once per 32 (slide 11). The deck's phrase is that the control energy that dominates a scalar core becomes a rounding error. Second, keep data close. The cheapest storage is the one nearest the lanes, which is the register file, so GPUs have register files larger than their L1 caches (256 KB of registers against 128 KB of combined L1 and shared memory per Volta SM), give the programmer shared memory to manage reuse explicitly, and reward algorithms that read each DRAM word once and compute on it many times (slide 15). Third, go wide and slow. Since dynamic energy per operation scales with the square of the supply voltage and frequency scales roughly linearly with it, two lanes at half the clock do the same work for less energy than one lane at full clock; the GPU runs thousands of lanes at 0.7–1.5 GHz where a CPU runs a few cores at 3–4 GHz. Esmaeilzadeh and colleagues' 2011 ISCA paper, "Dark silicon and the end of multicore scaling," made the complementary point that at a fixed power budget only a fraction of a chip's transistors can switch at once, so general-purpose parallelism alone cannot use the area, and specialized units that do more per joule are the way to light more of it. Fourth, move the memory closer and specialize the datapath. HBM2 stacks DRAM dies beside the GPU on a silicon interposer and connects them with thousands of short wires at a low clock, cutting the per-bit interface energy that dominated the DRAM entry in the table (slide 20); a tensor core does 64 multiply-adds per instruction with operands that move once from registers to a fixed adder tree (slide 23), which is instruction amortization and data locality at once.

Keckler et al. (2011), 40 nm: $E_{\text{DFMA}} \approx 50$ pJ; $E_{\text{SRAM, 64 b}} \approx 14$ pJ; $E_{\text{wire}} \approx 240$ fJ/bit/mm, so $E_{\text{64 b, 10 mm}} \approx 240 \times 10^{-3} \times 64 \times 10 \approx 154$ pJ; $E_{\text{DRAM}} \approx (8\text{–}15) + (20\text{–}30)$ pJ/bit, so $E_{\text{DRAM, 64 b}} \approx 1.8$ to 2.9 nJ. Ratios: $E_{\text{DRAM, 64 b}}/E_{\text{DFMA}} \approx 2{,}500/50 = 50$; operands for one instruction (256 bits) from DRAM, "more than 10 nJ," about 200× the arithmetic. Control amortization under SIMT: $E_{\text{fetch+decode}}/32$ per operation. Dynamic power $P \propto CV^2 f$ with $f \propto V$ near the operating point, so halving f and lowering V accordingly cuts energy per operation by more than half.

Terms

Dennard scaling The 1974 observation that shrinking a transistor's dimensions, voltage and current together keeps power density constant, so smaller transistors gave faster chips at the same power; it ended around 2005 when supply voltage stopped falling.

power wall The condition after Dennard scaling in which a chip's power budget, not its transistor count, limits performance, so clock speeds stall and performance per watt decides the design.

energy per operation The joules spent to perform one elementary action (an arithmetic operation, a memory read, a wire traversal); the quantity Keckler et al. tabulate and the deck treats as the architecture's constraint.

picojoule 10^{-12} joule; the unit of the energies on the slide. A nanojoule is 1,000 pJ.

fused multiply-add The operation $a \times b + c$ with a single rounding, the basic arithmetic instruction of a GPU lane; counted as two floating-point operations in peak figures.

data movement Transferring operands between storage and arithmetic units over wires, buses and memory interfaces; on the 2011 numbers it costs more energy than the arithmetic at every level above the register file.

dark silicon Esmaeilzadeh et al.'s (2011) term for the fraction of a chip's transistors that must stay idle at a fixed power budget; the argument for specialized units over more general cores.

performance per watt Operations per second divided by power, or equivalently the reciprocal of energy per operation; the metric that replaced clock speed after 2005.

HBM2 High Bandwidth Memory, second generation (JEDEC JESD235A, January 2016): DRAM dies stacked and connected to the processor through a silicon interposer with a 1,024-bit interface per stack, up to 256 GB/s and 8 GB per stack; used by Pascal and Volta (slide 20).

interposer A thin silicon substrate on which a processor die and memory stacks sit side by side, carrying thousands of short wires between them; the packaging behind HBM.

Intuition

Computing is cheap and shipping is expensive: an FMA is a stamp, a DRAM access is a courier. Every rule of GPU design (issue to 32 lanes at once, hoard registers, tile through shared memory, stack the DRAM next to the die, build a unit that multiplies whole matrices) is a way of posting more letters per courier trip.

6 The memory system and the roofline (slides 14–15)

6.1 Slide 14: The memory system: wide DRAM, coalescing, small caches

Bandwidth comes from wide buses and many requests in flight; the programmer turns 32 addresses into one transaction.

What the slide shows A coalescing diagram at the top left. In the first row, 32 lanes each read four bytes at consecutive addresses (`x[i]` for thread i); an arrow leads to one 128-byte segment, one transaction with every byte used. In the second row the same 32 lanes read with a stride of 32 elements (`x[32*i]`); the arrow leads to 32 separate segments, 32 transactions of which the warp uses one byte in 32. A panel below, "What changed, 2006 to 2017," gives the DRAM generations (GDDR3 at 86 GB/s on a 384-bit bus for G80; GDDR5 at 144–288 GB/s for Fermi and Kepler; HBM2 at 720–900 GB/s on a 4,096-bit bus for Pascal and Volta: wider and slower per pin, not faster) and the caches (none for global loads on G80; Fermi's L1 per SM and 768 KB L2 with ECC; Volta's 6 MB of L2). The right panel lists the rules the hardware imposes: request granularity, shared-memory banks, caches that are small per thread (38 bytes of L2 per resident thread on V100), and ECC.

Details The GPU's memory system is built for bandwidth and tolerates latency, the reverse of a CPU's priorities, and the two design choices that follow are a very wide DRAM interface and a request path that merges the loads of a warp. The width first. G80's six 64-bit GDDR3 channels made a 384-bit bus; at 900 MHz double data rate that is $384/8 \times 1.8$ GB/s $= 86.4$ GB/s. Fermi's C2050 kept 384 bits and moved to GDDR5 at 3 Gb/s per pin for 144 GB/s; Kepler's K40 reached 288 GB/s on the same width at 6 Gb/s. Pascal's GP100 changed the technology: four HBM2 stacks, each with a 1,024-bit interface, give a 4,096-bit bus running at only 1.4 Gb/s per pin for 720 GB/s, and Volta's V100 raised the pin rate to reach 900 GB/s. The JEDEC HBM2 standard (JESD235A, January 2016) specifies eight independent 128-bit channels per stack and allows up to 256 GB/s and 8 GB per stack; P100 ran its stacks at 180 GB/s each. The deck's summary is exact: the bus got wider and slower per pin, not faster, because a slow wide interface over an interposer costs less energy per bit than a fast narrow one over a circuit board (slide 13), and because latency was never the goal.

Coalescing is how a warp's loads use that width. A warp load instruction carries 32 addresses, and the load/store unit sorts them into the aligned memory segments they touch: 128-byte lines, served from Fermi on as 32-byte sectors. Each distinct segment becomes one request to the L1, L2 and, on a miss, DRAM. When thread i reads the 4-byte element `x[i]`, the 32 addresses span exactly $32 \times 4 = 128$ bytes, one aligned line, and the warp's load is one transaction with every returned byte used. When thread i reads `x[32*i]`, consecutive lanes are 128 bytes apart, each lands in its own line, and the warp issues 32 transactions; each returns at least a 32-byte sector of which four bytes are used, so the useful fraction of the traffic is one eighth at best and one thirty-second if whole lines are fetched. The bandwidth the kernel sees is the hardware bandwidth divided by that waste. The rule for the programmer is therefore consecutive threads, consecutive addresses, and its data-structure corollary is structure-of-arrays rather than array-of-structures: a warp reading the x field of 32 particles stored as `struct {float x, y, z;}` touches every twelfth byte, while a separate array of x values is read in one line. G80's rules were stricter (alignment and in-order lanes were required for coalescing); Fermi's L1 relaxed them to any pattern within a line, and later generations kept Fermi's rule.

Shared memory has its own version of the rule. It is built from 32 banks, each 4 bytes wide, with consecutive words in consecutive banks; a warp instruction that touches 32 different banks completes in one cycle, and one in which two lanes address different words in the same bank serializes into two. The classic case is a column read of a 32-by-32 tile stored row-major: every lane hits bank 0, a 32-way conflict. Padding the row length to 33 words moves each row's start to the next bank and the conflict disappears. Broadcast is the exception: lanes reading the same word of the same bank are served together.

The caches arrived with Fermi and remained small by CPU standards for a reason the slide quantifies. A V100 has 6 MB of L2 shared by 80 SMs; with 2,048 threads resident per SM it holds $6 \times 2^{20}/163{,}840 \approx 38$ bytes per resident thread (derived), less than one line each. The caches do not hold working sets; they catch

the reuse within a few microseconds (the halves of a line another warp wants, the second access to a row of a matrix) and smooth irregular access. Reuse over longer spans must be explicit, in registers and shared memory, which is what a tiled matrix multiply does: load a tile into shared memory once, read it from there 32 or 64 times. G80 had no cache for ordinary global loads at all, only the texture and constant caches inherited from graphics; Fermi added the configurable L1 and the 768 KB L2 (slide 16); Kepler and Maxwell grew the L2 to 1.5 and 3 MB; Volta merged L1 and shared memory into a 128 KB array per SM and reached 6 MB of L2 (slide 21). Fermi also introduced ECC on the register files, caches and DRAM, at a cost in bandwidth and capacity on GDDR5 (the C2050's 3 GB became 2.625 GB with ECC on); HBM2 carries ECC natively, so on Pascal and Volta it is free. The supercomputers of slide 26 made ECC a condition of purchase, which is one reason the compute dies and the consumer dies diverged.

Bus bandwidth $=$ width (bits)$/8 \times$ data rate per pin: G80, $384/8 \times 1.8$ Gb/s $= 86.4$ GB/s; P100, $4{,}096/8 \times 1.4$ Gb/s ≈ 720 GB/s; V100, $4{,}096/8 \times 1.75$ Gb/s ≈ 900 GB/s. Coalesced warp load of 4-byte elements: $32 \times 4 = 128$ bytes, one line, one transaction. Stride-32 load: 32 transactions, useful fraction $4/32 = 1/8$ per sector or $4/128 = 1/32$ per line. L2 per resident thread on V100: $6\text{ MB}/(80 \times 2{,}048) \approx 38$ B (derived). Bank of a 4-byte word at address a: $(a/4) \bmod 32$; a row-major 32×33 tile puts column c of row r in bank $(33r + c) \bmod 32 = (r + c) \bmod 32$, distinct for distinct r.

Terms

coalescing The merging of a warp's 32 memory addresses into the minimum set of aligned segment requests; complete when consecutive threads read consecutive words, absent when they stride.

memory transaction One request from the SM to the memory hierarchy for an aligned segment (a 32-byte sector or a 128-byte line); the unit in which bandwidth is consumed.

sector A 32-byte aligned unit of a cache line, the granularity of requests to L2 and DRAM from Fermi on.

cache line A 128-byte aligned unit of memory as the caches track it; four sectors.

structure of arrays A layout with one array per field, so that a warp reading one field reads consecutive addresses; the coalescing-friendly alternative to an array of structures.

GDDR Graphics double data rate DRAM: discrete memory chips on the circuit board with a wide, fast per-pin interface; GDDR3 on G80 and GT200, GDDR5 on Fermi through Maxwell.

memory bus width The number of data wires between GPU and DRAM: 384 bits on G80 and Fermi, 512 on GT200, 4,096 on the HBM2 parts.

bank conflict Two or more lanes of a warp addressing different words in the same shared-memory bank, which serializes the access; avoided by padding or by permuting the layout.

broadcast (shared memory) The case in which several lanes read the same word; served in one access with no conflict.

L2 cache The chip-wide cache shared by all SMs and the memory controllers: 768 KB on Fermi, 1.5 MB on Kepler, 3 MB on Maxwell, 4 MB on Pascal, 6 MB on Volta.

ECC Error-correcting code on registers, caches and DRAM, introduced on Fermi; required by HPC centers, costly on GDDR5, free on HBM2.

texture cache The read-only cache serving texture fetches, inherited from graphics; on G80 the only cache for data from DRAM.

Intuition

> The memory system is a freight railway with very wide wagons and a long line. Thirty-two passengers who board the same wagon travel for the price of one; thirty-two who each insist on a separate wagon pay thirty-two fares, and the train is no faster for it.

6.2 Slide 15: The roofline: which kernels a GPU can feed

Attainable FLOP/s is the smaller of peak arithmetic and bandwidth times intensity; most kernels sit on the slope.

What the slide shows A log-log plot of attainable GFLOP/s against arithmetic intensity in FLOP per byte of DRAM traffic, with four rooflines derived from vendor peaks: a dashed line for V100 tensor cores (125 TFLOP/s at 900 GB/s), a solid line for V100 FP32 (15.7 TFLOP/s), one for P100 FP32 (10.6 TFLOP/s at 720 GB/s) and a dotted one for the Fermi C2050 (1.0 TFLOP/s at 144 GB/s). Each roofline rises along a slope equal to the bandwidth and turns flat at the peak; the corner is the ridge point. Four kernel points at typical intensities sit on the plot: SAXPY near 0.17 FLOP/B, a seven-point stencil near 1, a large FFT near 8, and tiled single-precision matrix multiply far to the right at the V100 FP32 roof. The right panel explains the ridge points (17 FLOP/B for V100 FP32, 139 for its tensor cores, 7 for Fermi), the SAXPY result (150 GFLOP/s, 1% of peak, and correct), why matrix multiply reaches the roof, and how every generation raised the ridge, with Williams, Waterman and Patterson (2009) as the source of the model.

Details The roofline is Williams, Waterman and Patterson's 2009 model (*Communications of the ACM*) for the performance a kernel can attain on a machine described by two numbers, its peak arithmetic rate and its memory bandwidth. A kernel is described by one number, its arithmetic intensity I: the floating-point operations it performs per byte it moves to or from DRAM. The attainable rate is then $\min(\text{peak}, BW \times I)$. For small I the kernel cannot do arithmetic faster than the memory delivers operands, and its rate is the bandwidth times its intensity, the sloped part of the plot; for large I the arithmetic units are the limit, the flat roof. The ridge point, where the two meet, is peak/BW: a kernel with intensity below it is bandwidth-bound and gains nothing from more or faster lanes, one above it is compute-bound and gains nothing from more bandwidth. The model ignores latency, cache effects and instruction overheads, which is why it gives an upper bound; its value is that both inputs are on the datasheet and the kernel's intensity can be counted.

The ridge points on the slide are derived from vendor peaks. V100 FP32: $15{,}700/900 = 17.4$ FLOP/B. V100 tensor cores: $125{,}000/900 = 139$. P100 FP32: $10{,}600/720 = 14.7$. Fermi C2050: $1{,}030/144 = 7.2$. G80 (not plotted): $346/86.4 = 4.0$. Reading them in order is the point of the panel's last item. Over the decade peak arithmetic grew about 45 times and bandwidth about 10 times, so the ridge moved right by a factor of four, and the tensor cores moved it another eight: a kernel whose intensity was comfortably above Fermi's ridge of 7 is well below V100's 17 and nowhere near 139. Each generation turned more kernels into bandwidth-bound ones, and the architectural responses (larger register files, more shared memory, HBM2, and the pressure to raise intensity by fusing kernels) are on slides 13, 14 and 20.

SAXPY is the worked example of the sloped region. For each element it reads x_i and y_i and writes y_i, 12 bytes, and performs one multiply and one add, 2 FLOPs, so $I = 2/12 \approx 0.167$ FLOP/B. On a V100 the attainable rate is $900 \times 0.167 \approx 150$ GFLOP/s, about one percent of the 15.7 TFLOP/s peak. The slide's remark that this is correct, not a bug, is the roofline's main lesson for a student who has just computed the one percent: no amount of tuning raises a SAXPY kernel above its slope, and the kernel is well written when it reaches most of the bandwidth. The other points are typical rather than measured. A seven-point stencil on a 3D grid reads about seven values and writes one per eight or so FLOPs, and with caching of neighboring points lands near 1 FLOP/B; a large FFT moves each element a few times through $O(\log n)$ stages and lands in the high single digits; the exact values depend on the implementation, and the plot labels them as typical.

Matrix multiply is the kernel that reaches the roof, and the reason is that its intensity is under the programmer's control. A naive $n \times n$ multiply that reads its operands from DRAM for every multiply-add has an intensity near $2/8$ FLOP/B in single precision, like SAXPY. A tiled implementation loads a $T \times T$ tile of each operand into shared memory and registers once and uses each loaded element T times, so the intensity is about $T/4$ FLOP/B for 4-byte elements (derived): with $T = 64$, 16 FLOP/B; with register tiling on top, enough to pass the V100 ridge of 17 and sit on the roof. This is the property that made dense linear algebra the first GPU success (Volkov and Demmel's 2008 SC paper reached 60 percent of G80's peak in SGEMM by exactly this analysis), that made cuBLAS the library every program calls, and that made deep learning, whose convolutions and attention are matrix multiplies, the workload the architecture was later redesigned around (slide 19). The tensor cores raised the roof by eight and the ridge to 139, which only matrix multiply with large tiles can reach (slide 23); everything else in a network stays on the slope.

Roofline: attainable FLOP/s $= \min\big(\text{peak},\ BW \times I\big)$, with I in FLOP per byte of DRAM traffic. Ridge point $I^* = \text{peak}/BW$: G80 $346/86.4 = 4.0$; C2050 $1{,}030/144 = 7.2$; P100 $10{,}600/720 = 14.7$; V100 FP32 $15{,}700/900 = 17.4$; V100 tensor $125{,}000/900 = 139$ (all derived from vendor peaks). SAXPY: $I = 2$ FLOP$/12$ B $= 0.167$; on V100, $900 \times 0.167 = 150$ GFLOP/s $= 1\%$ of peak. Tiled SGEMM with $T \times T$ tiles of 4-byte elements: each loaded element is used T times, $I \approx 2T^3/(2 \cdot 4T^2) = T/4$ FLOP/B; $T = 64$ gives 16, $T = 128$ gives 32.

Terms

roofline model Williams, Waterman and Patterson's (2009) bound on attainable performance: the minimum of peak arithmetic rate and memory bandwidth times arithmetic intensity, plotted on log-log axes.

arithmetic intensity A kernel's floating-point operations per byte moved between DRAM and the chip; the horizontal axis of the roofline.

ridge point The intensity at which a roofline turns from sloped to flat, equal to peak divided by bandwidth; kernels below it are bandwidth-bound, above it compute-bound.

bandwidth-bound A kernel whose intensity is below the ridge point, so its rate is bandwidth times intensity and more arithmetic units do not help.

compute-bound A kernel whose intensity is above the ridge point, so its rate is limited by the arithmetic units and more bandwidth does not help.

attainable performance The roofline's upper bound on a kernel's rate; a well-tuned kernel reaches 60–80% of it, and the gap is latency, overhead and imperfect overlap.

stencil A kernel that updates each grid point from a fixed pattern of neighbors (seven points in the 3D case); a bandwidth-bound workload with intensity near 1 FLOP/B.

SGEMM Single-precision general matrix multiply, the BLAS level-3 routine; the kernel whose intensity grows with tile size and that reaches the roofline.

tiling Partitioning a computation into blocks that fit in shared memory or registers so that each loaded element is reused many times; the technique that raises a kernel's intensity. Kernel fusion, combining consecutive kernels so intermediates stay on chip, is the same idea across kernels.

cuBLAS NVIDIA's CUDA implementation of the BLAS routines, shipped with CUDA 1.0; the library through which most programs reach the roofline.

Intuition

The roofline is a diagnosis chart. Find your kernel's intensity on the horizontal axis, look up, and the line above it tells you whether to work on bytes or on FLOPs; a SAXPY at one percent of peak is healthy, and a matrix multiply at one percent is ill.

7 The generations: Tesla, Fermi, Kepler, Maxwell, Pascal (slides 16–21)

7.1 Slide 16: Tesla to Fermi (2008–2010): double precision, caches, ECC

GT200 made the GPU usable for science; Fermi made it a general processor and put it at the top of the TOP500.

What the slide shows Three panels. Top left, GT200 (the GeForce GTX 280 and the Tesla C1060, June 2008): 240 streaming processors in 30 SMs, 1.4 billion transistors on 65 nm, a 576 mm^2 die; the first FP64 unit (one per SM, 78 GFLOP/s double against 933 single on the C1060), twice the registers of G80, 4 GB of GDDR3 at 102 GB/s on the Tesla card, atomic operations on shared memory. Bottom left, Fermi, the GF100 die (Tesla C2050 and C2070, 2010): 512 CUDA cores in 16 SMs of 32, of which the C2050 shipped 448, 3.0 billion transistors on 40 nm; a cache hierarchy, ECC, IEEE 754-2008 fused multiply-add, FP64 at half the FP32 rate (515 against 1,030 GFLOP/s), a unified 40-bit address space, two warp schedulers per SM, concurrent kernels. The right panel argues that Fermi is the pivot of the whole history: it settled the SM template, it made GPUs eligible for supercomputers, it put a price on the general-purpose features, and its consumer version, the GeForce GTX 480 at 250 W, taught the lesson that produced Maxwell.

Details GT200 was the second CUDA-capable architecture and the first on which scientific computing was more than a demonstration. The GeForce GTX 280 was announced on June 16, 2008 and on sale the next day†; the Tesla C1060, the same die without display outputs and with 4 GB of memory, followed as the compute product. The architecture kept G80's organization (clusters of SMs sharing texture units, eight scalar processors per SM, a warp of 32 threads issued over four clocks) and scaled it: 30 SMs in 10 clusters of three, 240 processors, 1.4 billion transistors on TSMC's 65 nm process. The 576 mm^2 die area is a figure reported before launch from NVIDIA's own briefings rather than from a datasheet, and third-party caliper measurements of the package run larger; the deck's table marks the G80, GT200 and GF100 areas as third-party for this reason, while the later figures (GK110 551, GM200 601, GP100 610, GV100 815 mm^2) are NVIDIA's own, published in the Pascal and Volta whitepapers.

Three things made GT200 usable for science. Double precision arrived as one 64-bit fused multiply-add unit per SM, so 30 FP64 operations per clock against 240 single-precision multiply-adds (plus the dual-issue multiply that gives the 933 GFLOP/s figure); the C1060's 78 GFLOP/s of double precision was a twelfth of its single-precision rate, slow by any standard but enough for ports of molecular dynamics, lattice QCD and seismic imaging to begin. The register file doubled, from 8,192 to 16,384 32-bit entries per SM, which let more threads stay resident (1,024 per SM against 768) and hid more memory latency (slide 12). And the memory system widened to 512 bits of GDDR3 at 800 MHz, 102 GB/s on the C1060† (the GTX 280 ran its memory faster). The board power of 187.8 W for the C1060 is the datasheet figure†.

Fermi, announced as an architecture at the GPU Technology Conference on September 30, 2009 with the whitepaper, and shipping in volume as the Tesla 20-series in May 2010†, is the design the rest of this lecture treats as the template. Its SM (slide 10) has 32 CUDA cores in two groups of 16, two warp schedulers that each issue one warp instruction per clock, 16 load/store units, four special-function units, a 32,768-entry register file (128 KB), and 64 KB of on-chip SRAM configurable as 48 KB of shared memory and 16 KB of L1 cache or the reverse. The die holds 16 such SMs, 512 cores in all, but the first Tesla products shipped with 14 enabled (448 cores) at 1.15 GHz, which gives the C2050's 1.03 TFLOP/s single and 515 GFLOP/s double:

double precision at exactly half the single rate, because each SM does 16 FP64 fused multiply-adds per clock. Behind the SMs sits a 768 KB L2 cache shared by all of them, the first unified cache for global loads on an NVIDIA GPU, with ECC on the caches, the registers, the shared memory and the 3 GB of GDDR5 (2.625 GB with ECC enabled, since the check bits live in the same memory) at 144 GB/s†. Fermi also brought IEEE 754-2008 fused multiply-add in both precisions, a unified 40-bit virtual address space that made C++ pointers meaningful across local, shared and global memory, and the ability to run several kernels at once.

The right panel's argument is worth stating in full. First, the SM template: register file, warp schedulers, FP32 lanes, load/store units, special-function units, and an on-chip SRAM split between shared memory and L1. Every later generation changed the counts (slide 17's bar chart) and never replaced the structure. Second, eligibility for supercomputers: HPC centers would not buy accelerators without ECC, because a soft error in a week-long run on thousands of boards is a certainty rather than a risk, and would not buy them without competitive double precision. Fermi had both, and in November 2010 Tianhe-1A, with 7,168 Tesla M2050s (the passively cooled server version of the C2050), was first on the TOP500 at 2.57 PFLOP/s on HPL†; TSUBAME 2.0 in Tokyo (Tesla M2050) and Nebulae in Shenzhen (Tesla C2050) used the same Fermi die (slide 26). Third, the price of generality: caches, ECC and full-rate FP64 cost die area and power that a game never uses. From Kepler on, NVIDIA built separate dies, the compute dies (GK110, GP100, GV100) with these features and the consumer dies (GK104, GM204, GP104) without most of them. Fourth, power: the GeForce GTX 480 of March 2010, the consumer Fermi, drew 250 W† and was loud and hot by the standards of the day, which is the background to Maxwell's efficiency turn on the next slide.

Terms

GT200 NVIDIA's second CUDA architecture (2008): 240 scalar processors in 30 SMs, the first FP64 unit, 1.4 billion transistors on 65 nm; the GeForce GTX 280 and the Tesla C1060.

Tesla (product line) NVIDIA's compute products from 2007 (C870) to 2017 (V100): the same die as a GeForce or a dedicated compute die, without display outputs, with more memory and, from Fermi on, ECC. Not to be confused with the Tesla architecture of G80 and GT200.

FP64 unit A pipeline that performs a 64-bit (double-precision) fused multiply-add; one per SM on GT200, 16 per SM on Fermi's compute parts, 64 per SMX on GK110, 32 per SM on GP100 and GV100.

Fermi NVIDIA's third CUDA architecture (GF100, 2010): 512 cores in 16 SMs of 32, caches, ECC, fused multiply-add, half-rate FP64; the SM template of the decade.

ECC Error-correcting code memory: extra check bits that detect and correct single-bit errors in DRAM, caches and registers; a requirement for supercomputers, first on Fermi's Tesla products.

fused multiply-add A single instruction computing $a \times b + c$ with one rounding at the end, as IEEE 754-2008 specifies; Fermi's arithmetic is built on it, and peak FLOP/s counts it as two operations.

L2 cache A cache shared by all SMs, between them and DRAM; 768 KB on Fermi, 6 MB on Volta. Also the point at which ECC and atomics on global memory are resolved.

unified address space One virtual address range covering local, shared and global memory, so a pointer can be passed without knowing which memory it names; 40 bits on Fermi, 49 bits with page faulting on Pascal.

concurrent kernels Running several kernels from one application at the same time on different SMs; introduced on Fermi, extended by Hyper-Q on Kepler.

compute die A GPU die designed for the Tesla line with full-rate FP64, ECC and large caches (GK110, GP100, GV100), as opposed to the consumer dies that omit them.

TOP500 The twice-yearly list of the 500 fastest supercomputers ranked by the HPL benchmark; GPU-accelerated systems first reached the top in November 2010.

HPL The High-Performance Linpack benchmark, a dense LU factorization whose achieved FLOP/s ranks the TOP500; a compute-bound kernel close to the roofline's ridge.

Intuition

GT200 added the one thing scientists could not do without (double precision), and Fermi added the two things their machine rooms could not do without (error correction and a real memory hierarchy). After that the question was never whether a GPU could be a scientific processor, only how much of the die to spend on being one.

7.2 Slide 17: Kepler and Maxwell (2012–2015): wider SMs, then efficiency

Kepler put 192 lanes in an SM and 18,688 GPUs in Titan; Maxwell cut back to 128 lanes and doubled perf per watt.

What the slide shows A bar chart on the left: FP32 lanes per SM for the seven compute dies, with the number of SMs per die written above each bar. G80 and GT200 have 8 lanes per SM (16 and 30 SMs), Fermi 32 (16 SMs), Kepler's GK110 192 (15 SMX), Maxwell's GM200 128 (24 SMs), Pascal's GP100 and Volta's GV100 64 (56 and 80 SMs). A caption notes that lanes per SM went up and came back down, while total lanes and transistors per die grew every generation and transistors per lane fell from Fermi (about 5.9 million) to Kepler (2.5 million) before rising again with Pascal (4.0 million); the bar labels give the SMs enabled on the named product (14 for the C2050). Two panels on the right: Kepler GK110 (Tesla K20X in 2012, K40 in 2013) and Maxwell GM204 and GM200 (2014–15).

Details Kepler's compute die, GK110, was announced with the Tesla K20X at SC12 on November 12, 2012†; the K40 followed on November 18, 2013 and the dual-die K80 on November 17, 2014†. The die has 7.1 billion transistors on TSMC's 28 nm process and 551 mm^2 (NVIDIA's figure, from the Pascal whitepaper's comparison table); it carries 15 SMX units of 192 FP32 lanes, 2,880 in all, with the K20X shipping 14 SMX (2,688 lanes) and the K40 all 15. Each SMX has 64 FP64 units (a third of the FP32 rate), 32 special-function units, 32 load/store units, four warp schedulers each with two dispatch units, a 65,536-entry register file (256 KB), the Fermi-style 64 KB shared/L1 array (now configurable 48/16, 16/48 or 32/32) plus a 48 KB read-only data cache, and room for 2,048 resident threads in 64 warps with up to 255 registers per thread. The L2 grew to 1,536 KB. The K40's peak figures at its 745 MHz base clock are 4.29 TFLOP/s single and 1.43 double, and at its 875 MHz boost clock 5.04 and 1.68†; the deck's table quotes the base figures and says so. It carries 12 GB of GDDR5 at 288 GB/s on a 384-bit bus and has a 235 W board power†. The K80 put two GK210 dies (a GK110 variant with twice the register file and shared memory per SMX) on one board: 4,992 lanes, 24 GB, 480 GB/s, 8.74 TFLOP/s single and 2.91 double with boost, 300 W†.

Three Kepler features matter beyond the counts. Dynamic parallelism lets a kernel launch other kernels without returning to the host, so recursive or adaptive algorithms (tree traversal, adaptive mesh refinement) no longer round-trip through the CPU for every level. Hyper-Q gives the GPU 32 hardware work queues instead of one, so several CPU processes or MPI ranks can share one GPU without serializing behind each other; on Titan this turned idle GPU time into throughput for codes that could not fill the device from one rank. GPUDirect RDMA lets a network interface read and write GPU memory directly over PCIe, cutting the CPU memory copy out of GPU-to-GPU communication between nodes. Titan at Oak Ridge, a Cray XK7

with 18,688 K20Xs beside 16-core Opterons, was first on the TOP500 in November 2012 at 17.59 PFLOP/s† (slide 26).

Maxwell is the generation in which the lane count per SM fell and the energy argument of slide 13 became the design brief. The first Maxwell, GM204 in the GeForce GTX 980 (September 2014), had 5.2 billion transistors on 398 mm^2, 16 SMs of 128 lanes and a 165 W board power†; the large die, GM200 (8 billion transistors, 601 mm^2), appeared as the GeForce GTX Titan X, launched at GTC on March 17, 2015, and as the Tesla M40, announced November 10, 2015†. Maxwell's SM (NVIDIA's SMM) splits its 128 lanes into four quadrants, each with its own warp scheduler, instruction buffer and 16,384-entry register file, so that each scheduler feeds 32 lanes it owns rather than competing for a shared pool. The dispatch logic is simpler, the datapaths shorter, and the shared memory is a dedicated 96 KB separate from the L1, which merged with the texture cache. NVIDIA's claim of about twice Kepler's performance per watt on the same 28 nm process is a vendor figure† and is marked as such on the slide; the consumer reviews of the day broadly confirmed the direction.

Maxwell's compute story is the FP64 decision. GM200 has four FP64 units per SM, 96 on the die, so double precision runs at 1/32 of the single rate: the M40's 6.84 TFLOP/s of FP32 ("7 TFLOPS with GPU Boost" in the datasheet) comes with 210 GFLOP/s of FP64†. That made GM200 useless for the HPC codes of slide 26 and ideal for the deep-learning training of slide 19, where FP32 was the working precision and nobody used FP64. The M40, with 12 GB (later 24 GB) of GDDR5 at 288 GB/s and 250 W, was sold for exactly that purpose, and NVIDIA's press release for it measured the gain in AlexNet training time†. The HPC compute die skipped a generation: there was no 28 nm Maxwell successor to GK110, and the next one was Pascal's GP100 on 16 nm (slide 20).

The bar chart's lesson is the one the caption states. The number of lanes a scheduler must keep busy is a design parameter, not a law: Kepler's 192 proved hard to fill (each SMX needed enough independent warps and enough instruction-level parallelism to keep six groups of 32 lanes busy from four schedulers), Maxwell's 128 in four owned quadrants was easier, and Pascal and Volta settled on 64. What grew without interruption was the number of SMs per die (16, 30, 16, 15, 24, 56, 80) and the transistors spent per lane on registers, caches and scheduling.

Terms

Kepler NVIDIA's 2012 architecture; the compute die GK110 has 15 SMX units of 192 lanes, 7.1 billion transistors on 28 nm, and introduced dynamic parallelism, Hyper-Q and GPUDirect RDMA.

SMX Kepler's streaming multiprocessor: 192 FP32 lanes, 64 FP64 units, four warp schedulers with two dispatch units each, 65,536 registers.

dynamic parallelism A Kepler feature allowing a kernel running on the GPU to launch further kernels without host involvement.

Hyper-Q Kepler's 32 hardware work queues, which let several CPU processes or streams submit work to one GPU concurrently instead of serializing on a single queue.

GPUDirect RDMA Direct memory access between a network adapter and GPU memory over PCIe, bypassing a copy through host memory; Kepler and later.

GK210 The Kepler variant on the Tesla K80, with twice the register file and shared memory of GK110 per SMX; two per board.

Maxwell NVIDIA's 2014–15 architecture (GM204, GM200): 128 lanes per SM in four quadrants with their own schedulers and register files, a larger L2, dedicated 96 KB shared memory, FP64 at 1/32 rate; built for performance per watt.

SMM Maxwell's streaming multiprocessor, partitioned into four 32-lane quadrants.

performance per watt Useful operations per joule; the figure of merit that governed GPU design after Dennard scaling ended (slide 13).

boost clock A clock above the base rate that a GPU sustains while within its power and thermal limits; vendors quote peak FLOP/s at base or boost, and comparisons must say which.

Tesla M40 The Maxwell GM200 compute product (2015): 3,072 lanes, 6.84 TFLOP/s FP32, 12 or 24 GB GDDR5, 250 W; sold for deep-learning training, not HPC.

Titan (supercomputer) Oak Ridge's Cray XK7 (2012) with 18,688 Tesla K20Xs, first on the TOP500 in November 2012 at 17.59 PFLOP/s.

Intuition

Kepler asked how many lanes one scheduler could feed and found the answer was fewer than 192; Maxwell asked how little control logic a lane could get by with and found the answer was a quarter-SM of its own. The two generations together bracketed the number, 64 to 128, that every design since has used.

7.3 Slide 18: The generations in one table

Seven compute dies, 2006–2017: the counts change by an order of magnitude; the structure does not.

What the slide shows A single table with seven columns, one per compute die with its representative product beneath it: G80 (GeForce 8800 GTX), GT200 (Tesla C1060), GF100 (Tesla C2050), GK110 (Tesla K40), GM200 (Tesla M40), GP100 (Tesla P100) and GV100 (Tesla V100). The rows: year, process node, transistor count, die area, SMs times FP32 lanes per SM, peak FP32, FP64 and FP16-or-tensor TFLOP/s, memory type, capacity, bandwidth, L2 size, register file per SM, shared memory per SM, board power, and host link. A footnote gives the provenance: vendor specifications for the named products†, with G80 taken as the GeForce 8800 GTX (155 W being the later official figure), the GF100 column as the C2050 as shipped with 14 of 16 SMs, the K40 at its base clock (5.0 and 1.7 TFLOP/s with boost), the G80, GT200 and GF100 die areas as third-party measurements, register files in KB as 32-bit entries times four, and shared memory as the maximum configuration.

Details The table is the deck's reference sheet, and it is best read twice: down a column for a machine, along a row for a trend. Taking the rows in order.

Year and process. The seven dies span eleven years and five process nodes: 90 nm (G80, 2006), 65 nm (GT200, 2008), 40 nm (GF100, 2010), 28 nm for two generations (GK110 in 2012–13 and GM200 in 2015), 16 nm FinFET (GP100, 2016) and the tuned 12 nm FFN variant of it (GV100, 2017). The pause at 28 nm is why Maxwell is an efficiency generation: with no shrink available, the only way to more performance was to spend fewer joules per operation.

Transistors and area. Transistor count grows thirty-fold, from 681 million to 21.1 billion, while die area grows only from about 470 to 815 mm^2. The area figures for the first three dies are third-party measurements (NVIDIA did not publish them); the later four are NVIDIA's own from the Pascal and Volta whitepapers. Every compute die from GT200 on is among the largest chips its foundry made, and GV100 at 815 mm^2 is close to the reticle limit of the lithography tools, about the largest a single die can be (slide 22). The transistors went into more SMs, more and larger register files and caches, FP64 and tensor units, and memory controllers, not into clock speed: the core clocks of these products run from 1.35 GHz (G80) to 1.53 GHz (V100 boost), with the Fermi and Kepler parts lower.

SMs and lanes. The row reads 16×8, 30×8, 14×32, 15×192, 24×128, 56×64, 80×64: 128, 240, 448, 2,880, 3,072, 3,584 and 5,120 FP32 lanes. The GF100 entry is 14 rather than 16 SMs because the C2050

shipped with two disabled; GV100 has 84 SMs on the die and 80 enabled; GP100 has 60 and enables 56. Disabling a few SMs is how a die near the reticle limit is sold at a usable yield.

Peak arithmetic. FP32 rises 45-fold from 0.35 TFLOP/s (G80's multiply-add rate; 0.52 with the dual-issue multiply the marketing counted) to 15.7. FP64 appears on GT200 at 0.08 TFLOP/s, reaches half the FP32 rate on Fermi (0.52), a third on Kepler (1.4 at base clock), collapses to 1/32 on Maxwell (0.21), and returns to half rate on Pascal and Volta (5.3 and 7.8). The third row is the deep-learning row: nothing until Pascal's packed FP16 at 21.2 TFLOP/s, then Volta's tensor cores at 125. Across the table, the FP32 figure for a given product depends on which clock is quoted; the K40 is listed at base (4.3) with its boost figure (5.0) in the footnote, the M40 at its boost figure (6.8, rounded from 6.84), the P100 and V100 at their boost clocks of 1,480 and 1,530 MHz, which is how NVIDIA states them.

Memory. GDDR3 on the first two dies, GDDR5 on the next three, HBM2 on the last two. Capacity grows from 0.75 GB (the 8800 GTX; the Tesla C870 had 1.5 GB) to 16 GB, and to 32 GB on the 2018 V100. Bandwidth grows tenfold, from 86 to 900 GB/s, in two steps: a tripling across the GDDR generations (86, 102, 144, 288, 288) and a near-tripling at the move to HBM2 (720, 900). The bandwidth row grows more slowly than the FP32 row, which is the point of slide 3: bytes per FLOP fell from 0.25 to 0.06 (derived), and the roofline ridge (slide 15) rose accordingly.

Caches and on-chip storage. There is no L2 on G80 or GT200 (only texture and constant caches); Fermi's 768 KB grows to 1.5, 3, 4 and 6 MB. The register file per SM is 32 KB on G80, 64 KB on GT200, 128 KB on Fermi and 256 KB from Kepler on; it stops growing because 65,536 entries per SM is the register budget the programming model assumes (slide 12). Shared memory per SM goes 16, 16, 48, 48, 96, 64, 96 KB (the maximum configuration; on Fermi and Kepler it trades against L1, on Volta the 128 KB array is shared between L1 and shared memory). Note that from Fermi onward the register file is larger than the L1 and shared memory combined: the GPU's fastest storage is its registers, and it has more of them than cache.

Power and link. Board power stays within a narrow band, 155 to 300 W, while arithmetic grows 45-fold: performance per watt is where the decade's progress shows. The 155 W for the 8800 GTX is the official maximum board power NVIDIA later listed; launch reviewers were told a higher figure[†], and the column should be read as approximate for that card. The host link goes from PCIe 1 to PCIe 3 and then to NVLink and NVLink 2 (slide 25), the last two carrying GPU-to-GPU traffic that PCIe never could.

What does not change is the shape. Each die is a set of identical SMs behind a shared L2 and a wide memory, each SM a register file, schedulers, lanes and an SRAM array, each lane an FP32 fused multiply-add pipeline. The programming model of slides 9–10 maps onto any column of this table with the counts changed, which is why a CUDA program from 2007 runs on a V100 and why the lecture spends its time on the structure rather than the numbers.

Peak FP32 (derived) = SMs × lanes per SM × 2 FLOP per fused multiply-add × clock: G80, $16 \times 8 \times 2 \times 1.35$ GHz $= 0.35$ TFLOP/s; K40 at base, $15 \times 192 \times 2 \times 0.745$ GHz $= 4.29$ TFLOP/s; V100, $80 \times 64 \times 2 \times 1.53$ GHz $= 15.7$ TFLOP/s. Bytes per FLOP = bandwidth / peak: G80, $86.4/346 = 0.25$; V100, $900/15{,}700 = 0.057$. Register file in KB = 32-bit entries $\times 4/1{,}024$: $65{,}536 \times 4/1{,}024 = 256$ KB.

Terms

process node The name of a semiconductor manufacturing generation (90, 65, 40, 28, 16, 12 nm); smaller nodes pack more transistors per area and, until about 2005, ran them at lower power per transistor.

reticle limit The largest area a lithography stepper can expose in one shot, about 850 mm^2; GV100 at 815 mm^2 is at it, so a monolithic GPU can grow no larger.

yield harvesting Selling a die with some units disabled (56 of 60 SMs on GP100, 80 of 84 on GV100, 14 of 16 on the C2050) so that dies with defects in a few SMs remain sellable.

GDDR Graphics double-data-rate DRAM, a family of high-speed memories soldered around the GPU on the board; GDDR3 on G80 and GT200, GDDR5 on Fermi through Maxwell.

HBM2 High Bandwidth Memory, second generation: DRAM dies stacked and connected to the GPU through a silicon interposer with a 1,024-bit interface per stack; JEDEC JESD235A (January 2016), used by Pascal and Volta.

board power The maximum power a GPU board is designed to draw, in watts; the TDP row of the table. Compute products ran from 155 to 300 W over the decade.

SXM2 NVIDIA's mezzanine module form factor for the P100 and V100, carrying NVLink and allowing 300 W; the PCIe card versions run at 250 W and lower clocks.

Intuition

Cover the first row and the last two rows of the table and every column describes the same machine: SMs of lanes with registers and shared memory behind a shared L2 and a wide DRAM. Uncover them and you see the only three things that changed: how many, how fast, and what the memory is made of.

7.4 Slide 19: Deep learning arrives (2012–2016) and the architecture answers

AlexNet trained on two GeForce cards; five years later the compute die had a unit for nothing but matrix multiply.

What the slide shows A bar chart on the left, on a log scale, for five GPUs a deep-learning trainer might have used between 2012 and 2017: the GeForce GTX 580, the Tesla K40, M40, P100 and V100. Two bars per GPU: peak FP32 TFLOP/s (1.58, 4.3, 6.8, 10.6, 15.7) and the fastest precision the trainer could use (the same through the M40, then P100's FP16 at 21.2 and V100's tensor cores at 125). The vendor peaks carry a dagger. The right panel tells the story in four steps: 2012 (AlexNet on two GTX 580s), 2014 (cuDNN), what the workload asked for, and what the hardware answered.

Details In 2012 Krizhevsky, Sutskever and Hinton trained AlexNet, a convolutional network of about 60 million parameters, on the ImageNet dataset and won the ILSVRC competition by a margin that ended the debate about whether learned features could beat engineered ones. The paper states that training took "between five and six days on two GTX 580 3GB GPUs"; the network was split across the two cards because neither had the memory to hold it, with the layers divided between them and communication only at certain layers. The GTX 580 was a consumer Fermi (GF110, 512 cores) with a peak of 1.58 TFLOP/s in single precision, and the authors wrote their own CUDA kernels for convolution. The architectural fact underneath is that a convolution can be rearranged as a matrix multiplication (unroll the input patches into the columns of a matrix, the filters into the rows of another), so the whole training workload reduces to large dense matrix products, the one common kernel that reaches the roofline (slide 15).

In 2014 NVIDIA released cuDNN (Chetlur et al., arXiv October 2014), a library of tuned convolution, pooling and activation routines with a stable interface. Before it, every framework carried its own GPU kernels and most were slow; after it, Caffe, Torch, Theano and, from 2015, TensorFlow called the same routines and inherited every improvement NVIDIA made to them. This is the single most consequential software decision of the period for the hardware story: it tied the deep-learning frameworks to CUDA, and it meant that any architectural feature NVIDIA added for training (FP16 arithmetic, tensor cores) reached every user through a library update rather than through rewritten kernels. Training moved from gaming cards to

the Tesla line (K40, K80, M40) in data centers, where ECC, more memory and server form factors mattered more than price.

The workload asked for four things, listed in the panel. Dense matrix throughput, because convolution and the fully connected layers are matrix multiply. Memory capacity, because the activations of every layer must be kept for the backward pass, and batch size times activation size exceeded a GeForce card's memory for most networks of interest (the M40's 12 and 24 GB were sold on this point). Fast links between GPUs, because data-parallel training replicates the model on each GPU and must average gradients across them every step, and the averaging traffic over PCIe became the bottleneck as the GPUs got faster (slide 25). And tolerance of low precision: stochastic gradient descent is robust to noise in the arithmetic, so 16-bit values suffice for most of the work if the accumulations and the weight updates are protected, which the FP64 and FP32 rates of the HPC parts were irrelevant to.

The hardware answered in two generations. Pascal's GP100 (slide 20) added packed half-precision arithmetic at twice the FP32 rate (21.2 TFLOP/s), HBM2 for bandwidth and capacity, and NVLink for the inter-GPU traffic. Volta's GV100 (slides 22–23) added tensor cores, matrix units that do a $4 \times 4 \times 4$ half-precision multiply-add per clock, for 125 TFLOP/s, eight times the FP32 rate; and Micikevicius et al. (2017) published the mixed-precision training recipe (FP32 master weights, FP16 forward and backward passes, loss scaling) that made the FP16 rate usable without loss of accuracy. The chart's right-hand bars show the result: the fastest usable precision had tracked FP32 for a decade and then pulled away from it by a factor of eight in two years. The rest of the deck's story about the compute die, from Pascal on, is the story of a chip designed around this workload.

Terms

AlexNet The convolutional network of Krizhevsky, Sutskever and Hinton (2012) that won ImageNet; trained in five to six days on two GeForce GTX 580s with 3 GB each.

ImageNet The 1.2-million-image classification dataset and competition (ILSVRC) on which AlexNet's 2012 result started the deep-learning era.

convolution as matrix multiply Rearranging the input patches of a convolution into a matrix (im2col) so that the layer becomes one dense matrix product, the kernel GPUs run best.

cuDNN NVIDIA's CUDA Deep Neural Network library (2014): tuned convolution, pooling and activation primitives behind a stable interface, adopted by every major framework.

data-parallel training Replicating a model on several GPUs, giving each a slice of the batch, and averaging the gradients across GPUs each step; its communication is the all-reduce of slide 25.

activations The intermediate outputs of each layer in a network, stored during the forward pass for use in the backward pass; the main consumer of memory capacity in training.

FP16 The IEEE binary16 format: 1 sign bit, 5 exponent bits, 10 mantissa bits; Pascal computes on it at twice the FP32 rate, Volta's tensor cores take it as input.

mixed-precision training Training with FP16 arithmetic for the forward and backward passes, FP32 master copies of the weights, and a scaled loss so that small gradients remain representable (Micikevicius et al. 2017).

Tesla K80 The dual-GK210 Kepler board of 2014 (4,992 lanes, 24 GB, 8.74 TFLOP/s FP32 with boost, 300 W), the common training GPU in cloud data centers before Pascal.

Intuition

For a decade the compute die was designed for people who needed 64-bit arithmetic and error correction. In 2012 a different customer appeared who needed neither, only matrix multiply and memory, and within five years the most expensive part of the die was a unit built for that customer alone.

7.5 Slide 20: Pascal GP100 (2016): 16 nm FinFET, HBM2, NVLink

The first compute die built for deep learning as well as HPC: 15.3B transistors, 720 GB/s, FP16 at double rate, NVLink.

What the slide shows Top left, a diagram of the GP100 package: the GPU die in the center, drawn as six columns of ten SM blocks (six graphics processing clusters of ten SMs, 56 of 60 enabled) under a bar labeled "4 MB L2; 8 memory controllers, 4,096 bits", flanked by four HBM2 stacks, two on each side, all on one silicon interposer (CoWoS). A caption gives the numbers: 610 mm^2 of die on about 1,000 mm^2 of interposer; the stacks put thousands of short, slow wires where GDDR5 had hundreds of fast ones, for 2.5 times the Tesla M40's bandwidth (NVIDIA quotes 3× against GM200†) at less energy per bit (slide 13). Two stat tiles: 15.3 billion transistors on 16 nm FinFET, and 720 GB/s from 16 GB of HBM2 in four stacks. The right panel, "What Pascal added†", lists the SM, FP16 at twice the FP32 rate, NVLink 1.0, unified memory with page faulting, compute preemption, and the products.

Details The Tesla P100 was announced at GTC on April 5, 2016, shipped first inside NVIDIA's own DGX-1 server in June, and reached OEM servers early in 2017; a PCIe card version was announced at ISC in Frankfurt on June 20, 2016†. GP100 is 15.3 billion transistors on TSMC's 16 nm FinFET process, 610 mm^2 (NVIDIA's own figure). The die holds six graphics processing clusters of ten SMs each, 60 SMs and 30 texture processing clusters in all, with 56 SMs enabled on the P100. Each SM has 64 FP32 lanes in two processing blocks of 32, each block with its own warp scheduler and two dispatch units; 32 FP64 units, so double precision runs at half the single rate; a 256 KB register file; and 64 KB of shared memory, now separate from the L1 and texture cache. The SM is half the size of Maxwell's: the same 2,048 threads and 65,536 registers are shared among 64 lanes instead of 128, so each thread can hold twice the registers at full occupancy and each lane is easier to keep busy. The L2 is 4 MB. At the SXM2 module's boost clock of 1,480 MHz the peak rates are 5.3 TFLOP/s FP64, 10.6 FP32 and 21.2 FP16; the PCIe card at 250 W runs slower, at 4.7, 9.3 and 18.7†.

The FP16 rate is Pascal's first concession to deep learning. GP100's FP32 lanes execute a paired half-precision operation, two FP16 fused multiply-adds packed in one 32-bit register (CUDA's `half2` type), so the FP16 throughput is twice the FP32 throughput with no additional hardware. The consumer Pascal die, GP104 in the GeForce GTX 1080 (7.2 billion transistors, 314 mm^2, 2,560 lanes, 8 GB of GDDR5X at 320 GB/s, 180 W)†, omits both the fast FP16 and the half-rate FP64, which is the compute-die versus consumer-die split of slide 16 in its clearest form.

The memory system is the second concession and the slide's diagram. HBM2 stacks four or eight DRAM dies vertically, connected by through-silicon vias, with a 1,024-bit interface per stack organized as eight independent 128-bit channels; the JEDEC standard (JESD235A, January 2016) allows up to 256 GB/s and 8 GB per stack. GP100 uses four stacks of four dies, 4 GB each, at 180 GB/s per stack, so 16 GB and 720 GB/s over a 4,096-bit bus driven by eight 512-bit memory controllers. The stacks cannot sit on a circuit board next to the GPU, because 4,096 signal wires will not route through one; they sit with the GPU on a silicon interposer, a passive piece of silicon with fine wiring, in TSMC's CoWoS (chip-on-wafer-on-substrate) packaging, and the interposer at about 1,000 mm^2 is larger than the GPU die itself. The gain, as slide 13 argues, is energy: thousands of short wires at a low signaling rate move each bit for a fraction of the energy of GDDR5's hundreds of long wires at a high rate, and the bandwidth triples (from 288 to 720 GB/s) within the same board power. HBM2 also carries ECC natively, without the capacity penalty Fermi's GDDR5 paid.

NVLink is the third. GP100 has four NVLink ports, each a pair of eight-lane sub-links at 20 Gb/s per lane, so 20 GB/s in each direction and 40 GB/s bidirectional per link, 160 GB/s per GPU (derived; slide 25 gives the protocol and the DGX-1 topology). Against PCIe 3.0 x16's 16 GB/s per direction, that is five times the GPU-to-GPU bandwidth, and on IBM's POWER8 servers the CPU had NVLink ports too, so the CPU-to-GPU path was also five times faster; x86 CPUs never got NVLink.

Two architectural features complete the list. Unified memory with page faulting gives the CPU and GPU a single 49-bit virtual address space (512 TB) in which a page can live on either side and migrates on demand when touched by the other: a pointer obtained on the host works in a kernel, and the data follows the first access rather than being copied in advance. Compute preemption at instruction granularity lets the GPU interrupt a running kernel, save its state and run something else (a display update, a debugger, another process's work), which earlier generations could do only at kernel boundaries. The products were the Tesla P100 in SXM2 (300 W) and PCIe (250 W) form, the DGX-1 with eight SXM2 P100s in the hybrid cube mesh of slide 25, and the upgrade of Piz Daint at CSCS to 5,320 P100s (slide 26).

HBM2 bandwidth (derived): 4 stacks $\times$ 180 GB/s $=$ 720 GB/s; equivalently 4,096 bits $\times$ 1.4 Gb/s per pin $/8 = 717$ GB/s. NVLink 1.0 per GPU: 4 links $\times$ 2 directions $\times$ 20 GB/s $=$ 160 GB/s. Peak rates at 1,480 MHz: FP32 $56 \times 64 \times 2 \times 1.48 = 10.6$ TFLOP/s; FP64 $56 \times 32 \times 2 \times 1.48 = 5.3$; FP16 at twice FP32 $= 21.2$.

Terms

Pascal NVIDIA's 2016 architecture; the compute die GP100 has 56 enabled SMs of 64 lanes, 15.3 billion transistors on 16 nm FinFET, HBM2, NVLink and double-rate FP16.

GPC Graphics processing cluster: a group of SMs with shared raster and geometry units; GP100 has six of ten SMs each.

FinFET A transistor whose channel is a raised fin wrapped by the gate on three sides, giving better control at small dimensions; TSMC's 16 nm process was its first FinFET node for GPUs.

half2 CUDA's packed type holding two FP16 values in one 32-bit register, on which Pascal performs two half-precision operations per lane per instruction.

GDDR5X A faster variant of GDDR5 (JEDEC JESD232, January 2016) used on the consumer Pascal GTX 1080 at 10 Gb/s per pin.

silicon interposer A passive silicon die with fine wiring on which a GPU and its HBM2 stacks are mounted side by side, carrying the thousands of signals between them.

CoWoS TSMC's chip-on-wafer-on-substrate packaging, which places dies on a silicon interposer and the interposer on a package substrate; used for GP100 and GV100.

through-silicon via A vertical electrical connection through a silicon die, used to stack the DRAM dies of HBM2.

NVLink NVIDIA's point-to-point GPU interconnect: version 1.0 on Pascal with four 40 GB/s links per GPU (160 GB/s), version 2.0 on Volta with six 50 GB/s links (300 GB/s).

unified memory A single virtual address space spanning CPU and GPU memory; on Pascal, 49 bits with hardware page faulting so that pages migrate on demand.

compute preemption The ability to interrupt a running kernel at an instruction boundary, save its state and resume it later; introduced on Pascal.

DGX-1 NVIDIA's eight-GPU server (2016 with P100, 2017 with V100), the GPUs connected in an NVLink hybrid cube mesh (slide 25).

Intuition

Pascal moved the memory from the edge of the board to the edge of the die, and the neighbor GPU from the far side of a PCIe switch to the far end of a private wire. Both are the same move: shorten the distance that the bytes travel, because by 2016 the bytes, not the arithmetic, were what the power budget paid for.

7.6 Slide 21: Reading a Pascal or Volta SM: four schedulers, many lanes

The SM is partitioned into quarters, each with its own scheduler and registers; the whole shares L1 and shared memory.

What the slide shows A block diagram of one Volta SM, after Choquette, Giroux and Foley (IEEE Micro, 2018). At the top, the L1 instruction cache. Below it, four identical partitions side by side, each with a warp scheduler and dispatch unit, a 64 KB register file, 16 FP32 and 16 INT32 lanes, 8 FP64 units, two tensor cores, 8 load/store units and 4 special-function units. Across the bottom, the 128 KB array that serves as L1 data cache and shared memory (up to 96 KB of it as shared memory) and the four texture units. A caption totals the SM: 64 FP32, 64 INT32 and 32 FP64 lanes, 8 tensor cores, 256 KB of registers, 2,048 threads, 80 such SMs on a V100. The right panel explains how to read it: the four quarter-SMs and their issue rate, the separate INT32 lanes, registers above cache, and the derivation of the peak rates.

Details The partitioned SM is Maxwell's idea carried through Pascal and Volta. A quarter-SM, which NVIDIA calls a processing block, is a self-contained issue unit: one warp scheduler, one dispatch unit, a 16,384-entry register file (64 KB), and its own arithmetic lanes. The scheduler picks one of the warps assigned to its partition each clock and issues one instruction from it. Because the partition has 16 FP32 lanes and a warp has 32 threads, a warp-wide FP32 instruction occupies the lanes for two clocks; with four partitions issuing in parallel the SM executes 64 FP32 operations per clock, which is the figure the peak numbers assume. Pascal's GP100 has two such blocks of 32 lanes per SM (so a warp instruction takes one clock on its 32 lanes) and Volta's GV100 four blocks of 16; both arrive at 64 lanes per SM, and both give each partition its own registers and scheduler so that no arbitration between partitions is needed on the issue path.

Volta's partitions contain more kinds of unit than Pascal's. Beside the 16 FP32 lanes sit 16 INT32 lanes that execute concurrently: address arithmetic, loop counters and index computations, which in every earlier generation took issue slots away from floating-point work, now run alongside it, and the whitepaper's claim that this raises sustained throughput on real kernels is a vendor one[†]. Eight FP64 units per partition (32 per SM) give double precision at half the FP32 rate, as on Pascal. Two tensor cores per partition, eight per SM, are the subject of slide 23. Eight load/store units and four special-function units (transcendentals, reciprocals, square roots) complete the block. The instruction cache at the top of the SM feeds a small L0 instruction cache in each partition, so the four schedulers do not contend for instruction fetch.

Below the partitions, Volta merges what Pascal kept apart. Pascal had a 64 KB shared memory and a separate 24 KB L1 and texture cache per SM; Volta has one 128 KB SRAM array that the driver configures as shared memory (up to 96 KB) and L1 data cache (the remainder, at least 32 KB), with the L1 and texture caches unified. The whitepaper's argument is that a kernel which does not use shared memory gets a 128 KB L1 instead of a 24 KB one, and that a kernel using shared memory as a programmer-managed cache can often be replaced by one relying on the L1 at a smaller loss than before; the measured latencies (slide 24) show the shared memory at 19 cycles and the L1 at 28, close enough that the simpler code is often good enough.

The panel's third point is the one to carry away. Each SM has 256 KB of registers and 128 KB of L1 plus shared memory: the register file is twice the size of the cache, and it is the register file, not the cache, that

holds a resident warp's working set. This inverts the CPU picture, where registers are a few hundred bytes and caches are megabytes, and it is the reason occupancy is a register budget (slide 12): 65,536 registers divided by 2,048 threads is 32 registers per thread at full occupancy, and a kernel that needs 64 registers per thread can keep only half the warps resident.

The peak rates follow from the diagram. Eighty SMs, each with 64 FP32 lanes doing one fused multiply-add (two FLOP) per clock, at the SXM2 boost clock of 1.53 GHz: $80 \times 64 \times 2 \times 1.53 \times 10^9 = 15.7$ TFLOP/s. The 32 FP64 units per SM give $80 \times 32 \times 2 \times 1.53 = 7.8$ TFLOP/s. The tensor cores' 125 TFLOP/s are derived on slide 23. The full GV100 die has 84 SMs and 5,376 lanes; the V100 enables 80 for yield (slide 18).

Peak FP32 (V100 SXM2, derived): 80 SMs $\times$ 64 lanes $\times$ 2 FLOP $\times$ 1.53 GHz $=$ 15.7 TFLOP/s. Peak FP64: $80 \times 32 \times 2 \times 1.53 = 7.8$ TFLOP/s. Issue: one warp instruction per partition per clock over 16 lanes, so two clocks per 32-thread warp instruction; four partitions give 64 lane-operations per clock per SM. Registers per thread at full occupancy: $65{,}536/2{,}048 = 32$.

Terms

processing block NVIDIA's name for a partition of an SM with its own warp scheduler, dispatch unit, register file and lanes; two per SM on Pascal, four on Volta. Also called a quarter-SM here.

dispatch unit The logic that sends an issued warp instruction to the appropriate lanes; one per scheduler on Volta, two per scheduler on Kepler and Pascal.

INT32 lane A 32-bit integer arithmetic pipeline; Volta gives each partition 16 of them, separate from the FP32 lanes, so integer and floating-point instructions issue concurrently.

special-function unit A pipeline for transcendental functions (sine, cosine, reciprocal, reciprocal square root, logarithm, exponential) at lower throughput than the FP32 lanes; SFU.

load/store unit A pipeline that computes addresses and issues memory requests for a warp's loads and stores; 8 per partition on Volta.

L0 instruction cache A small per-partition instruction cache on Volta fed from the SM's L1 instruction cache, so that the four schedulers fetch independently.

unified L1 and shared memory Volta's 128 KB per-SM SRAM array configured at run time between L1 data cache and shared memory (up to 96 KB shared); Pascal kept them separate.

texture unit A graphics-era unit that fetches and filters texture samples; four per SM on Volta, also usable for cached read-only loads in compute kernels.

Intuition

Read the SM diagram from the inside out: a scheduler owns 16 lanes and 64 KB of registers and never waits on its neighbors; four of them share one SRAM array and one instruction cache; eighty of them share one L2 and one memory. Each level shares only what it must, which is how the design keeps 163,840 threads from getting in each other's way.

8 Volta and the tensor core (slides 22–24)

8.1 Slide 22: Volta GV100 (2017): the largest die, and a new kind of unit

21.1 billion transistors on 815 mm^2, near the reticle limit; 80 SMs, 640 tensor cores, 900 GB/s of HBM2, NVLink 2.

What the slide shows Four stat tiles across the top: 21.1 billion transistors on TSMC's 12 nm FFN process; 815 mm^2 of die, reticle-limited; 125 TFLOP/s of tensor arithmetic (FP16 inputs, FP32 accumulation); 300 GB/s of NVLink 2.0 over six links. Two panels below. Left, "The chip†": 84 SMs on the die and 80 enabled, 5,120 FP32 lanes, 2,560 FP64 units, 640 tensor cores, 6 MB of L2, 16 GB (32 GB from 2018) of HBM2 on four stacks at 900 GB/s, 20 MB of register files in total, the peak rates at the 1.53 GHz boost clock (15.7 FP32, 7.8 FP64, 125 tensor), and the products: Tesla V100 in SXM2 (300 W) and PCIe (250 W; 7, 14 and 112 TFLOP/s) form, the DGX-1V and the DGX Station. Right, "What is new, in order of importance": tensor cores, independent thread scheduling, the reworked SM, NVLink 2.0 with coherence, and CUDA 9's cooperative groups and WMMA API.

Details Volta was announced at GTC on May 10, 2017, with the architecture whitepaper (WP-08608) following in August; the 32 GB version, NVSwitch and the DGX-2 came on March 27, 2018†. The launch press release gave round numbers ("21 billion transistors", "120 TFLOPS") that the whitepaper later refined to 21.1 billion and 125†. GV100 is built on TSMC's 12 nm FFN process, a tuned variant of the 16 nm FinFET node used for Pascal rather than a full shrink ("FFN" marks a variant developed with NVIDIA), and its 815 mm^2 die is the largest NVIDIA had made and at the reticle limit of the lithography tools: a monolithic GPU cannot be made larger, which is one reason the generations after Volta looked elsewhere for growth.

The die has six graphics processing clusters, 84 SMs and 42 texture processing clusters; the V100 enables 80 SMs, giving 5,120 FP32 lanes, 5,120 INT32 lanes, 2,560 FP64 units and 640 tensor cores (slide 21 gives the SM; slide 23 the tensor core). The register files total 80×256 KB $= 20$ MB, more than three times the 6 MB L2, which is the slide's way of restating slide 21's point that the GPU's largest fast storage is its registers. Memory is four HBM2 stacks at 900 GB/s, 16 GB with four-high stacks at launch and 32 GB with eight-high stacks from 2018, with ECC native to HBM2; the memory clock rose a quarter from P100's 720 GB/s on the same 4,096-bit bus. The peak rates at the SXM2 boost clock of 1,530 MHz are 15.7 TFLOP/s FP32, 7.8 FP64 and 125 tensor TFLOP/s; the PCIe card at 250 W and a lower clock is rated at 7, 14 and 112†. The whitepaper does not list a plain FP16 rate for V100 (it would be 31.4 TFLOP/s, twice FP32, as on Pascal), because the tensor cores made the packed-FP16 path a secondary way to use half precision.

The right panel ranks the changes, and the ranking is the argument of this section. First, tensor cores: a matrix unit in each processing block that performs a $4 \times 4 \times 4$ half-precision matrix multiply-add per clock, giving eight times the FP32 rate for the dense matrix products that dominate deep-learning training and inference (slide 23). This is the feature that made Volta a different kind of processor from its predecessors: for the first time the compute die contained an arithmetic unit that only one class of kernel could use. Second, independent thread scheduling (slide 24): each thread has its own program counter and call stack, so lanes that diverge within a warp can interleave and synchronize instead of running to completion one path at a time; a change to the execution model of slide 11 that had been fixed since G80. Third, the SM (slide 21): separate INT32 lanes beside the FP32 lanes, a unified 128 KB L1 and shared-memory array, four partitions each with a scheduler. Fourth, NVLink 2.0: six links at 25 GB/s per direction, 300 GB/s per GPU, with cache coherence and address translation services when the partner is an IBM POWER9, so that CPU and GPU memory form one coherent space in Summit's node (slide 26); and the NVSwitch of 2018, which connects sixteen V100s at full NVLink bandwidth (slide 25). Fifth, software: CUDA 9 shipped with Volta and added cooperative groups (named sets of threads that can synchronize, from a few lanes to the whole grid) and the WMMA (warp matrix multiply-accumulate) API through which a warp drives the tensor cores; cuBLAS and cuDNN 7 used the tensor cores automatically for half-precision matrix multiply and convolution.

The products close the panel. The Tesla V100 shipped as an SXM2 module (300 W, NVLink) and a PCIe card (250 W), in the DGX-1V (eight SXM2 V100s, the P100 DGX-1's successor with NVLink 2.0 links) and

the DGX Station (four V100s in a water-cooled deskside box), and, through IBM, in the Summit and Sierra nodes of slide 26. The 32 GB V100 of 2018 was the part in the DGX-2.

Terms

Volta NVIDIA's 2017 architecture; the compute die GV100 has 80 enabled SMs of 64 lanes with 8 tensor cores each, 21.1 billion transistors on 12 nm FFN, 815 mm^2, HBM2 at 900 GB/s and NVLink 2.0.

12 nm FFN TSMC's tuned variant of its 16 nm FinFET process used for GV100; a refinement rather than a new node, with the N denoting the NVIDIA-specific variant.

tensor core A matrix arithmetic unit in each Volta processing block that computes a $4 \times 4 \times 4$ fused multiply-add on FP16 inputs with FP32 accumulation each clock; 64 FMAs per clock per unit, 8 units per SM. Slide 23.

independent thread scheduling Volta's execution model in which each thread has its own program counter and call stack, so that diverged threads in a warp can interleave and synchronize; before Volta a warp had one program counter. Slide 24.

NVLink 2.0 Volta's interconnect: six links per GPU at 25 GB/s per direction (50 GB/s bidirectional), 300 GB/s per GPU, with cache coherence and address translation services toward POWER9 CPUs.

NVSwitch NVIDIA's 2018 NVLink crossbar switch with 18 ports at 50 GB/s each; twelve of them connect the sixteen V100s of a DGX-2 at full bandwidth. Slide 25.

cooperative groups A CUDA 9 programming interface that names groups of threads at any granularity (a few lanes, a warp, a block, the whole grid) and provides synchronization over them.

WMMA Warp matrix multiply-accumulate, the CUDA 9 API through which a warp loads matrix fragments into registers, calls the tensor cores and stores the result; the programmer's interface to tensor cores.

DGX Station NVIDIA's deskside system with four V100s connected by NVLink, water-cooled; the small sibling of the DGX-1.

HBM2 stack height The number of DRAM dies stacked per HBM2 device: four-high stacks gave the V100 16 GB, eight-high stacks 32 GB from 2018.

Intuition

Every generation before Volta made the same machine bigger and more efficient. Volta made it bigger, to the physical limit of a die, and then added a unit that the ordinary programming model cannot see: the first sign that the general-purpose throughput processor of 2006 was becoming a host for specialized engines.

8.2 Slide 23: Tensor cores: a 4×4×4 matrix multiply-add per clock

$D = AB + C$ on 4×4 tiles, FP16 in and FP32 out: 64 fused multiply-adds per clock per tensor core, 512 per SM.

What the slide shows At the top, the operation one tensor core performs each clock, drawn as four 4×4 grids: $D = A \times B + C$, with A and B in FP16 and C and D in FP32 (or FP16); the diagonal cells are shaded to suggest the sixteen independent dot products. A caption: each of the sixteen outputs is a four-term dot product, multiplied exactly and summed in FP32, so 64 fused multiply-adds per clock against the 16 the partition's FP32 lanes manage. Below, a panel "Why it is eight times faster" with the derivation of 125

TFLOP/s. On the right, "How it is used, and what it costs": the WMMA programming interface and the libraries, the precision and the mixed-precision training recipe, and the question of feeding a unit with a 139 FLOP per byte ridge point.

Details A tensor core takes three 4×4 matrices and produces a fourth: $D_{ij} = \sum_{k=1}^{4} A_{ik}B_{kj} + C_{ij}$ for each of the sixteen (i, j). The inputs A and B are FP16; each product $A_{ik}B_{kj}$ of two 11-bit significands is computed exactly (a 22-bit result fits comfortably in FP32), and the four products plus the addend C_{ij} are summed in FP32, with D returned as FP32 or rounded to FP16. Sixteen outputs, each a four-term dot product, is 64 multiply-adds, performed in one clock by a fixed array of multipliers feeding adder trees. An FP32 lane does one multiply-add per clock, and a processing block has 16 of them; the block's two tensor cores do 128. That ratio, eight, is the one on the slide.

The derivation of the peak follows. Each SM has four processing blocks with two tensor cores each, eight in all, so $8 \times 64 = 512$ fused multiply-adds, or 1,024 FLOP, per clock per SM. Eighty SMs at 1.53 GHz give $80 \times 1{,}024 \times 1.53 \times 10^9 = 125 \times 10^{12}$ FLOP/s, the 125 TFLOP/s of the datasheet (derived); the FP32 lanes on the same clock give $80 \times 64 \times 2 \times 1.53 = 15.7$. The subtitle's "512 per SM" counts multiply-adds; the whitepaper's "1,024 FLOP per clock per SM" counts each as two operations, and the two statements agree. The launch press release's "120 TFLOPS" was the same quantity at a slightly lower assumed clock[†].

Why this is faster than the same arithmetic on the FP32 lanes is the energy argument of slide 13 applied to one instruction. On the lanes, every multiply-add is a separate instruction: fetched, decoded, its three operands read from the register file, its result written back, with the warp scheduler's bookkeeping on each. In the tensor core, one instruction moves the 4×4 tiles from registers into the unit once, the products and sums flow through a fixed datapath with no intermediate register traffic, and the sixteen results are written once. The per-element overhead that dominates a scalar lane's energy (slide 13 put instruction overhead and register access at tens of picojoules against a few for the arithmetic) is paid once per 64 operations instead of once per operation. The price is generality: the unit computes only this one shape of operation, and only in these precisions.

Programming the tensor cores goes through the warp. The WMMA API in CUDA 9 presents a warp-level operation on $16 \times 16 \times 16$ tiles (other shapes were added later): the 32 threads of a warp cooperatively load fragments of A, B and C into their registers with `load_matrix_sync`, call `mma_sync`, which the compiler maps onto a sequence of hardware tensor-core instructions on $4 \times 4 \times 4$ pieces, and write the result with `store_matrix_sync`. The fragment layout in registers is opaque to the programmer, which is what lets NVIDIA change the hardware shape underneath. Most users never call WMMA: cuBLAS selects tensor-core kernels for half-precision GEMM when the matrix dimensions allow, and cuDNN 7 does the same for convolutions, so a framework built on them gained the speedup by changing a data type. Markidis et al. (2018) measured the API on a V100: the throughput of WMMA kernels against cuBLAS, and the numerical error of the FP16-input, FP32-accumulate scheme against FP32 and FP64 matrix multiply, finding the accumulation accurate and the FP16 rounding of inputs the dominant error.

The precision question is the one the right panel spends most of its space on. FP16 has a 10-bit stored mantissa (11 bits of significand with the hidden bit), five exponent bits and a sign bit: about three decimal digits, and a range from roughly 6×10^{-5} (the smallest normal number; subnormals reach 6×10^{-8}) to 65,504. Neural-network weights and activations fit that range; gradients often do not, with many values below 10^{-5} that flush to zero in FP16. Micikevicius et al. (2017) gave the recipe that made the tensor cores usable for training: keep a master copy of the weights in FP32 and apply the updates to it, so that small updates are not lost to rounding; run the forward and backward passes in FP16 (the matrix products on tensor cores, accumulating in FP32); and multiply the loss by a scale factor before the backward pass, which shifts the gradients up into FP16's range, then divide the weight gradients by the same factor before the update. With these three steps they reported no loss of accuracy against FP32 training on image classification, speech recognition and machine translation models, at a fraction of the time and memory[†]. Mixed precision in this

form became the default training mode of every framework within two years.

Feeding the unit is the last point, and it comes straight from the roofline of slide 15. At 125 TFLOP/s against 900 GB/s the ridge point is $125{,}000/900 = 139$ FLOP per byte of DRAM traffic (derived). A matrix multiply with tiles large enough to reuse each loaded element 139 times from registers and shared memory reaches the roof; almost nothing else in a neural network does. The element-wise activations, the normalization layers, the softmax and the data movement between layers have intensities of order one FLOP per byte and run at the bandwidth limit, during which the tensor cores idle. The consequence, visible in every profile of a Volta training run, is that the matrix multiplies take a shrinking fraction of the time and the "memory-bound rest" a growing one; the architectural responses (larger tiles, fused kernels, more formats) belong to the generations after this lecture's scope.

One tensor core per clock: $D_{ij} = \sum_{k=1}^{4} A_{ik}B_{kj} + C_{ij}$, $i, j \in \{1, \ldots, 4\}$: 16 outputs $\times$ 4 multiply-adds $= 64$ FMA $= 128$ FLOP. Per SM: $8 \times 64 = 512$ FMA $= 1{,}024$ FLOP per clock. Peak (V100 SXM2, derived): $80 \times 1{,}024 \times 1.53\,\text{GHz} = 125$ TFLOP/s; ratio to FP32, $1{,}024/128 = 8$. Ridge point: $125{,}000/900 = 139$ FLOP per byte. FP16: 1 sign, 5 exponent, 10 mantissa bits; largest finite value 65,504; smallest normal $2^{-14} \approx 6.1 \times 10^{-5}$; smallest subnormal $2^{-24} \approx 6.0 \times 10^{-8}$. Loss scaling: compute gradients of $s \cdot L$ in FP16, then update with $(1/s)$ times the result in FP32.

Terms

matrix multiply-add The operation $D = AB + C$ on matrices; the tensor core's native instruction on 4×4 tiles, and the building block of every dense linear-algebra and neural-network kernel.

FP32 accumulation Summing products in 32-bit floating point even when the inputs are 16-bit, so that rounding error does not grow with the length of the dot product.

fragment In the WMMA API, the portion of a matrix tile held in one thread's registers; a warp's 32 fragments together hold a 16×16 tile in a layout the hardware defines.

mma sync The WMMA call `mma_sync` that performs a warp-wide matrix multiply-accumulate on loaded fragments using the tensor cores.

GEMM General matrix multiply, $C \leftarrow \alpha AB + \beta C$, the BLAS routine that cuBLAS maps onto tensor cores for half-precision inputs.

significand The fraction part of a floating-point number including the implicit leading bit; 11 bits in FP16, 24 in FP32, 53 in FP64.

subnormal number A floating-point value below the smallest normal number, represented with reduced precision; FP16 subnormals extend the range down to about 6×10^{-8}.

master weights The FP32 copy of a model's parameters kept during mixed-precision training so that small updates are not lost to FP16 rounding.

loss scaling Multiplying the training loss by a constant before the backward pass so that gradients land within FP16's range, and dividing the weight gradients by the same constant before the update.

ridge point The arithmetic intensity at which a roofline's bandwidth slope meets its compute ceiling, peak FLOP/s divided by bandwidth; 139 FLOP per byte for V100's tensor cores. Slide 15.

Intuition

A tensor core is what you get when you ask what the instruction overhead of slide 13 would buy if it were paid once per sixty-four operations instead of once per operation. The answer is an eightfold rate, on the condition that the sixty-four operations always have the same shape.

8.3 Slide 24: Independent threads, and what microbenchmarks measure

Volta gives each thread a program counter; measured latencies show where the cycles go in a modern SM.

What the slide shows Top left, two timelines of a warp whose lanes take two paths, A and B, that must synchronize. Under Pascal (one program counter per warp) path A runs to completion and then path B; a note warns that if A waits for a flag that B sets, the warp never reaches B and deadlocks. Under Volta (a program counter and stack per thread) the scheduler interleaves A and B, so both make progress and the code is starvation-free; `__syncwarp()` reconverges the lanes when the programmer asks. Below, "Why it mattered": code that assumed lock-step execution within a warp could break, and code that needed a lock between threads of one warp could finally be written; cooperative groups name the sets of threads that synchronize. The right panel lists latencies measured on a V100 by Jia et al. (2018) in clock cycles (shared memory 19, L1 data hit 28, L2 hit about 193, global memory about 375 with a TLB hit and 1,029 with a TLB miss, an FP32 fused multiply-add 4), draws three lessons from them, notes the register file, and names the method, tracing it back to Wong et al. (2010) on GT200.

Details From G80 through Pascal, a warp had one program counter and one call stack. When its lanes disagreed at a branch, the hardware pushed the reconvergence point and the not-taken path onto a per-warp stack, executed the taken path with the other lanes masked off until the reconvergence point, then popped the stack and executed the other path (slide 11). The lanes on one path therefore could not execute an instruction until the lanes on the other path had finished theirs; the paths were serialized in time, not interleaved. For ordinary data-dependent branches this costs only time. For synchronization between the threads of one warp it is fatal: if the lanes on path A spin waiting for a flag that lanes on path B will set, the warp executes A's spin loop forever and never reaches B. Programmers learned not to write such code, and also learned the inverse habit of relying on the lock step: "warp-synchronous" code that read a value another lane had just written without a barrier, which was correct only because the lanes executed every instruction together.

Volta's independent thread scheduling changes the model. Each thread has its own program counter and call stack, kept by the hardware, and the scheduler may advance any subset of a warp's threads at each issue; a diverged warp still executes one path's instruction per issue slot over the lanes on that path, so the SIMD datapath and the cost of divergence are unchanged, but the scheduler can alternate between the paths rather than finishing one first. The flag-and-spin case now works, because the waiting lanes yield issue slots to the lanes that will set the flag, and the whitepaper's term for the guarantee is starvation-free: a thread that is waiting for another thread in its warp will eventually see it run. Reconvergence, which the hardware used to perform automatically at the post-dominator of the branch, is no longer guaranteed; the compiler reconverges where it can prove it safe, and the programmer calls `__syncwarp()` to force it. That is why pre-Volta warp-synchronous code could break on Volta (a lane may read before its neighbor has written, since lock step is no longer promised) and why CUDA 9 introduced the `_sync` variants of the warp shuffle and vote instructions, which take an explicit mask of participating lanes. Cooperative groups, also in CUDA 9, give the programmer named groups at every granularity, from a coalesced set of lanes to a thread block to the whole grid (with a launch API that guarantees all blocks are resident), each with a `sync()` method, so that the synchronization structure of a kernel is written down rather than assumed.

The microbenchmark numbers in the right panel are the architecture seen from the outside. Jia, Maggioni, Staiger and Scarpazza (2018) measured the V100 with the technique that Wong et al. (2010) had used on GT200: pointer chasing through arrays of chosen size and stride to find the latency of each level of the

memory hierarchy, and chains of dependent arithmetic instructions timed with the on-chip clock register to find pipeline latencies. Their figures for a V100: shared memory 19 cycles, L1 data cache hit 28, L2 hit about 193, global memory about 375 cycles when the address translation hits in the TLB and 1,029 when it misses, an FP32 fused multiply-add 4 cycles from issue to result. For comparison they measured the P100 at 82 cycles for an L1 hit and about 234 for the L2, and Wong et al. had found about 440 cycles for a global memory read and 38 for shared memory on the GT200 of 2008, with a 24-cycle FP32 pipeline.

Three things follow, and the panel draws them out. A warp that misses to DRAM is away for 400 to 1,000 cycles; at 1.53 GHz that is 0.25 to 0.7 microseconds, during which its SM can issue hundreds of instructions from other warps, which is the latency-hiding argument of slide 12 with numbers attached (and the reason that slide's Little's-law estimate used a range). An L2 hit at 193 cycles is about 50 FMAs' worth of latency: data that is "in cache" is far from free, and a kernel that depends on L2 hits still needs many warps or much instruction-level parallelism in flight. And the L1 at 28 cycles is five or six times slower than a CPU's first-level cache (4 to 5 cycles on contemporary cores), because the GPU's L1 serves up to 2,048 threads from four partitions through a crossbar and is sized and pipelined for throughput rather than for one thread's latency; the shared memory at 19 cycles is the fastest storage after the registers. The register file itself is read in one cycle but is banked, so an instruction whose operands fall in the same bank stalls a cycle; the compiler's register allocation, which decides both the bank conflicts and the occupancy (slide 12), is the first performance tool a CUDA programmer has, before any change to the source.

The method deserves a word, because the deck's practical slide (29) recommends it. Vendor documentation gives counts and peak rates; it does not give latencies, cache line sizes, replacement policies, bank mappings or the structure of the TLB, and these change between generations in ways that matter for performance. A few hundred lines of pointer chasing and timing, run once per new GPU, recover them. Wong et al. established the practice for GT200, Mei and Chu (2017) extended it across Fermi, Kepler and Maxwell, and Jia et al. did Volta (and Turing a year later); the lecture's figures for the memory hierarchy come from this literature rather than from datasheets.

Terms

program counter The register holding the address of the next instruction to execute; one per warp before Volta, one per thread from Volta on.

reconvergence The point after a divergent branch at which the lanes of a warp resume executing the same instruction; automatic at the branch's post-dominator before Volta, requested by the compiler or by `__syncwarp()` on Volta.

starvation-free Volta's guarantee that a thread waiting on another thread of the same warp will eventually see that thread scheduled, so that intra-warp locks and flags cannot deadlock.

warp-synchronous programming Pre-Volta code that relied on the lanes of a warp executing in lock step to omit synchronization; unsafe under independent thread scheduling.

syncwarp The CUDA 9 intrinsic `__syncwarp()` that forces the named lanes of a warp to reconverge and acts as a memory barrier among them.

warp shuffle Instructions that move a value from one lane's register to another's within a warp without going through memory; the `_sync` variants of CUDA 9 take an explicit lane mask.

microbenchmark A small program designed to measure one hardware property (a latency, a bandwidth, a cache size) by timing a controlled access pattern; the source of the latency figures on the slide.

pointer chasing A microbenchmark pattern in which each load's address comes from the previous load's result, so that loads cannot overlap and the time per load is the latency of the memory level that

holds the array.

TLB Translation lookaside buffer: a cache of virtual-to-physical address translations; a miss adds a page-table walk to a memory access, which is why the V100's global latency is 375 cycles with a hit and 1,029 with a miss.

clock register An on-chip counter readable from a kernel (`clock()`, `clock64()`) used to time short instruction sequences in cycles.

register bank conflict A stall of one cycle when an instruction's operands reside in the same bank of the register file; the compiler's allocation determines how often it happens.

Intuition

Before Volta a warp was one thread wearing thirty-two hats; Volta made it thirty-two threads that happen to share a datapath, and the hardware now keeps thirty-two places in thirty-two programs. The microbenchmarks are the other half of the same lesson: the datasheet tells you how wide the machine is, and only a stopwatch tells you how far away its memory is.

9 Systems: interconnect, supercomputers, alternatives (slides 25–27)

9.1 Slide 25: Beyond PCIe: NVLink, the DGX-1 cube, NVSwitch

Data-parallel training and multi-GPU HPC made the link between GPUs a first-class part of the architecture.

What the slide shows On the left, the topology of the first DGX-1 (2016): eight P100 modules drawn as two groups of four. Within each group every GPU is joined to the other three by solid lines (the ring plus both diagonals); four dashed lines join each GPU to its twin in the other group. The caption explains the drawing: the solid lines are three of a P100's four NVLink links, used within its group; the dashed line is the fourth, to the other group; the result is 160 GB/s of link bandwidth per GPU and a path of at most two hops between any pair. A small panel below, "Why not PCIe", gives the motivating arithmetic: PCIe 3.0 x16 moves 16 GB/s each way and is shared with everything else on the root complex, so an all-reduce of a 100 MB gradient over eight GPUs costs tens of milliseconds per training step. The right panel lists the numbers, all vendor-reported†: PCIe 3.0 x16 at 15.75 GB/s per direction; NVLink 1.0 on P100 at 20 GB/s per direction per link with four links, 160 GB/s per GPU, with POWER8 but not x86 CPUs on the other end; NVLink 2.0 on V100 at 25 GB/s per direction with six links, 300 GB/s per GPU, and cache coherence and address translation services with POWER9; NVSwitch in the DGX-2 (March 2018), an 18-port crossbar at 50 GB/s per port, twelve of which connect sixteen V100s at full bandwidth with 2.4 TB/s of bisection bandwidth and 512 GB of HBM2 as one address space; and the point of it all for training, that linked GPUs behave as one large GPU for data-parallel stochastic gradient descent.

Details The interconnect became part of the architecture because the two workloads that paid for compute GPUs both needed more than one of them. In HPC, a problem is decomposed across GPUs and neighbors exchange boundary data every time step; in deep learning, data-parallel training gives each GPU a slice of the batch and requires the gradients to be summed across all of them before every weight update. Both patterns move a volume of data proportional to the problem (the halo, or the model's parameters) at a frequency set by the compute speed, and as the GPUs got faster the link did not keep up. PCIe 3.0, introduced in 2012, provides 15.75 GB/s in each direction on sixteen lanes; a Pascal GPU can read its own HBM2 at 720 GB/s, forty-five times faster. The ratio means that a kernel which touches data on a neighboring GPU through PCIe is bandwidth-bound by the link, not the memory, by more than an order of magnitude.

The arithmetic in the slide's small panel is worth doing explicitly, because it is the one that drove the design. A ring all-reduce over p GPUs sends each GPU $2(p-1)/p$ times the data volume, so a 100 MB gradient over eight GPUs moves $2 \times 7/8 \times 100 \approx 175$ MB through each GPU's link. At the PCIe rate of about 16 GB/s per direction that is roughly 11 ms if the link is uncontended and nothing else is on the bus; in practice the CPU's root complex is shared with the other GPUs, the network card and storage, so the realistic figure is several times higher, tens of milliseconds. That is comparable to the compute time of a training step for the networks of the day, so without a better link a large part of each step would be communication. At 160 GB/s the same transfer takes about 1.1 ms (derived), and the step is compute-bound again. The gain is not in peak FLOP/s but in the fraction of them usable.

NVLink 1.0, introduced with GP100 (slide 20), is a point-to-point link: each link is eight differential pairs at 20 Gb/s per pair in each direction, two sub-links per link, giving 20 GB/s per direction and 40 GB/s bidirectional per link†; a P100 has four links, so 160 GB/s. Because NVLink is point-to-point, the topology is a design decision. With four links and eight GPUs a full crossbar is impossible, so the DGX-1 uses the hybrid cube mesh the slide draws: each GPU spends three links inside its group of four, making the group fully connected, and one link on the corresponding GPU in the other group. Any GPU can reach any other in at most two hops, and NCCL's ring all-reduce can be laid along the links so that every stage of the reduction uses link bandwidth rather than PCIe. On the host side, IBM's POWER8 (in the Minsky servers of 2016) had NVLink ports on the CPU, so CPU-to-GPU traffic also bypassed PCIe; x86 processors never did, which is one reason the two large NVLink systems of the period (slide 26) are POWER machines.

NVLink 2.0 on GV100 raised the per-link rate to 25 GB/s per direction and the link count to six, for 300 GB/s per GPU†, and added two features that matter beyond bandwidth: cache coherence, so that a GPU can cache CPU memory, and address translation services, so that a GPU can use the CPU's page tables. With POWER9 these make the CPU and GPU memories one coherent space, which is the hardware behind Summit's programming model (slide 26). The V100 DGX-1 uses the extra links to double some of the cube-mesh connections. NVSwitch, announced with the DGX-2 on March 27, 2018†, removes the topology problem by adding a switch chip: 18 NVLink ports, each at 25 GB/s per direction (50 GB/s bidirectional), about 900 GB/s of aggregate bidirectional bandwidth, two billion transistors on a 106 mm^2 die in TSMC's 12 nm FFN process (Hot Chips 30). Twelve of them, six per baseboard, connect sixteen V100 32 GB modules so that every GPU reaches every other at full NVLink rate; the bisection bandwidth is 2.4 TB/s (about 1.98 TB/s measured), the sixteen HBM2 stacks of 32 GB appear as a single 512 GB address space, and the machine was sold at $399,000 with about two PFLOP/s of FP16 tensor throughput†. The DGX-2 belongs to the Volta generation and marks the end of the lecture's period: from here on, the unit of GPU computing is the switched node, not the card.

Terms

PCI Express The serial bus through which a GPU is attached to a host; generation 3.0 (2012) carries 15.75 GB/s per direction on sixteen lanes, shared with everything else behind the CPU's root complex.

root complex The CPU-side PCIe controller through which all devices on the bus reach memory and each other; a shared bottleneck for multi-GPU traffic.

NVLink NVIDIA's point-to-point GPU interconnect: 20 GB/s per direction per link on Pascal (four links, 160 GB/s per GPU), 25 GB/s per direction on Volta (six links, 300 GB/s), with cache coherence to POWER9 CPUs.

hybrid cube mesh The DGX-1 topology: two fully connected groups of four GPUs, each GPU joined to its twin in the other group; at most two hops between any pair.

all-reduce The collective operation that sums a vector across all processes and leaves the result on each;

the step that synchronizes gradients in data-parallel training.

ring all-reduce An all-reduce in which processes pass partial sums around a ring; each process sends and receives $2(p-1)/p$ times the vector size regardless of p. The algorithm NCCL uses on NVLink.

data-parallel training Training in which every GPU holds the full model and a slice of each batch; gradients are all-reduced before each update.

NCCL The NVIDIA Collective Communications Library, which implements all-reduce and related collectives over NVLink and PCIe topologies.

NVSwitch An 18-port NVLink crossbar chip (2018; two billion transistors, 106 mm^2) that connects GPUs at full link bandwidth; twelve of them join the sixteen V100s of a DGX-2.

bisection bandwidth The bandwidth across a cut that divides a system's processors into two equal halves; 2.4 TB/s for the DGX-2.

DGX-1 NVIDIA's eight-GPU server (P100 in 2016, V100 in 2017) with GPUs in a hybrid cube mesh of NVLink and two x86 CPUs on PCIe.

DGX-2 NVIDIA's sixteen-GPU server (2018) with twelve NVSwitches, 512 GB of HBM2 and about 2 PFLOP/s of FP16 tensor throughput.

cache coherence The property that caches holding copies of the same memory see each other's writes; NVLink 2.0 provides it between a V100 and a POWER9.

Intuition

A single GPU is a city with wide streets; PCIe is the one two-lane bridge to the next city. NVLink adds four or six more bridges; NVSwitch replaces the bridges with a hub where every road meets. The city did not get faster, but the traffic that used to wait at the bridge no longer does.

9.2 Slide 26: GPUs in supercomputers: Tianhe-1A to Summit

Eight years from the first GPU-accelerated #1 to a machine where 97% of the FLOP/s sit on 27,648 V100s.

What the slide shows At the top left, one Summit node: two POWER9 processors joined by an X-bus, each connected by NVLink 2.0 to its three V100s, which are also linked to one another. The caption does the node's arithmetic: six GPUs at 7.8 TFLOP/s give 47 TFLOP/s of double precision at the boost clock, against the 42 TFLOP/s per node that ORNL quotes and about 1 TFLOP/s on the two CPUs (derived); the node holds 512 GB of DDR4, 96 GB of HBM2 and 1.6 TB of NVMe flash and connects to the fabric by dual-rail EDR InfiniBand. Below, a panel on Summit itself[†]: 4,608 nodes, 27,648 V100s, 200 PFLOP/s peak, 122.3 PFLOP/s on the HPL benchmark in June 2018 (first on the TOP500 list) rising to 148.6 by June 2019, 13 MW of design power; Sierra at Lawrence Livermore, the same design with four GPUs per node, second and third on the lists of the time; both from the 2014 CORAL procurement. The right panel traces the path: Tianhe-1A (NUDT, November 2010) with 7,168 Tesla M2050s and 2.57 PFLOP/s, first on the list, with TSUBAME 2.0 and Nebulae using the same GPUs; Titan (ORNL, November 2012), a Cray XK7 with 18,688 K20Xs beside 16-core Opterons, 17.6 PFLOP/s and first; Piz Daint (CSCS, 2016–17), a Cray XC50 with 5,320 P100s at 19.6 PFLOP/s, third in November 2017; and a note that the applications took most of a decade to port and that the U.S. exascale systems that followed are GPU machines.

Details The node is where the architecture of the preceding slides becomes a system, and the Summit node is the clearest example because it was designed around NVLink 2.0. Each POWER9 has NVLink ports as

well as PCIe, so each CPU talks to its three GPUs at NVLink rates and the GPUs talk to each other the same way; the two CPUs are joined by IBM's X-bus. The cache coherence of slide 25 means a CPU pointer is valid on the GPU and vice versa, which simplified porting: a code could move its hot loops to the GPUs and leave its data structures where they were, paying in bandwidth rather than in rewritten code. The balance of the node tells the story of the decade in one line: six GPUs at 7.8 TFLOP/s of FP64 each give $6 \times 7.8 = 46.8$ TFLOP/s (derived), while the two 22-core POWER9 processors contribute roughly half a TFLOP/s each, about 1 TFLOP/s together (derived, and approximate); about 98% of the node's double-precision arithmetic is on the GPUs. Scaled up, $4{,}608 \times 46.8 \approx 216$ PFLOP/s of GPU peak, a little above the 200 PFLOP/s the laboratory quotes, because the quoted figure uses the sustained rather than the boost clock; either way the GPUs are about 97% of the machine's peak, which is the subtitle's claim†. The memory tells the same story differently: 96 GB of HBM2 at 900 GB/s per GPU beside 512 GB of DDR4 at a fraction of that bandwidth, so the data a kernel is working on must be in HBM2 and the DDR4 is a staging area.

The path to Summit runs through three earlier machines, each marking a generation of the deck. Tianhe-1A, built by China's National University of Defense Technology, took first place on the November 2010 TOP500 list with 7,168 Fermi-generation Tesla M2050s and an HPL result of 2.566 PFLOP/s against a peak of 4.70† (NVIDIA's own release quoted the 2.507 PFLOP/s pre-list figure and did not name the GPU model). Fermi's ECC and half-rate FP64 (slide 16) were what made this possible; a GPU without them would not have been considered. The same list had TSUBAME 2.0 at Tokyo Tech fourth with 1.192 PFLOP/s on about 4,200 of the same GPUs, and Nebulae third†. Titan, the Cray XK7 at Oak Ridge, took first place in November 2012 with 18,688 Kepler K20Xs, one per node beside a 16-core AMD Opteron, at 17.59 PFLOP/s against a 27.1 PFLOP/s peak†; the TOP500 entry counts 261,632 "K20x cores", which is $18{,}688 \times 14$ SMX, a reminder that the list's core counts are not comparable across architectures. Titan was the first large machine on which most of the peak was unreachable without a port: an application that ran only on the Opterons saw a tenth of the node. Piz Daint at the Swiss National Supercomputing Centre is the Pascal entry: 5,320 Cray XC50 nodes with one P100 each, upgraded during 2016 and in production from December, third on both the June and November 2017 lists at 19.59 PFLOP/s and the fastest machine in Europe†.

Summit and Sierra were procured together in 2014 under the CORAL program, with delivery in 2018; Summit's HPL result was 122.3 PFLOP/s in June 2018, 143.5 in November 2018 and 148.6 in June 2019 as the machine was tuned, against a 200 PFLOP/s peak and a 13 MW design power (the June 2018 HPL run drew 8.8 MW)†. Sierra has 4,320 nodes of two POWER9s and four V100s, 17,280 GPUs and 125.6 PFLOP/s peak, with HPL results of 71.6 PFLOP/s (third, June 2018) and 94.64 (second, November 2018)†. The deck does not dwell on HPL itself, but one point belongs here: HPL is a dense LU factorization whose inner loop is matrix multiply, the one kernel of slide 15 that reaches the roof, so the gap between HPL and peak (about 60–75% on these machines) is a measure of the system, not the arithmetic. Most applications sit far lower, on the bandwidth-bound side of the roofline, which is why the port of the scientific codes was mostly a port of their memory access (slide 14): restructuring data so that warps coalesce, tiling so that reuse happens in shared memory, and reducing the traffic between CPU and GPU memory.

Terms

TOP500 The twice-yearly list of the fastest computers by the HPL benchmark; the slide's rankings are from its June and November lists.

HPL High-Performance Linpack, the dense LU factorization used to rank the TOP500; its inner loop is matrix multiply, so it measures a system's best case.

Rmax and Rpeak A TOP500 entry's achieved HPL performance and its theoretical peak; Summit's 122.3 against 200 PFLOP/s in June 2018.

Summit The Oak Ridge IBM–NVIDIA machine of 2018: 4,608 nodes of two POWER9s and six V100s,

200 PFLOP/s peak, first on the TOP500 from June 2018.

Sierra Lawrence Livermore's sibling of Summit: 4,320 nodes with four V100s each, 125.6 PFLOP/s peak.

CORAL The 2014 joint procurement by Oak Ridge, Argonne and Livermore under which Summit and Sierra were built.

POWER9 IBM's 2017 server processor, the host CPU of Summit and Sierra, with NVLink 2.0 ports on the CPU itself.

Titan Oak Ridge's Cray XK7 of 2012: 18,688 nodes with one K20X and one 16-core Opteron each, 17.59 PFLOP/s on HPL, first in November 2012.

Tianhe-1A NUDT's 2010 machine with 7,168 Tesla M2050s, 2.566 PFLOP/s on HPL, the first GPU-accelerated system at the top of the TOP500.

Piz Daint CSCS's Cray XC50 with 5,320 P100s, 19.59 PFLOP/s on HPL, third on the 2017 lists.

Cray XK7 and XC50 Cray's node designs with one GPU per node (K20X in the XK7, P100 in the XC50) and Cray's interconnect.

OpenACC A directive-based model for offloading loops to accelerators, used alongside CUDA in porting codes to Titan.

EDR InfiniBand The 100 Gb/s generation of the InfiniBand network; Summit's nodes have two rails of it.

Intuition

The node diagram is the deck in miniature: two general processors, six throughput processors, and the throughput processors hold almost all the arithmetic and almost all the bandwidth that matters. Porting a code to Summit meant moving it from the two boxes to the six, and the memory system decided how hard that was.

9.3 Slide 27: The alternatives, and why they lost the decade

AMD's GCN and Vega, Intel's Larrabee and Xeon Phi, and the "100×" argument: software and perf/W decided it.

What the slide shows Three panels. Top left, AMD: GCN (2012) as a SIMT design close to Fermi's, with 64-wide wavefronts, a scalar unit and caches; the FirePro S9150 (2014) with 2.5 TFLOP/s of FP64, more than the K40; Vega's Radeon Instinct MI25 (2017) with 12.3 TFLOP/s of FP32 and HBM2; and the software sequence Close To Metal (2006), Stream, OpenCL, then ROCm and HIP (2016), by which time CUDA had a decade of libraries and users. Bottom left, Intel: Larrabee (2008), many small x86 cores with 16-wide vector units and coherent caches meant to run graphics in software, cancelled as a GPU in December 2009; Xeon Phi, the same idea as a coprocessor (Knights Corner, 2012, about 1 TFLOP/s FP64) and then as a self-booting CPU (Knights Landing, 2016, up to 72 cores and 3.5 TFLOP/s FP64, 16 GB of MCDRAM at 490 GB/s), discontinued in 2018. Right, the arguments of the decade: the "100×" claims of early GPGPU papers against untuned CPU code, and Lee et al.'s ISCA 2010 re-tuning that found 2.5× on average over fourteen kernels for a GTX 280 against a Core i7-960; why GPUs won anyway (the real 3–5× compounded, performance per watt, a memory system built for bandwidth, consumer volume paying for the dies); and why it was NVIDIA's decade in particular (CUDA from 2007 and its libraries, the Tesla line with ECC and FP64 first, a compute

die every two years, every deep-learning framework on cuDNN by 2015).

Details AMD's hardware was never the problem. Graphics Core Next, which shipped in the Radeon HD 7970 at the turn of 2011 to 2012 (the exact dates are not verified here), replaced AMD's earlier VLIW shader design with a SIMT architecture that a Fermi programmer would recognize: compute units with four 16-wide SIMD units executing 64-thread wavefronts over four cycles, a scalar unit for addresses and loop control (an idea Volta adopted as separate INT32 lanes, slide 21), a local data share corresponding to shared memory, and read-write caches. In double precision the FirePro S9150, announced August 6, 2014, offered 2.53 TFLOP/s against the K40's 1.43 at base clock, with 16 GB of GDDR5 at 320 GB/s and 235 W†; the Radeon Instinct MI25 on the Vega architecture, announced December 12, 2016 with full specifications in June 2017, offered 12.3 TFLOP/s of FP32 and 24.6 of FP16 with 16 GB of HBM2 at 484 GB/s†, numbers between the P100 and the V100. What AMD lacked was the software stack and the decade of users on it. Close To Metal (announced in November 2006, a date not verified here) exposed the hardware at a low level and was replaced by Stream and then by OpenCL as AMD's primary model; OpenCL's vendor neutrality meant its libraries and tools were nobody's responsibility in particular. The Boltzmann Initiative of November 16, 2015 announced a CUDA-like stack, HCC and HIP, with a headless Linux driver†; it shipped as ROCm in 2016 (version 1.0 in April, a date not verified here; version 1.3 on November 14, 2016†), with HIP as a source-to-source route from CUDA. By then cuBLAS, cuFFT and cuDNN were in their sixth to ninth years, and the deep-learning frameworks had chosen.

Intel's two attempts tested a different hypothesis: that a many-core x86 with wide vectors and coherent caches could do the GPU's job while keeping the CPU's programming model. Larrabee (Seiler et al., SIGGRAPH 2008) proposed small in-order x86 cores with 16-wide vector units, four hardware threads each, and a coherent L2, running the graphics pipeline in software; Intel announced on December 4, 2009 that it would not ship as a discrete graphics product†. The design became the Xeon Phi coprocessor. Knights Corner launched November 12, 2012, as the 5110P: 60 cores at 1.053 GHz, 1.011 TFLOP/s of FP64, 8 GB of GDDR5 at 320 GB/s, 225 W, on a PCIe card† (the 7120P of 2013 had 61 cores and 1.2 TFLOP/s; the slide's "about 1 TFLOP/s" covers both). Knights Landing, launched June 20, 2016, dropped the host and booted as a CPU: the 7290 had 72 cores at 1.5 GHz, about 3.46 TFLOP/s of FP64 at 245 W, with 16 GB of on-package MCDRAM at 490 GB/s in front of DDR4†. It was competitive with the P100 on double precision and offered a familiar model, but the vector code that reached its peak was as hard to write as CUDA, the MCDRAM was a cache or a scratchpad that had to be managed, and the deep-learning frameworks did not target it. Intel announced its discontinuation in July 2018 with final shipments in July 2019†. Both of Intel's ideas returned in other forms: coherent CPU–GPU memory in NVLink 2.0 (slide 25), and high-bandwidth memory on a processor package in every later GPU and, eventually, in CPUs.

The "100×" argument is the part of the story that students should carry into reading any paper with a speedup in its abstract. Many early GPGPU results compared a hand-tuned GPU kernel with a straightforward single-threaded CPU implementation, and reported factors of 50 to 100. Lee et al. at Intel (ISCA 2010) took fourteen throughput kernels, tuned both the GTX 280 and the Core i7-960 implementations with the same care, and found the GPU 2.5× faster on average, with a range from a little below 1× to about 15× depending on the kernel's demands. The honest comparison is the ratio of the two machines' peaks: the GTX 280 had about 4.4× the memory bandwidth of the quad-core Nehalem (141.7 against 32 GB/s) and six to nine times its single-precision FLOP/s depending on how the GPU's dual-issue multiply is counted, so a well-tuned bandwidth-bound kernel should see roughly the bandwidth ratio and a compute-bound one somewhat more; the slide's 3–5× is the range most kernels fall in. The slide's point is that this smaller, honest number was enough. It was real on every generation, it compounded (a 4× advantage at the same power held from Fermi to Volta while both sides scaled), and three structural facts stood behind it: the energy argument of slide 13 favored many slow lanes over few fast ones; the GPU's memory system (GDDR5, then HBM2) was designed for bandwidth while the CPU's was designed for latency and capacity; and the consumer graphics market

paid for the development of each die. Why NVIDIA rather than AMD is then a question about software and timing: CUDA in 2007 with a C compiler and, within two years, cuBLAS and cuFFT; a Tesla line that had ECC and half-rate FP64 in 2010 when no competitor had both; a new compute die on a two-year cadence (GF100, GK110, GP100, GV100); and cuDNN in 2014, so that when AlexNet's successors needed a platform every framework targeted the same library (slide 19).

Terms

Graphics Core Next AMD's GPU architecture from 2012: compute units of four 16-wide SIMD units executing 64-wide wavefronts, a scalar unit, a local data share and read-write caches; the basis of the FirePro and Radeon Instinct compute cards.

wavefront AMD's name for the group of threads scheduled together, 64 wide on GCN against NVIDIA's 32-wide warp.

FirePro S9150 AMD's 2014 server GPU: 2.53 TFLOP/s FP64, 5.07 FP32, 16 GB of GDDR5 at 320 GB/s, 235 W.

Radeon Instinct MI25 AMD's 2017 Vega compute card: 12.3 TFLOP/s FP32, 24.6 FP16, 16 GB of HBM2 at 484 GB/s.

Close To Metal ATI's 2006 low-level interface to its GPUs for computing, the first of AMD's compute software stacks.

ROCm AMD's open compute platform from 2016 (Boltzmann Initiative, announced November 2015): a Linux driver, compilers and libraries in the CUDA mold.

HIP AMD's C++ dialect and source-to-source translator that lets CUDA code be compiled for AMD GPUs.

Larrabee Intel's 2008 design for a many-core x86 GPU with 16-wide vector units and coherent caches; cancelled as a graphics product in December 2009 and reborn as Xeon Phi.

Xeon Phi Intel's many-core x86 line: Knights Corner (2012), a PCIe coprocessor with about 1 TFLOP/s of FP64, and Knights Landing (2016), a self-booting 72-core CPU with 3 TFLOP/s and 16 GB of MCDRAM; discontinued in 2018.

MCDRAM The 16 GB of on-package high-bandwidth memory on Knights Landing, about 490 GB/s, usable as a cache or as addressable memory.

100x myth The pattern of early GPGPU papers comparing tuned GPU code with untuned CPU code; Lee et al. (ISCA 2010) measured 2.5× on average when both were tuned.

performance per watt Operations per joule, the figure of merit that decided architecture after Dennard scaling ended (slide 13).

Intuition

Three companies built wide machines; the one that won was the one whose users could already program it when the workload arrived. Read every speedup in a paper as a question about what the other side was allowed to do.

10 What carried forward, practice, summary, sources (slides 28–31)

10.1 Slide 28: What stayed constant for a decade, and what changed

From G80 to Volta the programming model and the SM template held; the counts, memory and datapaths moved.

What the slide shows Two panels. The left, "Constant", lists six things that held from G80 (2006) to Volta (2017): the programming model of grids, blocks, threads and 32-thread warps with shared memory and barriers inside a block, so that CUDA code from 2007 runs on a V100; SIMT execution with masks and a warp scheduler; latency hidden by resident warps and registers rather than by large caches or out-of-order issue; the SM template of register file, schedulers, FP32 lanes, load/store units, special function units and on-chip SRAM; a device with its own memory driven from a host; and the constraint behind all of it, energy per operation. The right, "Changed", lists six things that moved: scale (128 to 5,120 lanes, 0.68 to 21.1 billion transistors, 86 to 900 GB/s, at clocks between 0.75 and 1.53 GHz); generality followed by specialization (FP64, caches, ECC and pointers in 2008–10; FP16 and tensor cores in 2016–17); memory technology (GDDR3 to GDDR5 to stacked HBM2, and a unified virtual address space with page migration); the interconnect (PCIe to NVLink to NVSwitch); thread semantics (one program counter per warp to one per thread); and the customer (graphics, then HPC, then deep learning).

Details The constant list is a claim about what a GPU *is*, as opposed to what any particular one contains, and each item can be checked against the earlier slides. The programming model (slides 9–10) is the most consequential constant: because a kernel is written for one thread and launched as a grid of blocks, and because blocks are independent, the same source scales from the 16 SMs of G80 to the 80 of GV100 without change, and the hardware is free to schedule blocks wherever resources exist. Binary compatibility did not hold (each generation has its own machine code, SASS, reached through the PTX intermediate form), but source compatibility and, more importantly, performance-model compatibility did: the advice of 2008 (coalesce, use shared memory, avoid divergence, keep occupancy up) is the advice of 2017 with the numbers changed. SIMT with masks (slide 11) and latency hiding by multithreading (slide 12) are the execution model that makes the programming model cheap to implement, and the SM template (slides 10 and 21) is the physical form both took; Fermi fixed it, and Kepler, Maxwell, Pascal and Volta varied the counts. The host-device split (slides 9 and 20) is the item on the list under the most pressure by 2017, as the next slide's questions say, but through Volta every GPU in the deck is a device with its own memory and its own address space, reached by copies or by page migration. The sixth constant is the one the others follow from: the energy cost of moving a bit relative to computing on it (slide 13) is what makes wide, slow, data-local designs win, and it did not change direction across the decade.

The changed list is the record of what that constraint did to the numbers. Scale is the plainest: forty times the lanes and thirty times the transistors at a constant clock, because clock frequency stopped scaling with Dennard scaling around 2005 and the only way to spend transistors was in parallel. Generality came first because the early customers were scientists: GT200's FP64 unit, Fermi's caches, ECC and 40-bit unified addressing with pointers, Kepler's dynamic parallelism and Hyper-Q were all features that made the device behave like a processor a compiler and an operating system could target. Specialization came back when the customer changed: Pascal's packed FP16 and Volta's tensor cores (slides 20 and 23) are datapaths that only matrix-heavy workloads use, and the compute die spent area on them because by 2016 deep learning paid for the compute die. Memory technology moved from GDDR3 at 86 GB/s on a 384-bit bus to HBM2 at 900 GB/s on 4,096 bits, which the roofline (slide 15) shows was not enough to keep pace with arithmetic: bytes per FLOP fell from 0.25 to 0.06 even as bandwidth rose tenfold. The interconnect (slide 25) changed the unit of design from the card to the node, and the thread semantics change in Volta (slide 24) is the one place the execution model itself moved, to let diverged lanes synchronize. The customer line is the deck's historical thesis compressed: each of the four moves of slide 4 was made for the customer of the time, and the hardware of 2017 shows the layering, with the graphics pipeline still present on the compute die, the HPC

features of Fermi still present, and the tensor cores on top.

Terms

source compatibility The property that a CUDA program written for one generation compiles and runs on later ones; held from G80 through Volta, unlike binary compatibility.

PTX NVIDIA's virtual instruction set, into which CUDA compiles and from which the driver generates each generation's machine code (SASS); the mechanism behind forward compatibility.

SASS The native machine code of a particular NVIDIA GPU generation, produced from PTX by the driver or offline.

performance-model compatibility The property that the rules for writing fast code (coalescing, shared-memory reuse, occupancy, avoiding divergence) held across generations with only the numbers changing.

host-device model The arrangement in which a CPU program launches kernels on a GPU that has its own memory; constant through Volta and the item under most pressure after it.

specialization Spending die area on units that serve one class of workload (FP16 datapaths, tensor cores) rather than on general lanes; the direction the compute die took from 2016.

unified virtual addressing A single address space spanning CPU and GPU memory, introduced at 40 bits on Fermi and extended to 49 bits with page migration on Pascal.

Intuition

Ask of any GPU feature whether it belongs to the constant list or the changed list. If a lecture on Volta still needs the Fermi SM diagram to explain it, the diagram is the architecture and the rest is a parts list.

10.2 Slide 29: In practice: reading a spec sheet, estimating, measuring

First-order performance of a GPU kernel follows from four numbers on the datasheet and one on the kernel.

What the slide shows Three columns of rules, each a short paragraph. "Reading a spec sheet": find the SM count and lanes per SM (peak FLOP/s is SMs times lanes times two times the clock), the memory bandwidth, the memory capacity, the registers and shared memory per SM, and the host link; check which precision a peak refers to and whether it uses a boost clock; divide peak FLOP/s by bandwidth to get the ridge point; ignore the graphics figures (texture rate, ROPs, pixel fill). "Estimating a kernel": count the bytes the kernel must move from DRAM once and the FLOPs it performs; their ratio is its arithmetic intensity; left of the ridge the time is bytes over bandwidth, right of it FLOPs over peak; then check the conditions for reaching the roof (enough blocks, coalesced loads, no divergence, enough warps or instruction-level parallelism for the latency, few enough registers for the occupancy); a kernel at 60–80% of its roofline is done. "Measuring": use the profiler for achieved occupancy, DRAM throughput against peak, L2 hit rate, warp stall reasons and divergence; compare measured bytes with compulsory bytes; compare achieved FLOP/s with the roofline at the measured intensity rather than with peak; microbenchmark what the datasheet omits; and report the GPU, clocks, precision and problem size with every number.

Details *Reading a spec sheet.* A datasheet for a compute GPU carries a dozen numbers, of which five determine first-order performance. The first is peak arithmetic, and it should be recomputed rather than read: for the V100 SXM2, 80 SMs $\times$ 64 FP32 lanes $\times$ 2 FLOP per fused multiply-add $\times$ 1.53 GHz $=$ 15.7 TFLOP/s (derived), which matches the sheet; the same formula with 32 FP64 lanes gives 7.8, and with 8 tensor cores at 64 FMAs each gives 125 (slide 23). Recomputing exposes two things the sheet may not say:

which clock was used (the K40's 4.29 TFLOP/s is at its 745 MHz base clock, 5.04 at the 875 MHz boost, and a sheet may quote either), and which precision the headline refers to (the "120 TFLOP/s" of Volta's launch is tensor FP16, not FP32). The second number is memory bandwidth, the third capacity (which decides whether the problem fits without going to the host), the fourth the per-SM resources (registers and shared memory, which set occupancy and tile sizes), and the fifth the host link (PCIe 3.0 at 16 GB/s per direction or NVLink at 80–150), which decides how expensive it is to move data in and out. Dividing the first by the second gives the ridge point, 17 FLOP/B for the V100's FP32 and 139 for its tensor cores (slide 15); any kernel whose intensity is below the ridge cannot use the arithmetic no matter how it is written. The remaining figures on the sheet, texture fill rate, raster output units, pixels per second, describe the graphics pipeline and have no bearing on a compute kernel.

Estimating a kernel. The roofline estimate needs one number about the kernel, its arithmetic intensity, and the way to get it is to count compulsory traffic: the bytes that must cross from DRAM at least once because the kernel reads or writes them. Take a seven-point stencil on a 512^3 grid of double-precision values, one sweep. Each output point needs seven neighbors and does about eight floating-point operations (six adds and a multiply, counted as roughly eight); with a cache or shared-memory tiling that captures the reuse between neighbors, each input is read from DRAM about once and each output written once, so the compulsory traffic is 2×8 bytes per point and the intensity is about $8/16 = 0.5$ FLOP per byte. That is far left of the V100's ridge of 17, so the kernel is bandwidth-bound and the estimate is time $\approx$ bytes/bandwidth: $512^3 \times 16$ bytes $= 2.15$ GB, over 900 GB/s, about 2.4 ms per sweep (derived), at an achieved rate of about 0.5 TFLOP/s, 7% of the FP64 peak, which is the right answer and not a sign of a bad kernel. Had the same kernel been written without tiling, so that each point's seven neighbors were fetched separately, the traffic would be about four times larger and the time about four times longer. For a kernel right of the ridge, say a large SGEMM at intensity 100, the estimate is FLOPs over peak; a $4{,}096^3$ SGEMM does $2 \times 4{,}096^3 = 137$ GFLOP and takes about 8.8 ms at 15.7 TFLOP/s (derived), or about 1.1 ms on tensor cores if its tiles are shaped for them. The conditions for reaching either roof are the deck's earlier rules: enough blocks that every SM has several resident (slide 10), coalesced loads (slide 14), warps that do not diverge (slide 11), enough resident warps or independent loads per thread to cover 400 to 1,000 cycles of memory latency (slides 12 and 24), and a register count per thread that permits that occupancy. A measured 60–80% of the applicable roof is the usual ceiling once these are satisfied; chasing the rest costs more than it returns.

Measuring. The profiler (nvprof through CUDA 10, Nsight Compute after) answers the questions the estimate raises. Achieved occupancy against theoretical says whether the launch filled the SMs; DRAM throughput against peak says whether a bandwidth-bound kernel is at its roof; the L2 hit rate and the ratio of measured to compulsory bytes say whether traffic is wasted on uncoalesced or repeated loads (a ratio of 1 is the goal; 4 means the stencil above was not tiled); warp stall reasons (memory dependency, execution dependency, barrier, instruction fetch) say where the issue slots went; and branch efficiency says whether warps diverged. The single most common error is to compare achieved FLOP/s with peak: a stencil at 7% of FP64 peak is at 100% of its roofline, and a profiler reading of "low compute utilization" is expected. Anything the datasheet does not state, which includes every latency and the shared-memory bank structure, is measured by microbenchmark, as Wong et al. (2010) did for GT200 and Jia et al. (2018) for Volta (slide 24). Finally, every reported number should carry the GPU model, the clocks at which it ran (boost behavior varies with temperature and power), the precision, and the problem size, because without them the number cannot be placed on a roofline and so cannot be interpreted.

Peak FLOP/s = SMs $\times$ lanes per SM $\times$ 2 $\times$ clock; V100 FP32: $80 \times 64 \times 2 \times 1.53\,\text{GHz} = 15.7$ TFLOP/s. Ridge point = peak FLOP/s / bandwidth; V100 FP32: $15.7 \times 10^{12} / 900 \times 10^{9} = 17.4$ FLOP/B. Intensity = FLOPs / compulsory bytes; a tiled seven-point FP64 stencil: $8/16 = 0.5$ FLOP/B. Time estimate: $\max\big(\text{bytes}/\text{bandwidth},\ \text{FLOPs}/\text{peak}\big)$; 512^3 stencil on a V100: $2.15\,\text{GB}/900\,\text{GB/s} \approx 2.4$ ms (derived).

Terms

datasheet The vendor's one- or two-page specification of a product; the source of the peak, bandwidth, capacity and power figures, and of the dagger marks in this deck.

boost clock A clock frequency above the base that a GPU sustains when power and temperature allow; vendor peaks are often quoted at it (the K40's 5.04 against 4.29 TFLOP/s).

arithmetic intensity FLOPs performed per byte of DRAM traffic; the kernel's coordinate on the roofline's horizontal axis.

compulsory traffic The bytes a kernel must move from or to DRAM at least once because it reads or writes them; the denominator of the intensity a good implementation can reach.

seven-point stencil The finite-difference pattern that updates each grid point from itself and its six axis neighbors; the standard example of a bandwidth-bound kernel.

achieved occupancy The profiler's measured average of resident warps per SM against the maximum, as opposed to the theoretical occupancy computed from resource use.

warp stall reasons The profiler's breakdown of why a scheduler could not issue: waiting on memory, on a previous instruction, at a barrier, on instruction fetch.

nvprof and Nsight Compute NVIDIA's command-line profiler through CUDA 10 and its kernel-level successor, which report occupancy, throughput, cache and stall metrics.

branch efficiency The profiler's measure of the fraction of warp lanes active across executed branches; divergence lowers it.

Intuition

Four numbers from the sheet give you the roof; one count from the kernel tells you where under it you can stand. Everything the profiler says afterward is an explanation of the distance between where you are and where the roof is.

10.3 Slide 30: Summary: what to take from this lecture

A GPU is a throughput machine: many simple lanes, many threads in flight, a wide memory, and a model that matches.

What the slide shows Two panels: eight things to remember, numbered, and four questions the next lecture inherits, with a footnote pointing to these notes, their exercises and the reading list.

Details The eight points, expanded. First, the work was always regular and independent: pixels in 1999, threads in 2007, matrix tiles in 2017; every design choice in the deck exploits that regularity, and a workload without it (pointer chasing, irregular graphs) is where GPUs do least well. Second, the four moves of slide 4 in one sentence: fixed-function stages became programmable (2001), programmable stages were unified into one pool (2005–06), the pool became a general processor under CUDA (G80, 2006–07), and the general processor grew a specialized matrix unit (Volta, 2017), each move forced by the one before. Third, SIMT: one instruction fetched and decoded per 32 lanes, each lane programmed as a thread; divergence serializes the paths and costs issue slots, and uncoalesced access multiplies memory transactions and costs bytes. Fourth, latency is hidden by having thousands of threads resident, each with its own registers, so that some warp is always ready; occupancy is therefore a register budget, and the register file is larger than the cache. Fifth, energy set the design: one instruction per 32 operations amortizes the control cost, large register files and shared memory keep operands within a few picojoules of the ALUs, and wide slow lanes beat few fast ones

once clock scaling ended. Sixth, the roofline is the model to carry: attainable performance is the lesser of peak and bandwidth times intensity, and the ridge rose from 4 FLOP/B on G80 to 17 on V100's FP32 lanes and 139 on its tensor cores, so each generation pushed more kernels into the bandwidth-bound region. Seventh, Fermi (2010) made the GPU a processor with caches, ECC, FP64 and pointers, and Pascal and Volta (2016–17) made it a deep-learning engine with FP16, HBM2, NVLink and tensor cores. Eighth, software decided the decade: CUDA from 2007, then cuBLAS, cuFFT and cuDNN, then frameworks built on them, so that the hardware with the best libraries won against hardware with equal or better peaks.

The four questions, expanded. How far can the tensor core go: Volta's $4 \times 4 \times 4$ tile with FP16 inputs became larger tiles and more formats in later generations, and the ridge for the matrix unit kept rising; the consequence is that the rest of a program, everything that is not a large matrix multiply, becomes relatively more expensive, and the balance between the matrix unit and the general lanes is a live design question. Where does the memory go: 900 GB/s against 125 TFLOP/s is 0.007 bytes per FLOP (derived), the lowest ratio in the deck, and stacked memory got taller and faster afterward without closing the gap; whether to put more memory on the package, more cache on the die, or more of the problem in registers is the same question slide 13 asked. Is the GPU still a device: NVLink coherence with POWER9 and NVSwitch (slide 25) made the node, not the card, the unit of design, and the host-device model of slide 9, with its copies and page migrations, is the constant of slide 28 under most pressure. Is 32 the right warp: AMD chose 64 for GCN and later offered 32 as well; a wider group amortizes instruction overhead over more lanes and pays more for divergence, a narrower one the reverse, and the right answer depends on how regular the workload is, which is the energy argument of slide 13 meeting the programmer's data.

Terms

regular work Computation in which many elements undergo the same operations with predictable, mostly contiguous memory access; the assumption every GPU design choice exploits.

matrix engine A datapath that performs a whole small matrix multiply-add per instruction (the tensor core); the specialization Volta added to the general lanes.

bytes per FLOP Memory bandwidth divided by peak arithmetic, a machine's balance; 0.25 on G80, 0.057 on V100's FP32 lanes, 0.007 against its tensor cores.

warp width The number of threads scheduled together, 32 on NVIDIA and 64 on AMD's GCN; a trade between instruction overhead and divergence cost.

Intuition

If the eight points compress further, they compress to this: a GPU trades a fast thread for many slow ones because moving data costs more than computing on it, and everything from warps to tensor cores is a consequence. The four questions are the places where that trade is being renegotiated.

10.4 Slide 31: Sources

Architecture papers and whitepapers first, then history, then the systems and the debates; † marks vendor material.

What the slide shows Two columns of citations in small type, grouped under four headings: architecture papers (the IEEE Micro, ACM Queue and CACM articles by the designers, from Lindholm et al. 2008 on G80 to Choquette et al. 2018 on Volta and Dally, Keckler and Kirk's 2021 retrospective, with the NVIDIA whitepapers, datasheets and Hot Chips talks marked as vendor material); graphics history (Clark 1982, Akeley 1993, the GeForce 3 and GeForce 6800 papers, Cg, the Xbox 360 architecture, Direct3D 10, Brook, and Peddie's three-volume history); microarchitecture, performance and energy (the microbenchmarking papers

of Wong, Mei and Chu, and Jia; Volkov; the roofline; the analytical and simulation models; Lee et al. on the 100× myth; dark silicon and Dennard scaling; the tensor-core and mixed-precision papers; the two textbooks); and deep learning, systems and alternatives (AlexNet, cuDNN, the early GPU machine-learning papers, Larrabee, and the TOP500, laboratory, vendor and JEDEC documents). The full list with identifiers and links is Appendix D of these notes and the paper kit.

Details The deck's convention is that a number carries a dagger when its source is a datasheet, a product page, a press release, a benchmark list or press coverage, and no dagger when it comes from a peer-reviewed architecture paper or an NVIDIA whitepaper that documents the design itself. The distinction matters because the two kinds of source answer different questions: the whitepapers say how many SMs, lanes, registers and bytes of cache a die has and how its units are arranged, while the datasheets say what a particular product shipped with and at what clock, power and price. Where the two give different numbers, the notes say which is meant. The clearest case is the K40, whose 4.29 TFLOP/s single and 1.43 double are at the 745 MHz base clock on its datasheet, while NVIDIA's own Pascal whitepaper lists it at 5.04 and 1.68 using the 875 MHz boost clock; slide 18's table uses the base figures and says so.

Several numbers rest on third parties because the vendor never published them. Die areas are the main example: NVIDIA's whitepapers give 551 mm^2 for GK110, 601 for GM200, 610 for GP100 and 815 for GV100, but the G80 (484), GT200 (576) and GF100 (about 529) figures are measurements or pre-launch reports by reviewers, and slide 18 marks them as such. The GeForce 8800 GTX's power is another: 155 W is the figure on NVIDIA's product page, while launch-day reviews recorded a thermal design power of about 185 W quoted to them by the company; the deck uses the official figure and the notes flag the discrepancy (slide 3). Transistor counts for the earliest parts vary between NVIDIA's launch statements and the databases that cataloged them later (the GeForce 256's 23 million against 17 million, slide 5).

The systems numbers come from benchmark lists and the laboratories: TOP500 entries for Tianhe-1A, Titan, Piz Daint, Summit and Sierra, Oak Ridge's and Livermore's own system descriptions, and HPCwire's contemporaneous reporting for dates; these are reliable for what they measure (HPL results, node counts, dates) and are marked with daggers because they are not architecture documents. A few items could not be verified against a primary source while these notes were prepared and are stated with that caveat where they appear: the GeForce GTX 980 whitepaper could not be retrieved, so the GM204 figures come from NVIDIA's GTX 1080 whitepaper's comparison table; the release date of ROCm 1.0 (April 2016) and of ATI's Close To Metal (November 2006) are the commonly reported dates; the RIVA 128's 1997 release is likewise taken from secondary accounts. Two corrections made during preparation are worth recording because the uncorrected versions circulate widely: Wong et al.'s 2010 microbenchmarks were run on a GT200 (GTX 280), not a G80, and measured about 440 cycles of global memory latency; and the much-quoted energy figures of 20 pJ per double-precision operation and 16 nJ per DRAM access come from Dally's later talks at 28 nm, while Keckler et al.'s 2011 paper gives 50 pJ per FMA and more than 10 nJ for one instruction's operands from DRAM at 40 nm (slide 13).

Intuition

A specification sheet tells you what was sold; an architecture paper tells you what was built; a benchmark list tells you what was measured. The dagger is a reminder of which of the three a number came from, and the reading list is arranged so that each slide can be traced to at least one of the second kind.

A Notation

symbol or term	meaning	notes
SM	streaming multiprocessor	the unit that holds a block's threads; 16 on G80, 80 on V100
lane, CUDA core	one FP32 arithmetic pipeline	8 per SM on G80, 32 on Fermi, 192 on Kepler, 128 on Maxwell, 64 on Pascal and Volta
warp	32 threads scheduled as one	one instruction per warp per issue; AMD's wavefront is 64
N_{SM}, L, f	SMs, lanes per SM, clock	peak FLOP/s $= 2N_{SM}Lf$ (the 2 counts a fused multiply-add as two operations)
B	memory bandwidth, bytes per second	86.4 GB/s on G80, 900 GB/s on V100
I	arithmetic intensity, FLOP per byte	of a kernel: FLOPs done per byte moved from DRAM
P_{att}	attainable FLOP/s	$\min(P_{peak}, BI)$, the roofline
I^*	ridge point	P_{peak}/B; 4 FLOP/B on G80, 17 on V100 FP32, 139 on V100 tensor
W, W_{max}	resident warps, maximum resident warps	occupancy $= W/W_{max}$; $W_{max} = 24$ on G80, 48 on Fermi, 64 from Kepler
R	registers per SM	8,192 on G80, 32,768 on Fermi, 65,536 from Kepler (32-bit entries)
λ, t	throughput, latency	Little's law: items in flight $= \lambda t$
FMA	fused multiply-add	$d = ab + c$ with one rounding; the unit of peak FLOP/s
FP64, FP32, FP16	64-, 32-, 16-bit IEEE floating point	FP16: 1 sign, 5 exponent, 10 mantissa bits
HBM2, GDDR	stacked high-bandwidth memory; graphics DRAM	HBM2: 1,024 bits per stack; GDDR5: 32 bits per chip
PCIe, NVLink	host link; GPU-to-GPU link	PCIe 3.0 x16: 15.75 GB/s per direction; NVLink 1.0 link: 20 GB/s per direction
†	secondary source	datasheet, product page, press release, benchmark list, press

B Formulas at a glance

Peak arithmetic. $P_{\text{peak}} = 2N_{\text{SM}}Lf$ for FP32; replace L by the FP64 unit count for double precision and by $64 \times 8 = 512$ FMA per clock for Volta's tensor cores. G80: $2 \times 128 \times 1.35\,\text{GHz} = 346$ GFLOP/s (518 counting the dual-issue multiply). V100: $2 \times 80 \times 64 \times 1.53 = 15.7$ TFLOP/s; $2 \times 80 \times 32 \times 1.53 = 7.8$ FP64; $2 \times 80 \times 512 \times 1.53 = 125$ tensor.

Bytes per FLOP. B/P_{peak}: 0.25 on G80, 0.14 on Fermi (C2050), 0.067 on K40, 0.068 on P100, 0.057 on V100 FP32, 0.0072 on V100 tensor (all derived from vendor peaks). Every generation is more compute-rich relative to its memory.

Roofline. $P_{\text{att}}(I) = \min\big(P_{\text{peak}}, BI\big)$; ridge $I^* = P_{\text{peak}}/B$. SAXPY: $I = 2/12 = 0.167$ FLOP/B, so on V100 $P_{\text{att}} = 900 \times 0.167 = 150$ GFLOP/s. Tiled matrix multiply with a $T \times T$ tile held on chip: $I \approx T/4$ FLOP/B in FP32 ($2T^3$ FLOPs per $2 \cdot 4T^2$ bytes loaded), so $T = 64$ gives 16 FLOP/B and $T \geq 128$ clears V100's FP32 ridge.

Little's law for latency hiding. Bytes in flight needed to saturate memory $= Bt$. V100: $900\,\text{GB/s} \times 0.25\,\mu\text{s}$ (375 cycles, TLB hit) $= 220$ KB; $\times 0.67\,\mu\text{s}$ (1,029 cycles, TLB miss) $= 600$ KB; per SM, divide by 80; per 128-byte request, divide by 128: about 25 to 60 outstanding warp-loads per SM. More warps or more independent loads per warp both supply them.

Occupancy. $W = \min\big(W_{\max},\ \lfloor R/(32r) \rfloor,\ \lfloor S/s \rfloor \times w_b,\ b_{\max} w_b\big)$ where r is registers per thread, S shared memory per SM, s shared memory per block, w_b warps per block and $b_{\max}$ the block limit per SM. Fermi example: $r = 64$ gives $32{,}768/(32 \times 64) = 16$ warps of a possible 48.

Divergence cost. A warp whose lanes follow k distinct paths of lengths $\ell_1, \ldots, \ell_k$ takes $\sum_i \ell_i$ issue slots instead of $\max_i \ell_i$; the worst case is 32 paths, 32× slower.

Coalescing. Thread i reading 4-byte element $x[i]$: 32 threads cover 128 contiguous bytes, one line, one transaction. Stride s elements: $\min(32, \lceil 128s/128 \rceil)$ lines touched per warp-load; at $s = 32$, 32 lines and 1/32 of the fetched bytes used.

Shared-memory bank conflicts. 32 banks of 4 bytes; address a maps to bank $(a/4) \bmod 32$; a warp accessing a column of a 32×32 array of floats hits one bank 32 times; padding the row to 33 words spreads it over all banks.

Energy per operation (Keckler et al. 2011, 40 nm). FP64 FMA 50 pJ; 64-bit read from an 8 KB SRAM 14 pJ; wire 240 fJ per bit per mm, so 64 bits over 10 mm ≈ 154 pJ; DRAM access and interface 28–45 pJ per bit, so 64 bits $\approx$ 1.8–2.9 nJ, roughly fifty FMAs. One instruction fetch and decode per 32 lanes (SIMT) divides the instruction overhead by 32.

Interconnect. PCIe 3.0 x16: $16 \times 8\,\text{GT/s} \times 128/130/8 = 15.75$ GB/s per direction. NVLink 1.0: 8 lanes × 20 Gb/s = 20 GB/s per direction per link, 4 links = 160 GB/s bidirectional per P100. NVLink 2.0: 25 GB/s per direction per link, 6 links = 300 GB/s per V100. NVSwitch: 18 ports × 50 GB/s = 900 GB/s.

Ring all-reduce time (order of magnitude). For n GPUs and a buffer of M bytes, each GPU sends and receives $2(n-1)M/n$ bytes; at link bandwidth b the time is about $2(n-1)M/(nb)$. $M = 100$ MB, $n = 8$: 11 ms at 16 GB/s (PCIe) against 1.1 ms at 160 GB/s (NVLink 1.0), ignoring latency and contention.

Supercomputer peak. Nodes × GPUs per node × per-GPU FP64 peak: Summit $4{,}608 \times 6 \times 7.8 = 216$

PFLOP/s on GPUs (the quoted 200 PFLOP/s uses a lower sustained clock); its HPL result was 122.3 PFLOP/s in June 2018, 57% of that figure.

C Exercises

1. **Peak rates.** From the generation table (slide 18), recompute the FP32 peak of each die from SMs $\times$ lanes $\times$ 2 $\times$ clock, looking up the boost clock in the datasheet, and compare with the vendor figure. Which columns disagree by more than 5%, and why (base against boost clock, dual-issue MUL on G80)?

2. **Bytes per FLOP.** Compute B/P_{peak} for every column of the table in FP32 and, where it exists, in FP64 and tensor precision. Plot it against year. What would the memory bandwidth of V100 have to be to match G80's ratio?

3. **Roofline.** Draw the roofline of a Tesla K40 (4.3 TFLOP/s, 288 GB/s) and place SAXPY (0.167 FLOP/B), a 7-point stencil (about 1 FLOP/B if nothing is cached, about 3 with ideal reuse), a large FFT (about 8), and SGEMM with 64×64 tiles. Which of these could use a GPU with twice the arithmetic and the same memory?

4. **Little's law.** A Fermi C2050 has 144 GB/s of bandwidth and about 500 cycles of global-memory latency at 1.15 GHz. How many bytes must be in flight to saturate memory? How many 128-byte warp-loads per SM (14 SMs)? If each thread issues one load before stalling, how many warps per SM are needed, and is that within the 48-warp limit?

5. **Occupancy.** A kernel uses 96 registers per thread and 24 KB of shared memory per block of 256 threads on a Kepler SMX (65,536 registers, 48 KB shared, 2,048 threads, 16 blocks). Compute the occupancy limit imposed by each resource and the resulting number of resident warps. Which resource binds, and what change would relax it?

6. **Divergence.** A warp executes `if (i % 2 == 0) f(); else g();` where f takes 40 instructions and g 60. How many issue slots does the warp take? How many if the condition were `i < 16`? Propose a remapping of thread indices to data that removes the divergence in the first case.

7. **Coalescing.** Thirty-two threads read 4-byte elements with stride 2, stride 8, and stride 32. For each, how many 128-byte lines are touched and what fraction of the fetched bytes is used? Repeat for a structure of three floats read field by field (array of structures) and for the same data stored as three arrays.

8. **Bank conflicts.** A 32×32 tile of floats in shared memory is read by column, thread t reading element (t, c). How many bank conflicts per warp? Show that padding each row to 33 floats removes them, and compute the shared-memory cost of the padding.

9. **Energy.** Using Keckler et al.'s 40 nm figures (50 pJ per FP64 FMA, 14 pJ per 64-bit SRAM read, 2.5 nJ per 64-bit DRAM access), estimate the energy of one SAXPY element computed (a) with both operands and the result in DRAM, and (b) with them in registers after one DRAM load each. What fraction of (a) is arithmetic? Why does a GPU's 300 W budget force the design toward (b)?

10. **SIMT amortization.** If an instruction fetch and decode costs about 100 pJ and is shared by 32 lanes, what is the per-operation control overhead on a GPU against a scalar in-order core that fetches per operation? Compare both to the 50 pJ FMA.

11. **Tensor cores.** Verify the 125 TFLOP/s figure for V100 from 8 tensor cores per SM, 64 FMAs per tensor core per clock, 80 SMs and 1.53 GHz. What tile size in a tiled GEMM gives an intensity above the tensor-core ridge of 139 FLOP/B if the operands are FP16 (2 bytes) and the accumulators FP32?

12. **Mixed precision.** FP16 has a 10-bit mantissa and a smallest normal value of about 6×10^{-5}. A gradient component of 10^{-7} is computed in FP16. What happens to it, and how does loss scaling by 2^{12} change the outcome? Why are the master weights kept in FP32?

13. **Interconnect.** A data-parallel training step must all-reduce 250 MB of gradients across 8 GPUs. Estimate the time with the ring formula at 16 GB/s (PCIe), 160 GB/s (NVLink 1.0) and 300 GB/s (NVLink 2.0). If one step's arithmetic takes 50 ms, what fraction of time is communication in each case?

14. **Summit.** From 4,608 nodes, 6 V100s per node and 7.8 TFLOP/s FP64 per GPU, compute the GPU peak and compare with the quoted 200 PFLOP/s and the June 2018 HPL figure of 122.3 PFLOP/s. What clock would make the GPU peak equal 200 PFLOP/s exactly?

15. **The 100× argument.** A paper reports a 60× speedup of a kernel on a GTX 280 (933 GFLOP/s, 142 GB/s) over a Core i7-960 (about 100 GFLOP/s peak with SSE, 32 GB/s). What speedup would you expect from the hardware ratios alone for a compute-bound kernel and for a bandwidth-bound one? What does the gap between 60× and those numbers tell you about the CPU code?

16. **Reading a spec sheet.** Take a datasheet for any GPU after Volta and extract the five numbers slide 29 asks for. Compute its ridge point in each supported precision and place SAXPY, a stencil and SGEMM on its roofline. State which precision's peak the headline TFLOP/s figure refers to.

D Reading list

The sources behind each part of the deck, in the order they are discussed, with one line on why each is worth reading. Entries with an arXiv identifier or a direct PDF link are downloaded by the paper kit's script; the rest are listed with a DOI or URL. Items marked *secondary source* are press releases, datasheets, product pages, benchmark lists or press coverage.

1. Overview and the long view (slides 1-4)

- **Dally, Keckler and Kirk (2021).** *Evolution of the graphics processing unit (GPU).* https://doi.org/10.1109/MM.2021.3113475. IEEE Micro 41(6). The architects' own twenty-year retrospective, from G80 to Ampere: why unified shaders, why SIMT, why the datapath specialized. The best single companion to this deck.
- **Nickolls and Dally (2010).** *The GPU computing era.* https://doi.org/10.1109/MM.2010.41. IEEE Micro 30(2). The state of the art at Fermi's launch: the Tesla line from G80 to GF100, the programming model, and the case that GPUs had become general processors.
- **Garland and Kirk (2010).** *Understanding throughput-oriented architectures.* https://doi.org/10.1145/1839676.1839694. CACM 53(11), open access. The clearest statement of the latency-versus-throughput framing that slide 2 uses: many threads, simple cores, wide memory.
- **Fatahalian and Houston (2008).** *A closer look at GPUs.* https://doi.org/10.1145/1400181.1400197. CACM 51(10). A short, precise account of how a GPU core differs from a CPU core (SIMD lanes, multithreading, shared state), written as G80 and CUDA arrived.
- **Owens, Houston, Luebke, Green, Stone and Phillips (2008).** *GPU computing.* https://doi.org/10.1109/JPROC.2008.917757. Proceedings of the IEEE 96(5); open copy at escholarship.org/content/qt0cv1p1nc/qt0cv1p1nc.pdf. The survey written at the moment GPGPU became GPU computing: hardware, programming models, applications.
- **Luebke and Humphreys (2007).** *How GPUs work.* https://doi.org/10.1109/MC.2007.59. IEEE Computer 40(2). A four-page primer on the pipeline and its programmable stages, for readers who need the graphics side of slides 5-6 explained from zero.
- **Peddie (2022).** *The History of the GPU: Steps to Invention.* https://doi.org/10.1007/978-3-031-10968-3. Springer, volume 1 of three. The pre-history: from early raster hardware to the first consumer 3D chips; dense with dates and die photographs.
- **Peddie (2022).** *The History of the GPU: Eras and Environment.* https://doi.org/10.1007/978-3-031-13581-1. Springer, volume 2. 3dfx to unified shaders; the industrial history behind the timeline on slide 4.
- **Peddie (2023).** *The History of the GPU: New Developments.* https://doi.org/10.1007/978-3-031-14047-1. Springer, volume 3. The 2010s and the turn to AI; overlaps the last third of this deck.
- **Hennessy and Patterson (2017).** *Computer Architecture: A Quantitative Approach, 6th edition, chapter 4 (data-level parallelism in vector, SIMD and GPU architectures).* https://www.elsevier.com/books/computer-architecture/hennessy/978-0-12-811905-1. Textbook. Chapter 4 unifies vector machines, SIMD extensions and GPUs in one vocabulary and works a Pascal example; the 5th edition (2011) does the same with Fermi.

2. The graphics pipeline, fixed and programmable (slides 5-6)

- **Clark (1982).** *The Geometry Engine: a VLSI geometry system for graphics*. https://doi.org/10.1145/800064.801272. SIGGRAPH 1982; open copy at web.stanford.edu/class/cs349g/p127-clark.pdf. The transform stage of the pipeline as a custom chip: the origin of fixed-function graphics hardware and of Silicon Graphics.
- **Akeley (1993).** *RealityEngine graphics*. https://doi.org/10.1145/166117.166131. SIGGRAPH 1993. SGI's high-end pipeline built from boards of Geometry Engines and raster chips; the fully fixed-function pipeline at its peak.
- **Smith (The Register) (1999).** *Nvidia unveils GeForce 256 (August 31, 1999)*. https://www.theregister.com/1999/08/31/nvidia_unveils_256cpu_cray_geforce/. Press, secondary source. The launch coverage with NVIDIA's claims as made: 'nearly 23 million transistors', 480 Mpixels/s, and the coining of 'graphics processing unit'. Component databases list 17 M transistors.
- **Lindholm, Kilgard and Moreton (2001).** *A user-programmable vertex engine*. https://doi.org/10.1145/383259.383274. SIGGRAPH 2001. The GeForce 3's programmable vertex shader: the instruction set, the register model, and why the stage was made programmable. Slide 6.
- **Mark, Glanville, Akeley and Kilgard (2003).** *Cg: a system for programming graphics hardware in a C-like language*. https://doi.org/10.1145/882262.882362. ACM TOG 22(3), SIGGRAPH 2003. The shading language that made shaders look like C and whose authors went on to CUDA; read with Kilgard's two-page summary, arXiv cs/0302013.
- **Kilgard (2003).** *Cg in two pages*. arXiv:cs/0302013. A compact statement of the Cg programming model; the arXiv copy is the easiest to obtain.
- **Montrym and Moreton (2005).** *The GeForce 6800*. https://doi.org/10.1109/MM.2005.37. IEEE Micro 25(2). The last large pre-unified NVIDIA design: separate vertex and pixel processors, Shader Model 3.0, and the load-balancing problem that G80 solved.
- **Andrews and Baker (2006).** *Xbox 360 system architecture*. https://doi.org/10.1109/MM.2006.45. IEEE Micro 26(2). The Xenos GPU (ATI, 2005), the first unified-shader design to ship, a year before G80.
- **Blythe (2006).** *The Direct3D 10 system*. https://doi.org/10.1145/1141911.1141947. SIGGRAPH 2006. The API side of unification: Shader Model 4.0's common shader core and why the fixed distinction between vertex and pixel programs was dropped.
- **Peddie (IEEE Computer Society, Chasing Pixels) (2021).** *AMD's unified shader GPU history*. https://www.computer.org/publications/tech-news/chasing-pixels/amd-unified-shader-gpu-history. Secondary source. Settles the 'first unified shader' question: Xenos first in a console (2005), G80 first on the PC with a common API (DirectX 10, 2006).

3. GPGPU and the unified G80 with CUDA (slides 7-8)

- **Buck, Foley, Horn, Sugerman, Fatahalian, Houston and Hanrahan (2004).** *Brook for GPUs: stream computing on graphics hardware*. https://doi.org/10.1145/1015706.1015800. ACM TOG 23(3), SIGGRAPH 2004; project page graphics.stanford.edu/papers/brookgpu. Streams and kernels compiled to shaders: the programming model that CUDA's first author carried to NVIDIA. Slide 7.
- **Owens, Luebke, Govindaraju, Harris, Krueger, Lefohn and Purcell (2007).** *A survey of general-purpose computation on graphics hardware*. https://doi.org/10.1111/

j.1467-8659.2007.01012.x. Computer Graphics Forum 26(1); open copy at escholarship.org/content/qt9ns2d70c/qt9ns2d70c.pdf. The catalog of what ran on shaders before CUDA, and of the tricks (scatter via vertex processors, multi-pass reductions) that CUDA made unnecessary.

- **Steinkraus, Simard and Buck (2005).** *Using GPUs for machine learning algorithms*. https://doi.org/10.1109/ICDAR.2005.251. ICDAR 2005. The earliest paper training neural networks on a GPU, through the graphics API; three of the authors' ideas reappear in cuDNN a decade later.
- **Chellapilla, Puri and Simard (2006).** *High performance convolutional neural networks for document processing*. https://inria.hal.science/inria-00112631. IWFHR 2006, open access. Convolution unrolled into matrix multiply and run on a GPU: the formulation every later framework used.
- **Lindholm, Nickolls, Oberman and Montrym (2008).** *NVIDIA Tesla: a unified graphics and computing architecture*. https://doi.org/10.1109/MM.2008.31. IEEE Micro 28(2). The canonical G80 paper: streaming processors, the SM, SIMT (the term is introduced here), the memory system, and the graphics and compute pipelines sharing one array. Slides 8 and 11.
- **Nickolls, Buck, Garland and Skadron (2008).** *Scalable parallel programming with CUDA*. https://doi.org/10.1145/1365490.1365500. ACM Queue 6(2), open access. The programming model as its designers explain it: threads, blocks, grids, shared memory, barriers, and why blocks are independent. Slide 9.
- **Kirk (2007).** *NVIDIA CUDA software and GPU parallel computing architecture (ISMM 2007 keynote)*. https://doi.org/10.1145/1296907.1296909. The first appearance of CUDA's architecture in the academic literature, four months after the public beta.
- **eWeek (2006).** *Nvidia launches new architecture for its graphics chips (November 8, 2006)*. https://www.eweek.com/networking/nvidia-launches-new-architecture-for-its-graphics-chips/. Press, secondary source. Confirms that the GeForce 8800 and CUDA were announced on the same day.
- **NVIDIA (2007).** *Tesla product overview: C870, D870, S870 (June 2007)*. https://www.nvidia.com/docs/io/43395/tesla_product_overview.pdf. Vendor datasheet. The first compute-only products: 128 thread processors, 1.5 GB, 76.8 GB/s, 'up to 518 gigaflops'.
- **Khronos Group (2008).** *The Khronos Group releases OpenCL 1.0 specification (press release, December 2008)*. https://www.khronos.org/news/press/the_khronos_group_releases_opencl_1.0_specification. Vendor press release. The vendor-neutral standard with CUDA's model and different names (work-items, work-groups), six months after Apple's proposal.
- **Wasson (The Tech Report) (2006).** *Nvidia's GeForce 8800 graphics processor (launch review)*. https://www.techreport.com/reviews/2006q4/geforce-8800/index.x?pg=1. Press, secondary source. The launch-day architecture walkthrough with NVIDIA's briefing numbers, including the 185 W figure given to reviewers (155 W became the official maximum board power).

4. The CUDA model and its mapping to the chip (slides 9-10)

- **Kirk and Hwu (2016).** *Programming Massively Parallel Processors: A Hands-on Approach, 3rd edition*. https://www.elsevier.com/books/programming-massively-parallel-processors/kirk/978-0-12-811986-0. Textbook. The course that goes with slides 9-10 and 14: the CUDA model, memory hierarchy, tiling, and performance considerations; the 4th edition (Hwu, Kirk and El Hajj, 2022) adds later architectures.

- **Harris (2007).** *Optimizing parallel reduction in CUDA*. https://developer.download.nvidia.com/compute/cuda/1.1-Beta/x86_website/projects/reduction/doc/reduction.pdf. Vendor tutorial, open. Seven versions of one kernel, each removing a specific inefficiency (divergence, bank conflicts, idle threads, instruction overhead); the single best introduction to GPU performance thinking.
- **Volkov and Demmel (2008).** *Benchmarking GPUs to tune dense linear algebra*. https://bebop.cs.berkeley.edu/pubs/volkov2008-benchmarking.pdf. SC 2008 (Test of Time award, 2019), open copy. Microbenchmarks of G80 and GT200 used to design an SGEMM at 60% of peak; the register-blocking lesson every later GEMM followed.
- **Wittenbrink, Kilgariff and Prabhu (2011).** *Fermi GF100 GPU architecture*. https://doi.org/10.1109/MM.2011.24. IEEE Micro 31(2). The Fermi SM of slide 10 described by its architects: dual warp schedulers, the configurable L1/shared array, the L2, ECC, and the graphics pipeline around them.
- **NVIDIA (2009).** *NVIDIA's next generation CUDA compute architecture: Fermi (whitepaper v1.1)*. https://www.ece.lsu.edu/koppel/gp/refs/NVIDIA_Fermi_Compute_Architecture_Whitepaper.pdf. Vendor whitepaper (mirror; the nvidia.com original is the same document). The G80/GT200/Fermi comparison table, the SM diagram, the memory hierarchy and the 48 warps per SM that slide 10 quotes.
- **Glaskowsky (2009).** *NVIDIA's Fermi: the first complete GPU computing architecture*. https://www.nvidia.com/content/PDF/fermi_white_papers/P.Glaskowsky_NVIDIAFermi-TheFirstCompleteGPUComputingArchitecture.pdf. Analyst whitepaper commissioned by NVIDIA; secondary source. Places Fermi against CPUs and earlier GPUs with unusual clarity about what 'general-purpose' required.

5. SIMT, latency hiding and the energy argument (slides 11-13)

- **Fung, Sham, Yuan and Aamodt (2007).** *Dynamic warp formation and scheduling for efficient GPU control flow*. https://people.ece.ubc.ca/aamodt/papers/wwlfung.micro2007.pdf. MICRO 2007, open copy. The first academic analysis of branch divergence in SIMT and a hardware proposal to regroup threads across warps; the cost model of slide 11.
- **Hong and Kim (2009).** *An analytical model for a GPU architecture with memory-level and thread-level parallelism awareness*. https://doi.org/10.1145/1555754.1555775. ISCA 2009. The MWP/CWP model: how many warps a memory system can serve at once and how many the compute pipeline needs; the formal version of slide 12.
- **Bakhoda, Yuan, Fung, Wong and Aamodt (2009).** *Analyzing CUDA workloads using a detailed GPU simulator*. https://people.ece.ubc.ca/aamodt/papers/gpgpusim.ispass09.pdf. ISPASS 2009, open copy. GPGPU-Sim, the simulator most GPU architecture research used for a decade, and its first findings on interconnect and memory bottlenecks.
- **Rogers, O'Connor and Aamodt (2012).** *Cache-conscious wavefront scheduling*. https://engineering.purdue.edu/tgrogers/publication/rogers-micro-2012/rogers-micro-2012.pdf. MICRO 2012, open copy. Why more resident warps can hurt: L1 thrashing, and a scheduler that limits the active set; the counterpoint to maximal occupancy.
- **Volkov (2010).** *Better performance at lower occupancy (GTC 2010 talk)*. https://www.nvidia.com/content/GTC-2010/pdfs/2238_GTC2010.pdf. Vendor conference slides, secondary

source. Instruction-level parallelism within a thread as a substitute for thread-level parallelism; the argument slide 12 summarizes.

- **Volkov (2016).** *Understanding latency hiding on GPUs (PhD thesis, UC Berkeley EECS-2016-143).* https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-143.pdf. Open access. The rigorous version of Little's law for GPUs, measured on Fermi through Maxwell; the reference for anyone modeling occupancy.
- **Keckler, Dally, Khailany, Garland and Glasco (2011).** *GPUs and the future of parallel computing.* https://www.cs.toronto.edu/~pekhimenko/courses/csc2231-f17/Papers/GPUs.pdf. IEEE Micro 31(5), open copy (DOI 10.1109/MM.2011.89). The energy tables behind slide 13: 50 pJ per double-precision FMA at 40 nm, 14 pJ per 64-bit SRAM read, 240 fJ per bit-millimeter of wire, 28-45 pJ per bit of DRAM access and interface, and the Echelon design that followed from them.
- **Esmaeilzadeh, Blem, St. Amant, Sankaralingam and Burger (2011).** *Dark silicon and the end of multicore scaling.* https://doi.org/10.1145/2000064.2000108. ISCA 2011. At fixed power only part of a chip can be active; the argument for specialization that the tensor core later embodied.
- **Dennard, Gaensslen, Yu, Rideout, Bassous and LeBlanc (1974).** *Design of ion-implanted MOSFETs with very small physical dimensions.* https://doi.org/10.1109/JSSC.1974.1050511. IEEE JSSC 9(5). The scaling law whose breakdown around 2005 ended clock-speed growth and made throughput architectures inevitable.
- **Bohr (2007).** *A 30 year retrospective on Dennard's MOSFET scaling paper.* https://doi.org/10.1109/N-SSC.2007.4785534. IEEE SSCS Newsletter 12(1). Short, readable account of why the law stopped holding (leakage, threshold voltage) and what replaced it.
- **Lee et al. (Intel) (2010).** *Debunking the 100X GPU vs. CPU myth: an evaluation of throughput computing on CPU and GPU.* https://www.cs.umd.edu/class/fall2019/cmsc714/readings/Lee-GPU.pdf. ISCA 2010, open copy. Fourteen kernels tuned on both a GTX 280 and a Core i7-960: 2.5x on average. The paper that made honest comparisons the norm; slide 27.

6. The memory system and the roofline (slides 14-15)

- **Williams, Waterman and Patterson (2009).** *Roofline: an insightful visual performance model for multicore architectures.* https://doi.org/10.1145/1498765.1498785. CACM 52(4); open copy at escholarship.org/content/qt78h8v7mr/qt78h8v7mr.pdf. The model of slide 15: attainable performance as the minimum of peak and bandwidth times intensity.
- **Wong, Papadopoulou, Sadooghi-Alvandi and Moshovos (2010).** *Demystifying GPU microarchitecture through microbenchmarking.* https://www.eecg.utoronto.ca/~moshovos/CUDA08/arx/microbenchmark_report.pdf. ISPASS 2010 (DOI 10.1109/ISPASS.2010.5452013), open technical-report copy. Pointer-chasing and dependent-chain measurements on GT200: about 440 cycles to global memory, 38 to shared memory, a 24-cycle arithmetic pipeline, the cache and TLB structure.
- **Mei and Chu (2017).** *Dissecting GPU memory hierarchy through microbenchmarking.* arXiv:1509.02308. IEEE TPDS 28(1). The same method applied to Fermi, Kepler and Maxwell: cache sizes, set structure, latencies; fills the gap between Wong (GT200) and Jia (Volta).
- **JEDEC (2016).** *JEDEC updates High Bandwidth Memory standard JESD235A (press release, January 12, 2016).* https://www.jedec.org/node/5916. Standards body press release. HBM2: eight independent 128-bit channels per stack, up to 256 GB/s and 8 GB per stack, pseudo-channel mode; the memory of Pascal and Volta.

- **JEDEC (2016).** *JEDEC announces publication of GDDR5X graphics memory standard JESD232 (January 21, 2016)*. https://www.jedec.org/node/5917. Standards body press release. The other 2016 path to bandwidth, used on the consumer GP104 instead of HBM2.
- **NVIDIA (2016).** *GeForce GTX 1080 whitepaper (GP104)*. https://kolegite.com/EE_library/datasheets_and_manuals/GPU/GeForce_GTX_1080_Whitepaper_FINAL.pdf. Vendor whitepaper (mirror). The consumer Pascal with GDDR5X; its comparison table is also the accessible source for GM204 (GTX 980) figures.

7. The generations: Tesla, Fermi, Kepler, Maxwell, Pascal (slides 16-21)

- **NVIDIA (2010).** *Tesla C1060 computing processor board specification (BD-04111-001_v06)*. https://www.nvidia.com/docs/io/43395/bd-04111-001_v06.pdf. Vendor datasheet. GT200 as a compute product: 240 cores at 1.296 GHz, 4 GB GDDR3 at 102 GB/s, 187.8 W.
- **NVIDIA (2010).** *Tesla C2050 / C2070 GPU computing processor datasheet (July 2010)*. https://www.nvidia.com/docs/io/43395/nv_ds_tesla_c2050_c2070_jul10_lores.pdf. Vendor datasheet. Fermi as shipped: 448 cores, 515 GFLOP/s double, 1.03 TFLOP/s single, 144 GB/s, 238 W.
- **NVIDIA (2012).** *NVIDIA's next generation CUDA compute architecture: Kepler GK110 (whitepaper v1.0)*. https://www.nvidia.co.jp/content/apac/pdf/tesla/nvidia-kepler-gk110-architecture-whitepaper-jp.pdf. Vendor whitepaper (mirror of the English document). The SMX, dynamic parallelism, Hyper-Q, GPUDirect RDMA, and the compute-capability table from Fermi to GK110.
- **NVIDIA (2014).** *Tesla K80 / K40 datasheet*. https://hpc-test.llnl.gov/sites/default/files/Tesla-K80K40-datasheet.pdf. Vendor datasheet (mirror). K40 at 4.29 / 1.43 TFLOP/s base and 5.04 / 1.68 boost; K80 at 8.74 / 2.91 with boost, 24 GB, 480 GB/s.
- **NVIDIA (2016).** *Tesla M40 datasheet (January 2016)*. https://images.nvidia.com/content/tesla/pdf/nvidia-teslam40-datasheet.pdf. Vendor datasheet. Maxwell for training: 3,072 cores, 7 TFLOP/s with boost, 288 GB/s, 250 W, and the FP64 rate that kept it out of HPC.
- **NVIDIA (2016).** *NVIDIA Tesla P100 whitepaper (WP-08019-001_v01.1)*. https://techpowerup.com/gpu-specs/docs/nvidia-gp100-architecture.pdf. Vendor whitepaper (mirror of the same document). GP100 in full: the SM, FP16, HBM2 and CoWoS, NVLink 1.0, unified memory with page faulting, compute preemption, and Table 1 comparing K40, M40 and P100. Slide 20.
- **NVIDIA (2016).** *NVIDIA delivers massive performance leap for deep learning, HPC applications with NVIDIA Tesla P100 accelerators (press release, April 5, 2016)*. https://www.globenewswire.com/news-release/2016/04/05/1149699/0/en/NVIDIA-Delivers-Massive-Performance-Leap-for-Deep-Learning-HPC-Applications-With-NVIDIA-Tesla-P100-Accelerators.html. Vendor press release. The launch claims as made: 5.3 / 10.6 / 21.2 TFLOP/s, 720 GB/s, 160 GB/s NVLink, 15.3 billion transistors.
- **Foley and Danskin (2017).** *Ultra-performance Pascal GPU and NVLink interconnect*. https://doi.org/10.1109/MM.2017.37. IEEE Micro 37(2). GP100 and NVLink by their architects; the Hot Chips 28 talk by the same authors (old.hotchips.org, HC28.22.121) has the floorplan and PHY details.
- **NVIDIA (2015).** *New NVIDIA hyperscale accelerators boost machine learning throughput for web data centers (press release, November 10, 2015)*. https://www.globenewswire.com/news-

release/2015/11/10/1149678/0/en/New-NVIDIA-Hyperscale-Accelerators-Boost-Machine-Learning-Throughput-for-Web-Data-Centers.html. Vendor press release. The M40 announcement, and the moment NVIDIA named deep-learning training as a product category of its own.

- **Krizhevsky, Sutskever and Hinton (2012).** *ImageNet classification with deep convolutional neural networks*. https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf. NIPS 2012, open access (also CACM 60(6), 2017). Trained 'between five and six days on two GTX 580 3GB GPUs'; the paper that redirected the architecture. Slide 19.
- **Chetlur, Woolley, Vandermersch, Cohen, Tran, Catanzaro and Shelhamer (2014).** *cuDNN: efficient primitives for deep learning*. arXiv:1410.0759. The library that tied every deep-learning framework to CUDA; convolution as GEMM with tiling in on-chip memory, the kernels the tensor core later accelerated.
- **Raina, Madhavan and Ng (2009).** *Large-scale deep unsupervised learning using graphics processors*. https://icml.cc/imls/conferences/2009/papers/218.pdf. ICML 2009, open copy. Deep belief networks and sparse coding on a GTX 280: the first widely read case that GPUs change what deep learning can afford.
- **Ciresan, Meier, Gambardella and Schmidhuber (2010).** *Deep, big, simple neural nets for handwritten digit recognition*. arXiv:1003.0358. Neural Computation 22(12). Plain multilayer perceptrons trained on GPUs beat the MNIST state of the art; evidence that compute, not cleverness, was the missing ingredient.
- **Tom's Hardware (Angelini) (2015).** *Nvidia GeForce GTX Titan X 12GB review (March 17, 2015)*. https://www.tomshardware.com/reviews/nvidia-geforce-gtx-titan-x-gm200-maxwell,4091.html. Press, secondary source. The GM200 launch: 8 billion transistors, 601 mm2, 3,072 cores, FP64 at 1/32; the consumer die that became the Tesla M40.

8. Volta and the tensor core (slides 22-24)

- **NVIDIA (2017).** *NVIDIA Tesla V100 GPU architecture (whitepaper WP-08608-001_v1.1)*. https://www.ece.lsu.edu/gp/refs/volta-architecture-whitepaper.pdf. Vendor whitepaper (mirror of the same document). The GV100 SM with its four processing blocks, tensor cores, independent thread scheduling, the unified L1/shared memory, NVLink 2.0, and Table 1 comparing P100 and V100. Slides 21-24.
- **Choquette, Giroux and Foley (2018).** *Volta: performance and programmability*. https://doi.org/10.1109/MM.2018.022071134. IEEE Micro 38(2). Volta by its architects: why the SM was repartitioned, how tensor cores fit the pipeline, and what independent thread scheduling cost and bought.
- **NVIDIA (2018).** *Tesla V100 data sheet (March 2018)*. https://images.nvidia.com/content/technologies/volta/pdf/tesla-volta-v100-datasheet-letter-fnl-web.pdf. Vendor datasheet. SXM2 against PCIe: 7.8 / 15.7 / 125 against 7 / 14 / 112 TFLOP/s, 300 against 250 W, 16 or 32 GB.
- **Jia, Maggioni, Staiger and Scarpazza (2018).** *Dissecting the NVIDIA Volta GPU architecture via microbenchmarking*. arXiv:1804.06826. Citadel's measurement of V100: 19-cycle shared memory, 28-cycle L1, about 193-cycle L2, 375-cycle global memory with a TLB hit and 1,029 with a miss, 4-cycle FP32 FMA, tensor-core behavior, and the instruction encoding. Slides 12 and 24.
- **Markidis, Der Chien, Laure, Peng and Vetter (2018).** *NVIDIA tensor core programmability, performance and precision*. arXiv:1803.04014. IPDPSW 2018. The WMMA API measured: achieved TFLOP/s

against the 125 peak, and the precision lost to FP16 inputs with and without FP32 accumulation.

- **Micikevicius et al. (2017).** *Mixed precision training*. arXiv:1710.03740. ICLR 2018. FP32 master weights, FP16 forward and backward passes, loss scaling; the recipe that made tensor cores usable for training without accuracy loss. Slide 23.
- **NVIDIA (via Edge AI and Vision Alliance) (2017).** *NVIDIA launches revolutionary Volta GPU platform (press release, May 10, 2017)*. https://www.edge-ai-vision.com/2017/05/nvidia-launches-revolutionary-volta-gpu-platform-fueling-next-era-of-ai-and-high-performance-computing/. Vendor press release (mirror). The launch claims: 21 billion transistors, 640 tensor cores, 120 TFLOP/s (later 125), 900 GB/s.

9. Systems: interconnect, supercomputers, the alternatives (slides 25-27)

- **Danskin and Foley (2016).** *Pascal GPU with NVLink (Hot Chips 28 slides)*. https://old.hotchips.org/wp-content/uploads/hc_archives/hc28/HC28.22-Monday-Epub/HC28.22.10-GPU-HPC-Epub/HC28.22.121-Pascal-GPU-DanskinFoley-NVIDIA-v06-6_7.pdf. Conference slides, open. The GP100 floorplan, the HBM2 and CoWoS package, and the NVLink physical layer and the DGX-1 topology. Slide 25.
- **Ishii and Wells (NVIDIA) (2018).** *NVSwitch and DGX-2 (Hot Chips 30 slides)*. https://www.old.hotchips.org/hc30/2conf/2.01_Nvidia_NVswitch_HotChips2018_DGX2NVS_Final.pdf. Conference slides, open. The 18-port NVSwitch (2 billion transistors, 106 mm2, 900 GB/s aggregate) and the DGX-2's 16-GPU fabric with 2.4 TB/s of bisection bandwidth.
- **HPCwire (2018).** *Nvidia ups hardware game with 16-GPU DGX-2 server and NVSwitch (March 27, 2018)*. https://www.hpcwire.com/2018/03/27/nvidia-ups-hardware-game-with-16-gpu-dgx-2-server-and-nvswitch/. Press, secondary source. The DGX-2 and V100 32 GB announcement with the figures as given at GTC 2018.
- **Oak Ridge Leadership Computing Facility (2018).** *Summit by the numbers (June 2018)*. https://www.olcf.ornl.gov/wp-content/uploads/2018/06/Summit_bythenumbers_FIN-1.pdf. Facility fact sheet. 4,608 nodes, 27,648 V100s, 200 PFLOP/s peak, 13 MW, the node layout of slide 26.
- **Lawrence Livermore National Laboratory (2018).** *Sierra platform page*. https://hpc.llnl.gov/hardware/platforms/sierra. Facility page. 4,320 nodes, 17,280 V100s, 125.6 PFLOP/s peak: Summit's sibling with four GPUs per node.
- **TOP500 (2010).** *TOP500 list, November 2010*. https://top500.org/lists/top500/2010/11/. Benchmark list. Tianhe-1A at 2.566 PFLOP/s, the first GPU-accelerated number one; TSUBAME 2.0 and Nebulae in the top five.
- **NVIDIA (2010).** *Three of the world's top five supercomputers powered by NVIDIA Tesla GPUs (press release, November 15, 2010)*. https://www.globenewswire.com/news-release/2010/11/15/1149888/0/en/Three-of-the-World-s-Top-Five-Supercomputers-Powered-by-NVIDIA-Tesla-GPUs.html. Vendor press release. 7,168 GPUs in Tianhe-1A and the Fermi-era systems; note that it quotes the pre-list 2.507 PFLOP/s figure.
- **TOP500 (2012).** *TOP500 list, November 2012*. https://top500.org/lists/top500/2012/11/. Benchmark list. Titan at 17.59 PFLOP/s with 18,688 K20Xs, number one.

- **TOP500 (2017).** *TOP500 list, November 2017*. https://top500.org/lists/top500/list/2017/11. Benchmark list. Piz Daint at 19.59 PFLOP/s with 5,320 P100s, number three; Titan still number five.
- **TOP500 (2018).** *TOP500 list, June 2018*. https://www.top500.org/lists/2018/06/. Benchmark list. Summit's first appearance at 122.3 PFLOP/s, number one; Sierra at 71.6.
- **TOP500 (2018).** *TOP500 list, November 2018*. https://top500.org/lists/2018/11/. Benchmark list. Summit at 143.5 PFLOP/s, Sierra at 94.64, Piz Daint at 21.23; the 148.6 figure often quoted for Summit is from June 2019.
- **TOP500 News (2017).** *Swiss deploy world's fastest GPU-powered supercomputer*. https://www.top500.org/news/swiss-deploy-worlds-fastest-gpu-powered-supercomputer. Benchmark-site news, secondary source. The Piz Daint P100 upgrade's timeline and node count.
- **Seiler et al. (Intel) (2008).** *Larrabee: a many-core x86 architecture for visual computing*. https://doi.org/10.1145/1360612.1360617. ACM TOG 27(3), SIGGRAPH 2008; open copy at courses.cs.washington.edu/courses/cse591n/08au/Papers/larrabee_manycore.pdf. The x86 alternative: many small cores, 16-wide vectors, coherent caches, graphics in software. Slide 27.
- **Feldman (HPCwire) (2009).** *Intel cancels 2010 Larrabee debut (December 7, 2009)*. https://www.hpcwire.com/2009/12/07/intel_cancels_2010_larrabee_debut/. Press, secondary source. Intel's December 4, 2009 statement that Larrabee would not ship as a discrete graphics product.
- **Feldman (HPCwire) (2012).** *Intel brings manycore x86 to market with Knights Corner (November 12, 2012)*. https://www.hpcwire.com/2012/11/12/intel_brings_manycore_x86_to_market_with_knights_corner/. Press, secondary source. The Xeon Phi 5110P: 60 cores, 1.011 TFLOP/s double, 8 GB, 320 GB/s, 225 W (the article misprints the core count).
- **Feldman (TOP500 News) (2016).** *Intel takes on NVIDIA with Knights Landing launch (June 2016)*. https://www.top500.org/news/intel-takes-on-nvidia-with-knights-landing-launch/. Secondary source. Knights Landing SKUs (up to 72 cores, about 3.46 TFLOP/s double) and the 16 GB of MCDRAM at 490 GB/s.
- **HPCwire (2018).** *End of the road for Knights Landing Phi (July 25, 2018)*. https://www.hpcwire.com/2018/07/25/end-of-the-road-for-knights-landing-phi. Press, secondary source. The discontinuation notice: last orders August 2018, last shipments July 2019.
- **AMD (2014).** *AMD unleashes world's most powerful server GPU for HPC (press release, August 6, 2014)*. https://www.globenewswire.com/news-release/2014/08/06/1011324/0/en/AMD-Unleashes-World-s-Most-Powerful-Server-GPU-for-HPC.html. Vendor press release. The FirePro S9150: 2.53 TFLOP/s double, 5.07 single, 16 GB, 320 GB/s, 235 W; GCN hardware ahead of NVIDIA's on paper.
- **AMD (2015).** *AMD launches 'Boltzmann Initiative' to dramatically reduce barriers to GPU computing on AMD FirePro graphics (press release, November 16, 2015)*. https://ir.amd.com/news-events/press-releases/detail/652/amd-launches-boltzmann-initiative-to-dramatically-reduce-barriers-to-gpu-computing-on-amd-fireprotm-graphics. Vendor press release. HCC, HIP and the headless Linux driver that became ROCm; the start of AMD's CUDA-compatible software path.

- **AMD (2016).** *AMD releases new version of ROCm, the most versatile open source platform for GPU computing (press release, November 14, 2016).* https://ir.amd.com/news-events/press-releases/detail/732/amd-releases-new-version-of-rocm-the-most-versatile-open-source-platform-for-gpu-computing. Vendor press release. ROCm 1.3 at SC16; ROCm 1.0 had shipped in April 2016.

10. Practice, textbooks and retrospectives (slides 28-31)

- **Halfhill (Microprocessor Report) (2009).** *Looking beyond graphics (Fermi analysis).* https://www.halfhill.com/mpr/2009/234001.pdf. Analyst report, open copy; secondary source. A contemporaneous independent reading of Fermi's design choices and their cost.
- **NVIDIA (2014).** *Kepler GK110/210 compute architecture whitepaper.* https://techpowerup.com/gpu-specs/docs/nvidia-gk110-210-compute-architecture.pdf. Vendor whitepaper (mirror). The GK210 revision for the K80: doubled register file and shared memory per SMX.
- **NVIDIA (2017).** *CUDA C programming guide (version 9.0).* https://docs.nvidia.com/cuda/archive/9.0/cuda-c-programming-guide/index.html. Vendor documentation. The compute-capability tables (registers, shared memory, warps per SM for every generation), the memory model, and the Volta-era additions (cooperative groups, WMMA, independent thread scheduling).
- **Hwu, Kirk and El Hajj (2022).** *Programming Massively Parallel Processors: A Hands-on Approach, 4th edition.* https://www.elsevier.com/books/programming-massively-parallel-processors/hwu/978-0-323-91231-0. Textbook. The current edition of the CUDA course, with Volta and later architectures; the performance chapters are the practice column of slide 29 at book length.
- **Hennessy and Patterson (2011).** *Computer Architecture: A Quantitative Approach, 5th edition, chapter 4 and appendix.* https://www.elsevier.com/books/computer-architecture/hennessy/978-0-12-383872-8. Textbook. The edition whose GPU section is built on Fermi, with the vector-processor vocabulary that makes SIMT precise.
- **Peddie (IEEE Computer Society, Chasing Pixels) (2020).** *Famous graphics chips: Geometry Engine.* https://www.computer.org/publications/tech-news/chasing-pixels/geometry-engine. Secondary source. A short illustrated history of the Geometry Engine for readers who want the 1982 story in plain language.

Index of terms

100x myth, 56
12 nm FFN, 45

achieved occupancy, 60
activations, 39
active mask, 23
AlexNet, 39
all-reduce, 51
arithmetic intensity, 31, 60
arithmetic-bound, 25
attainable performance, 31

bandwidth-bound, 31
bank conflict, 29
bisection bandwidth, 52
blending, 11
block, 18
board power, 7, 38
boost clock, 36, 60
branch efficiency, 60
broadcast (shared memory), 29
Brook for GPUs, 14
bytes in flight, 25
bytes per FLOP, 7, 61

cache coherence, 52
cache line, 29
Cg, 12
clock register, 50
Close To Metal, 16, 56
coalescing, 29
compulsory traffic, 60
compute capability, 21
compute die, 33
compute preemption, 41
compute-bound, 7, 31
concurrent kernels, 33
constant memory, 18
convolution as matrix multiply, 39
cooperative groups, 45
CORAL, 54
CoWoS, 41
Cray XK7 and XC50, 54
cuBLAS, 16, 31
CUDA, 9, 18
CUDA core, 5, 20
cuDNN, 39

dagger, 4
dark silicon, 27
data movement, 27
data-parallel training, 39, 52
datasheet, 60
Dennard scaling, 27
depth test, 11
DGX Station, 45
DGX-1, 42, 52
DGX-2, 52
DirectX, 12
dispatch unit, 43
divergence, 23
draw call, 13
dynamic parallelism, 35

ECC, 29, 33
EDR InfiniBand, 54
energy per operation, 27

Fermi, 33
fill rate, 11
FinFET, 9, 41
FirePro S9150, 56
fixed-function hardware, 11
FLOP/s, 7
FP16, 7, 39
FP32, 7
FP32 accumulation, 47
FP64, 7
FP64 unit, 33
fragment, 11, 47
framebuffer, 11
full-screen quad, 14
fused multiply-add, 27, 33

G80, 16
gather, 14
GDDR, 29, 38
GDDR3, 16
GDDR5X, 41
GeForce 256, 9
GEMM, 47
Geometry Engine, 9

GigaThread engine, 20
GK210, 35
global memory, 18
GLSL, 13
GPC, 41
GPGPU, 9, 14
GPU, 4
GPUDirect RDMA, 35
Graphics Core Next, 56
graphics pipeline, 11
grid, 18
GT200, 33

half2, 41
HBM2, 4, 27, 38
HBM2 stack height, 45
HIP, 56
HLSL, 13
host, 6
host-device model, 58
HPL, 34, 53
hybrid cube mesh, 51
Hyper-Q, 35

IEEE 754, 14
ImageNet, 39
independent thread scheduling, 45
instruction-level parallelism, 25
INT32 lane, 43
interposer, 27

Kepler, 35
kernel, 6, 18

L0 instruction cache, 43
L1 cache, 20
L2 cache, 29, 33
lane, 5, 23
Larrabee, 56
latency, 5
latency hiding, 24
launch configuration, 18
Little's law, 24
load imbalance, 13
load/store unit, 20, 43
local memory, 18
loss scaling, 47

master weights, 47
matrix engine, 61
matrix multiply-add, 47
Maxwell, 35
MCDRAM, 56
memory bandwidth, 7
memory bus width, 29
memory divergence, 23
memory latency, 25
memory partition, 16
memory transaction, 29
memory-bound, 25
microbenchmark, 49
MIMD, 22
mixed-precision training, 39
mma sync, 47

NCCL, 52
NVLink, 9, 41, 51
NVLink 2.0, 45
nvprof and Nsight Compute, 60
NVSwitch, 45, 52

occupancy, 21
occupancy calculator, 21
OpenACC, 54
OpenCL, 9, 19
out-of-order execution, 25

Pascal, 41
pass, 14
PCI Express, 51
peak, 7
performance per watt, 27, 36, 56
performance-model compatibility, 58
picojoule, 27
pixel shader, 12
Piz Daint, 54
pointer chasing, 49
power wall, 27
POWER9, 54
predication, 23
process node, 37
processing block, 43
program counter, 49
programmable shader, 9
PTX, 58

Radeon Instinct MI25, 56
raster operations unit, 12

rasterization, 11
RealityEngine, 9
reconvergence, 49
reconvergence point, 23
register bank conflict, 50
register file, 16, 20
regular work, 61
render-to-texture, 14
resident thread, 6
reticle limit, 37
ridge point, 31, 47
ring all-reduce, 52
Rmax and Rpeak, 53
ROCm, 56
roofline, 6
roofline model, 31
root complex, 51

SASS, 58
SAXPY, 18
scalar ISA, 16
scatter, 14
scoreboard, 25
sector, 29
seven-point stencil, 60
SGEMM, 31
Shader Model 3.0, 12
shared memory, 16, 18
Sierra, 54
significand, 47
silicon interposer, 41
SIMD, 13, 22
SIMT, 6, 22
SMM, 36
SMX, 35
source compatibility, 58
special function unit, 20
special-function unit, 43
specialization, 58
starvation-free, 49
stencil, 31
stream, 14
streaming multiprocessor, 4, 20
streaming processor, 16
structure of arrays, 29
subnormal number, 47
Summit, 53
SXM2, 38
syncthreads, 18
syncwarp, 23, 49

tensor core, 7, 45
Tesla, 9
Tesla (product line), 33
Tesla C870, 16
Tesla K80, 39
Tesla M40, 36
texture, 11
texture cache, 29
texture fetch, 14
texture memory, 18
texture unit, 43
texture/processor cluster, 16
thread (CUDA), 18
thread-level parallelism, 25
through-silicon via, 41
throughput, 5
throughput processor, 4
Tianhe-1A, 54
tiling, 31
Titan, 54
Titan (supercomputer), 36
TLB, 25, 50
TOP500, 9, 34, 53
transform and lighting, 11

unified address space, 33
unified L1 and shared memory, 43
unified memory, 18, 41
unified shader architecture, 9
unified virtual addressing, 58

vertex, 11
vertex shader, 12
Volta, 4, 45
Voodoo, 9

warp, 5, 20
warp scheduler, 20
warp shuffle, 49
warp stall reasons, 60
warp width, 61
warp-synchronous programming, 23, 49
wavefront, 56
WMMA, 45
work-item, 16

Xenos, 16
Xeon Phi, 56
yield harvesting, 37